Design Document

for

Travelly

Version 1.0

Prepared by

Group #17 **Group Name: 10Bit**

Ayush Jha	**230258**	**ayushj23@iitk.ac.in**
Khushi Khandelwal	**230561**	**khushik23@iitk.ac.in**
Pranjali Singh	**220796**	**pranjalis22@iitk.ac.in**
Mohd Sarim Ali	**230657**	**sarimali23@iitk.ac.in**
Raghav Madan	**220853**	**raghavm22@iitk.ac.in**
Rigved Koli	**230855**	**brigved23@iitk.ac.in**
Ronit Kumar	**230875**	**ronitk23@iitk.ac.in**
Rushabh Pandya	**230732**	**rushabhp23@iitk.ac.in**
Sanket Agarwal	**230920**	**sanketag23@iitk.ac.in**
Tanmay Roy	**231082**	**tanmayroy23@iitk.ac.in**

Course: CS253

Mentor TA: *V Arvind*

Date: 06/02/26

Contents

Contents **2**

Revisions **3**

1 Context Design **4**

1.1 Context model 4
1.2 Human Interface Design 6

2 Architecture design **24**

2.1 Architectural Style 24
2.2 Architecture Diagram 24
2.3 Component Description 26
2.4 Mapping to Non-Functional Requirements 27

3 object-oriented design **29**

3.1 Use case diagram 29
3.2 Class details 42
3.3 Class Diagram 56
3.4 Sequence diagram 64
3.5 State diagram 72

4 Project Plan **82**

Appendix A - Group Log **83**

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Version 1.0	Ayush Jha Khushi Khandelwal Pranjali Singh Mohd Sarim Ali Raghav Madan Rigved Koli Ronit Kumar Rushabh Pandya Sanket Agarwal Tanmay Roy	First Version of Software Design Document	05/02/26

1. Context Design

1.1 Context Model

The Travelly system works seamlessly with its various interconnected subsystems to deliver a smooth and collaborative travel planning and expense managing experience for users. These subsystems interact with each other and the backend server to manage authentication, trip coordination, expense handling, media sharing, and route planning. Typically, using one subsystem naturally leads to the use of others—for example, creating a trip group enables expense tracking, document sharing, and route optimization.

The model and its subsystems are as follows:

- **User Authentication System:**
  Authenticates users through email/password, Google Sign-In, or OTP-based verification. It manages registration, login, session handling, and password recovery to ensure secure access.

- **User Profile Management System:**
  Maintains personal user information such as name, email, phone number, UPI ID, preferences, and profile photo, allowing users to update their details and manage account settings.

- **Trip Group Creation and Management System:**
  Allows users to create trips, generate invite links, add members, and manage group settings such as default currency and debt simplification. Members can join trips and access shared data.

- **Expense Tracking and Splitting System:**
  Enables users to record shared expenses with equal or custom splits. The system automatically updates balances and maintains transaction history for transparency among group members.

- **Balance Computation and Settlement System:**
  Computes net balances for each participant and generates reimbursement suggestions. Users can settle dues via UPI integration or manual confirmation, ensuring accurate financial reconciliation.

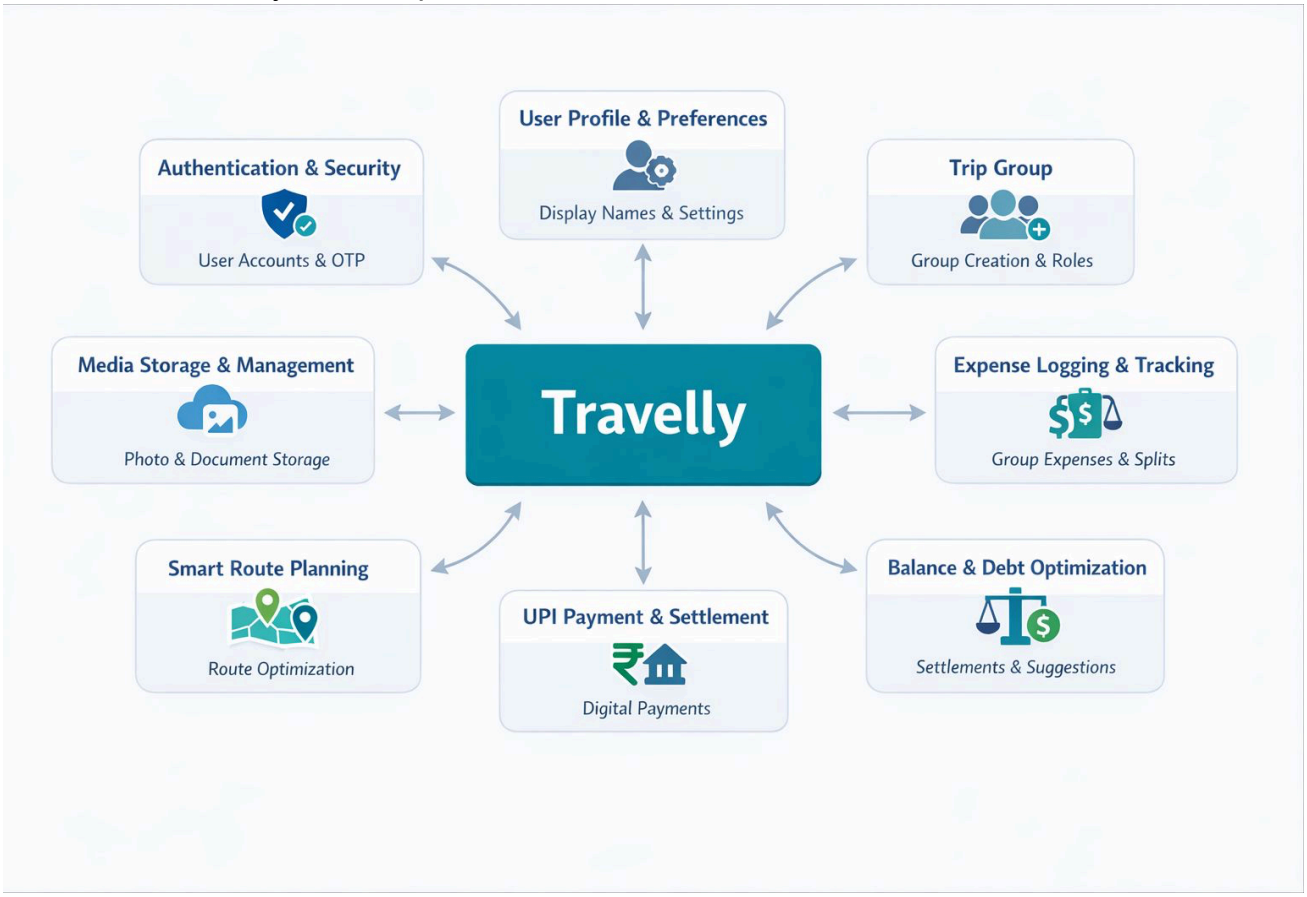
- **Payments Integration System:**
  Connects with external UPI applications to facilitate seamless payment redirection and confirmation, reducing friction in settling group expenses.

- **Media and Document Sharing System:**
  Allows group members to upload, view, download, and manage photos and documents related to a trip, providing central storage for travel memories and important files.

- **Notification System:**
  Sends alerts for settlements, payment reminders, group updates, and security events, keeping users informed of important actions and changes.

- **Route Planning and Optimization System:**
  Uses external map services to generate optimized travel routes based on selected locations, helping users efficiently plan their itinerary.

Together, these subsystems form an integrated ecosystem that supports collaborative trip planning, expense management, and smart navigation, enabling Travelly to provide a unified, user-friendly travel experience.



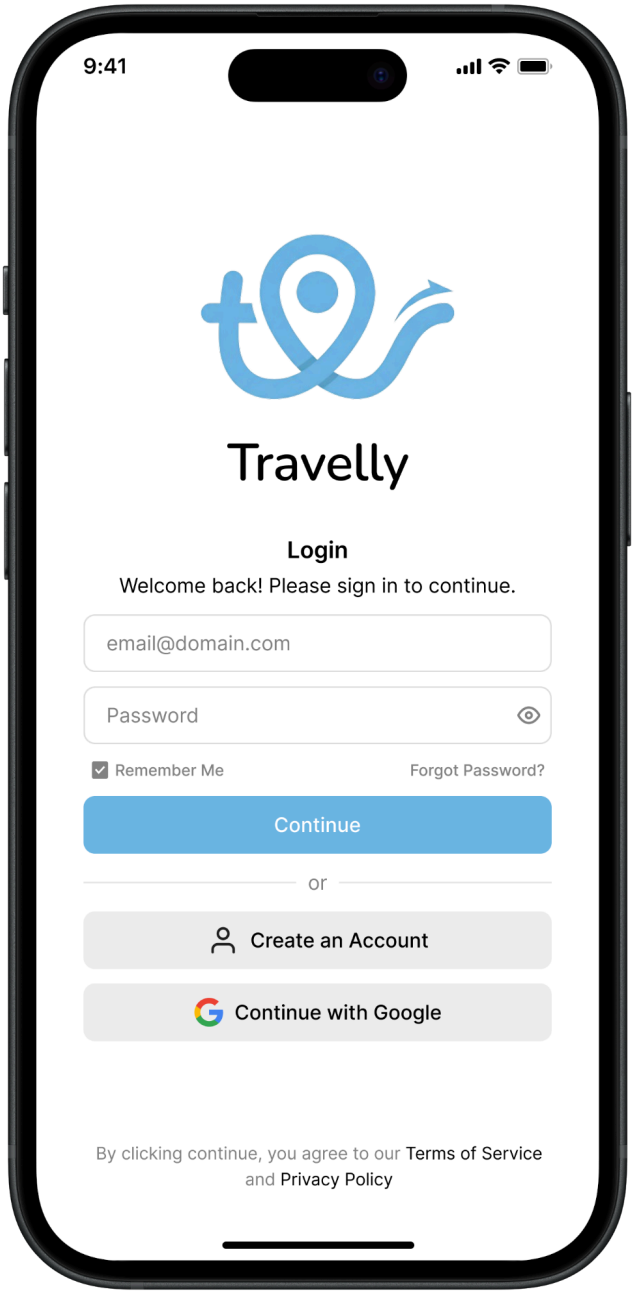
1.2 Human Interface Design

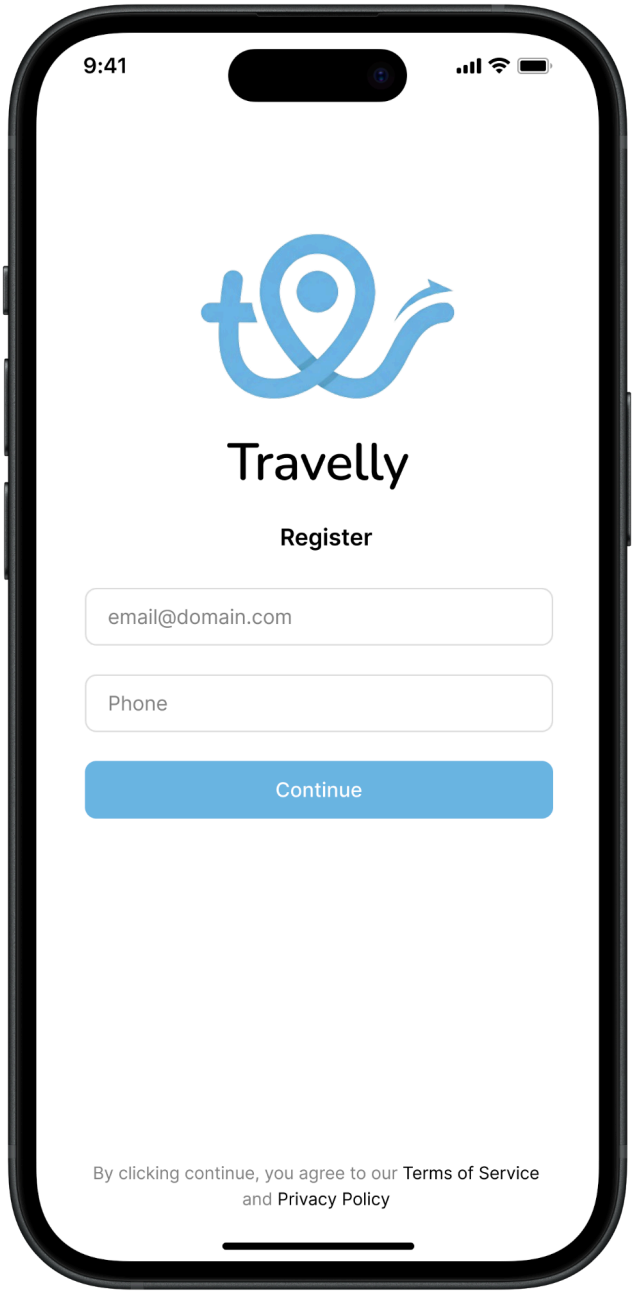
This segment presents preliminary interface designs intended to serve as a comprehensive blueprint for system designers and developers. By detailing the software's architecture and interaction flows, these documents delineate core functionalities, screen compositions, and user controls. The primary objective is to facilitate an intuitive and seamless user experience. Adhering to these visual guides ensures that the development process remains cohesive and that the final product maintains structural consistency.

Travelly

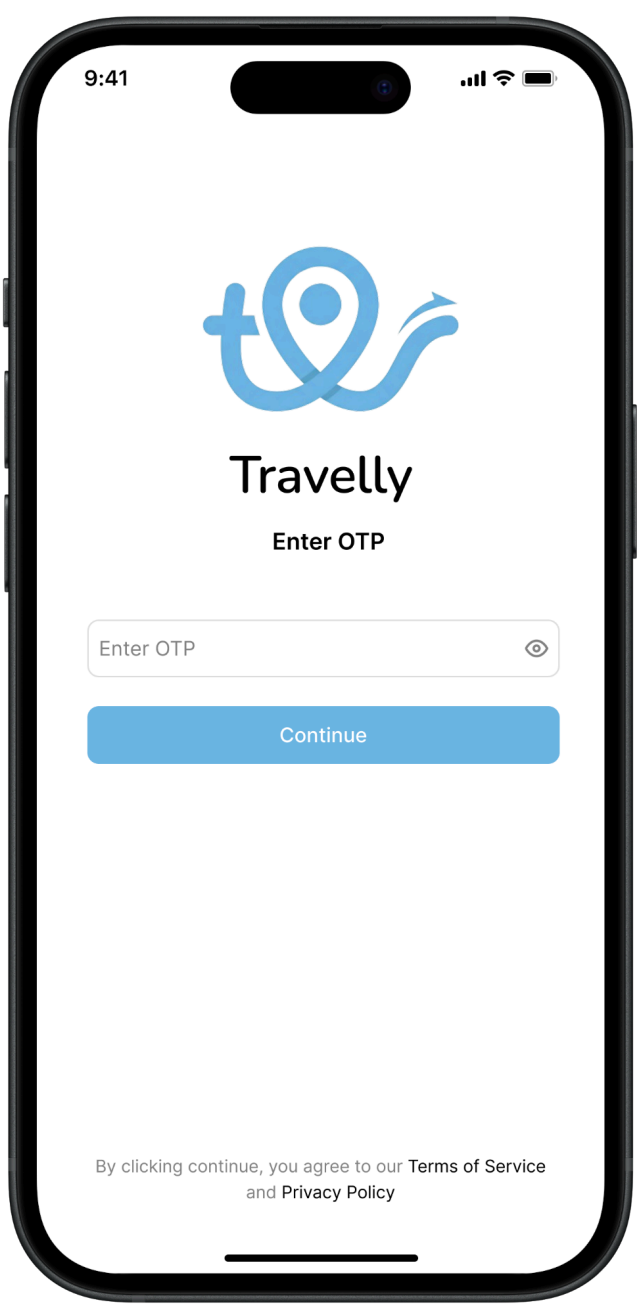
One Trip, One Place, Zero Stress

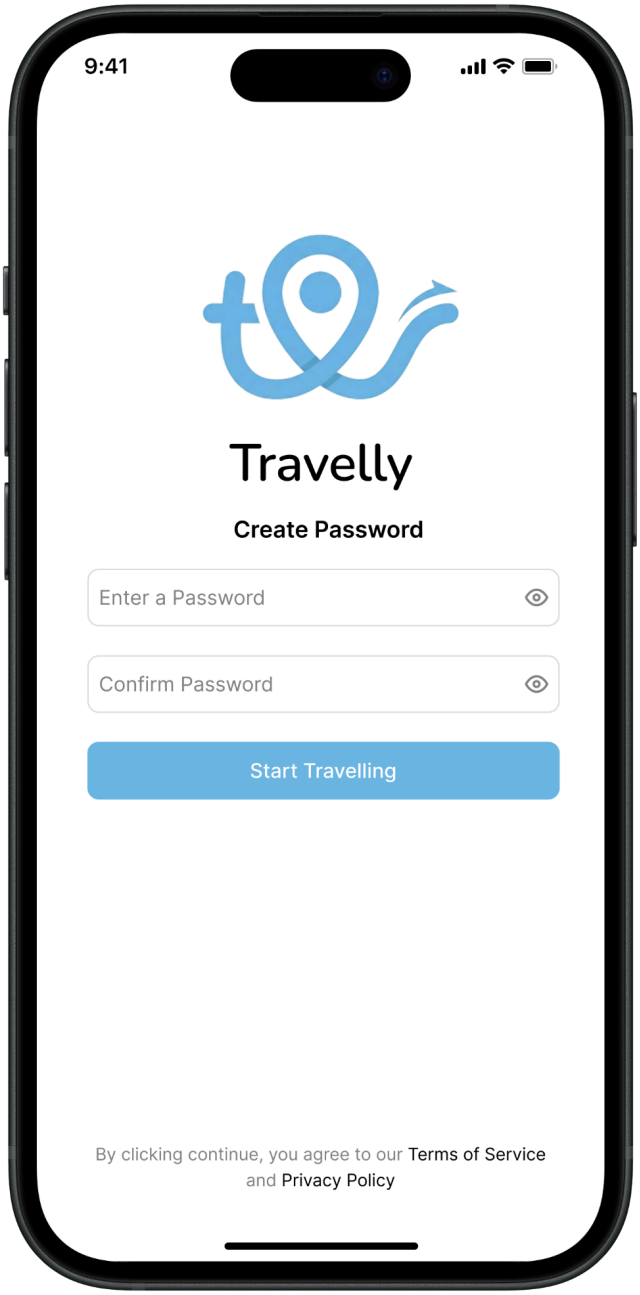
1.2.1 Authentication Designs





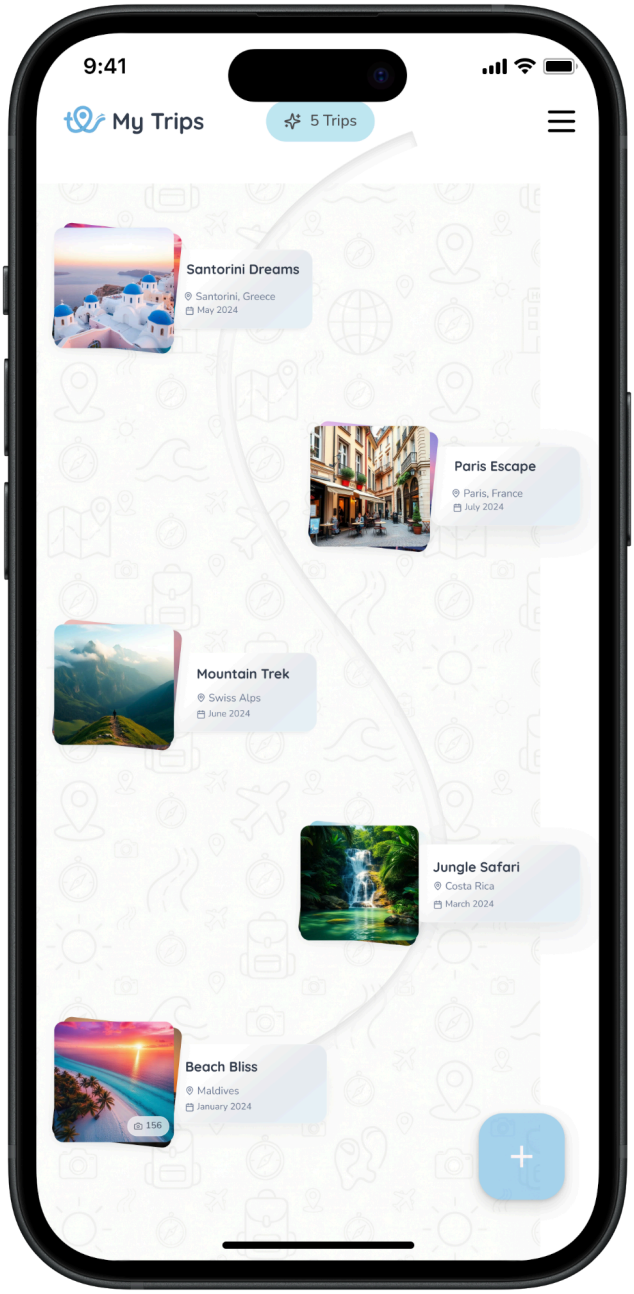
Users can Login using their Email ID or Google Account and New Users can Register

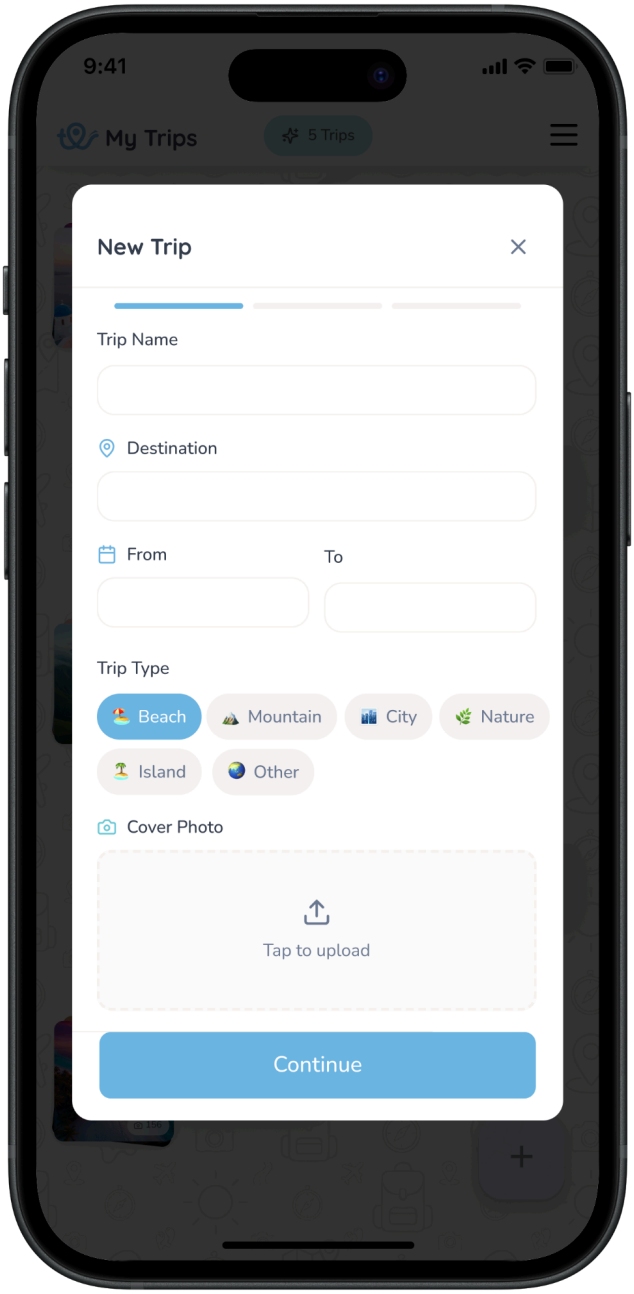




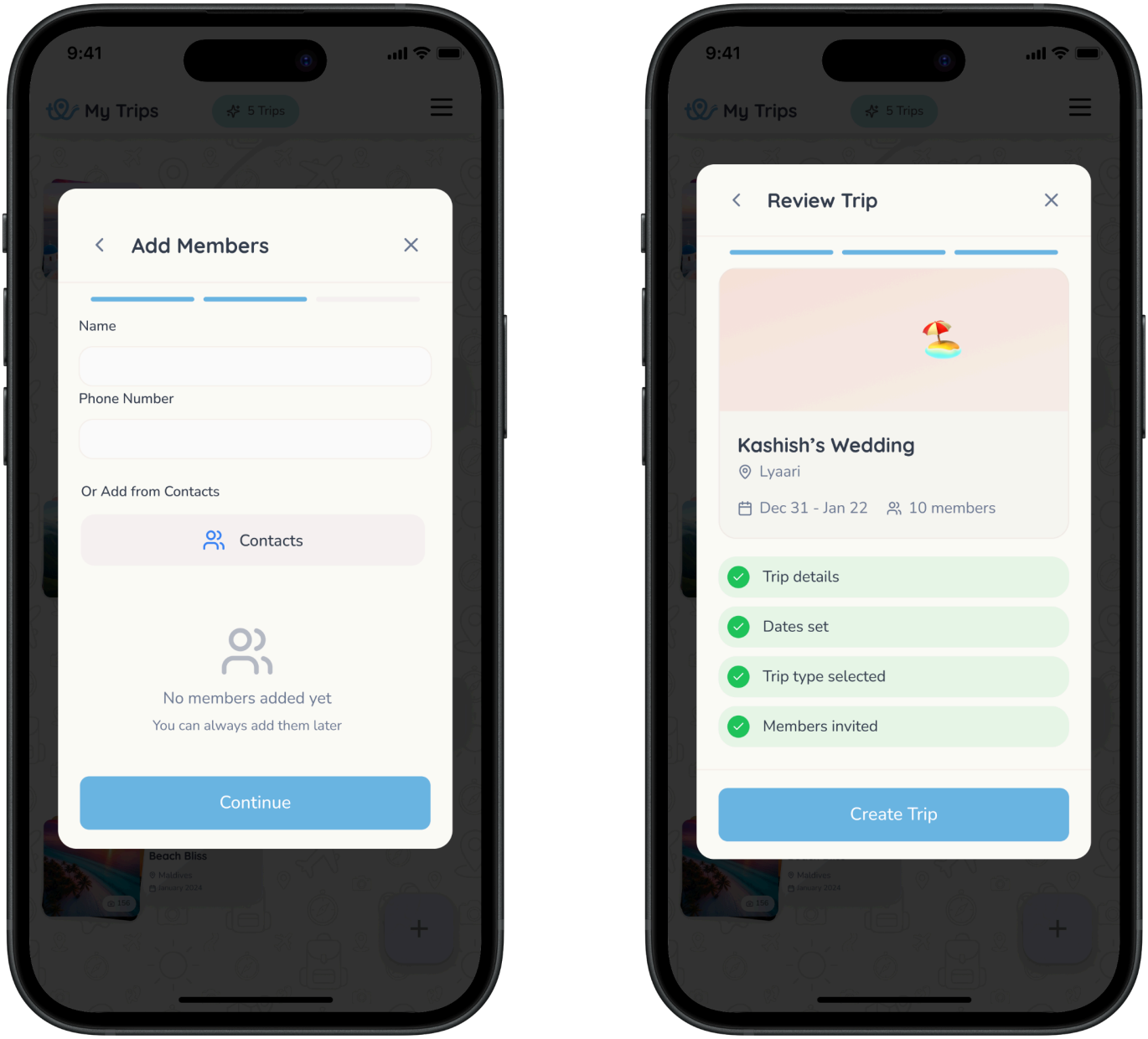
Users need to verify using OTP, and Create a strong Password

1.2.2 Trip Creation And Management Designs



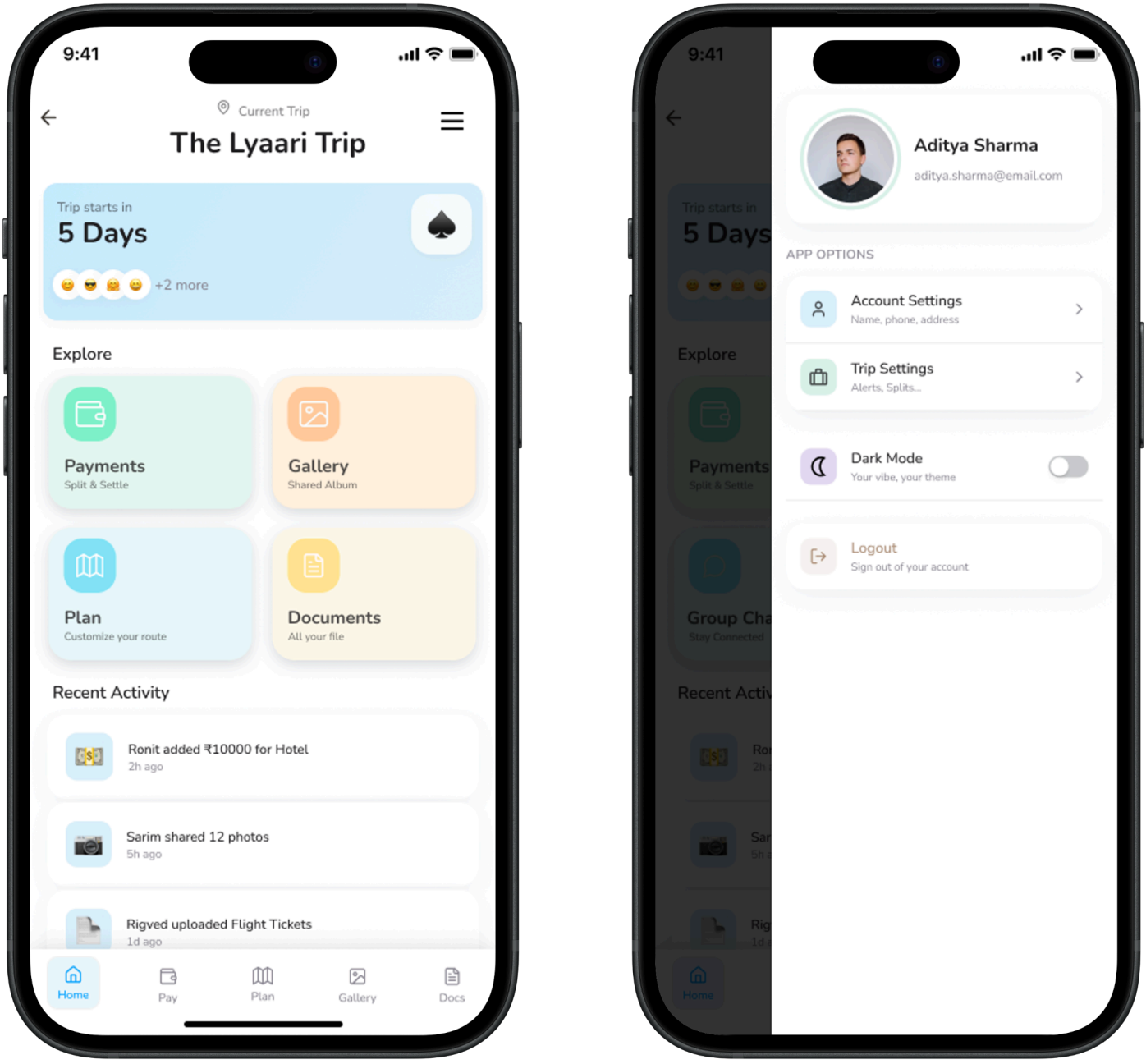


Users can view existing trips and create new trips by entering relevant trip details



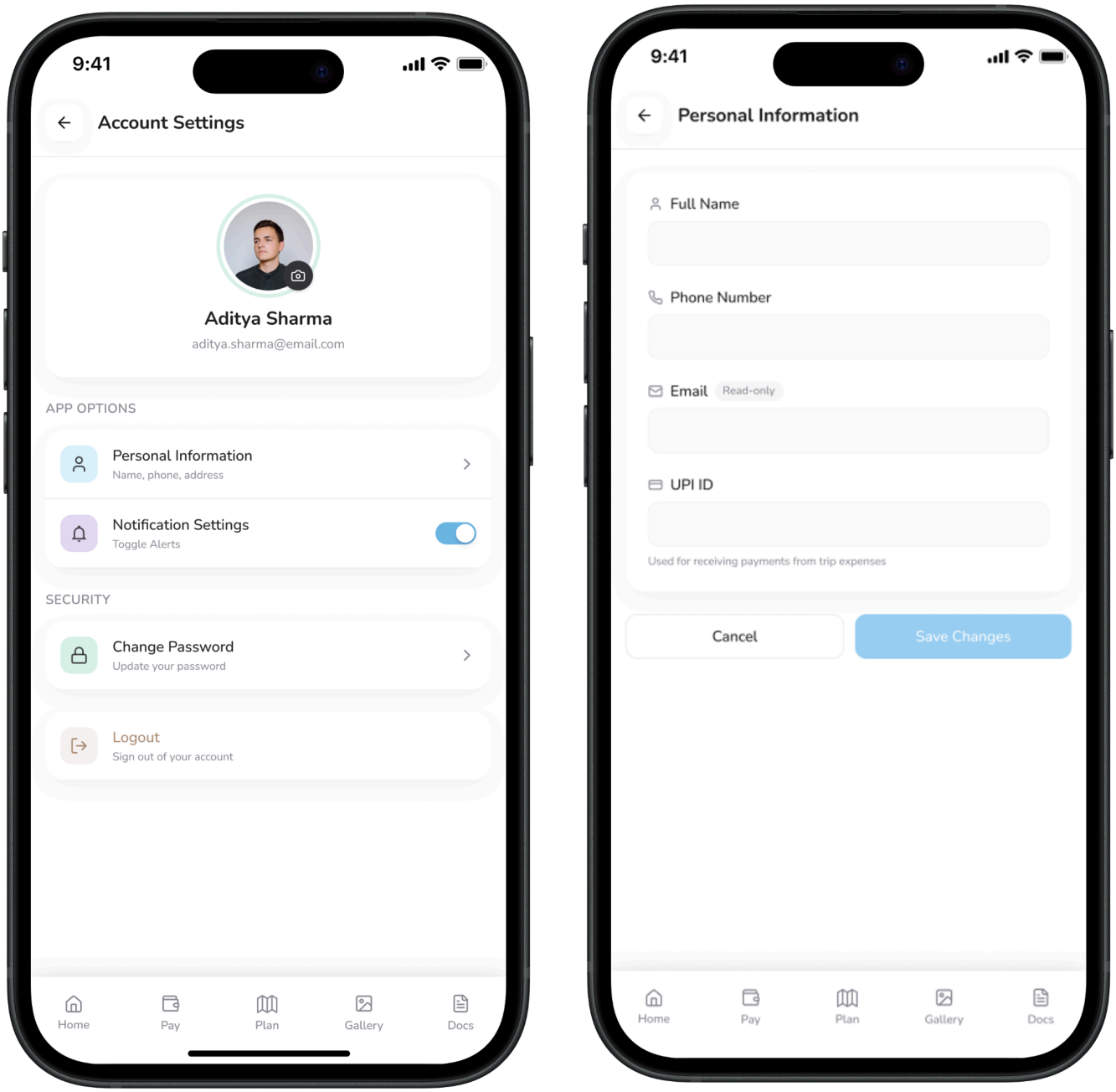
A minimal bottom-sheet flow to add members, review trip details, and create the trip.

1.2.3 Trip Dashboard And Sidebar Designs

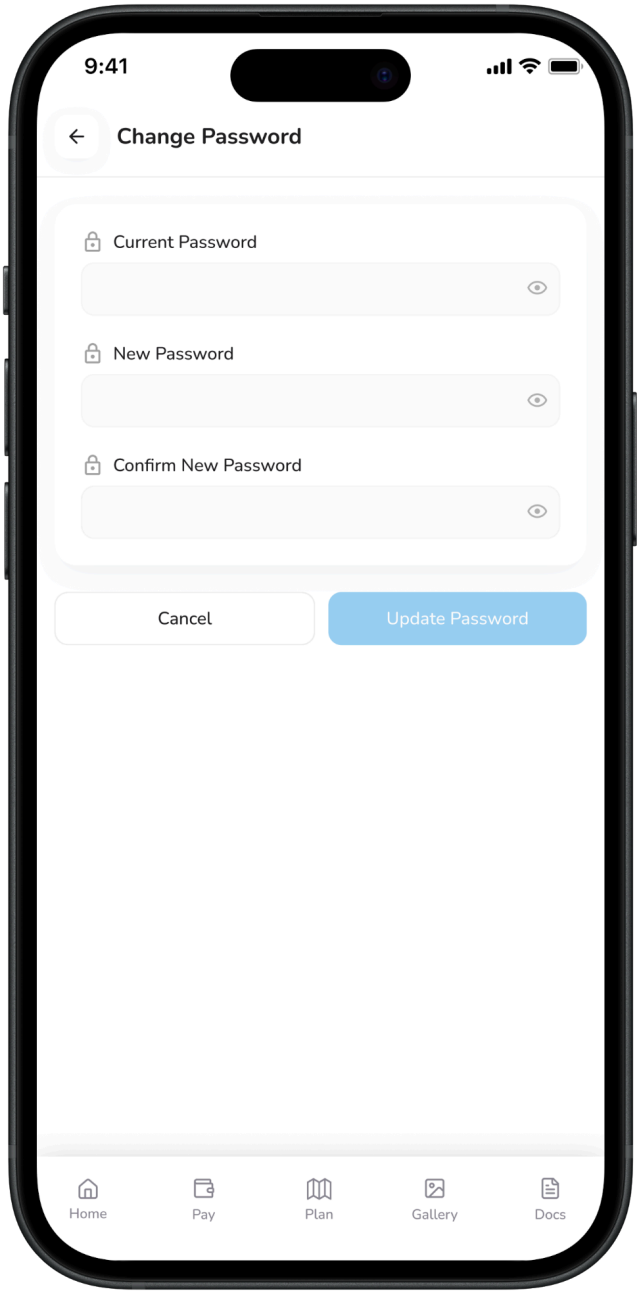


A trip dashboard with feature cards and a slide-out profile menu.

1.2.4 Account Settings

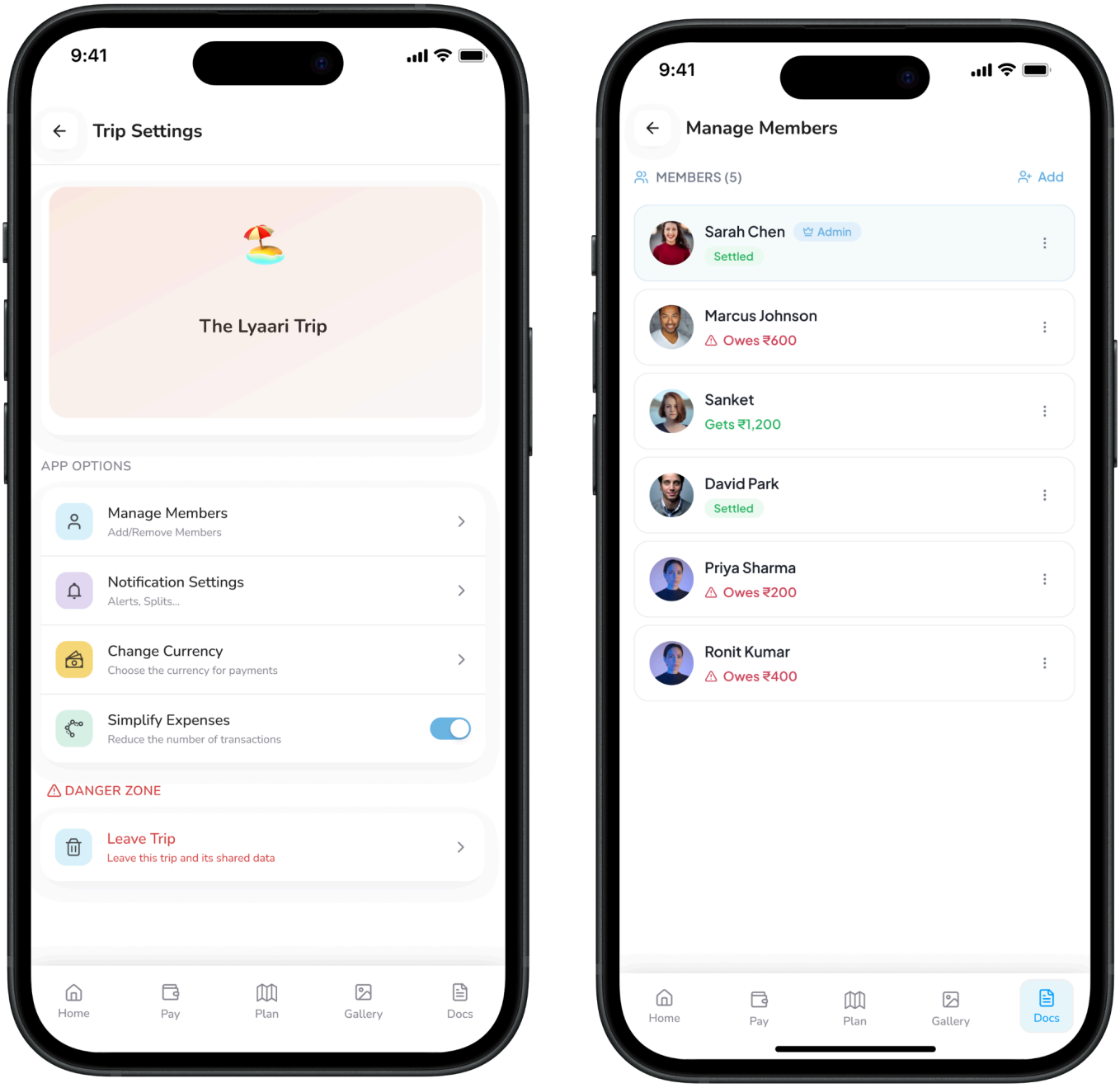


A trip dashboard with feature cards and a slide-out profile menu.

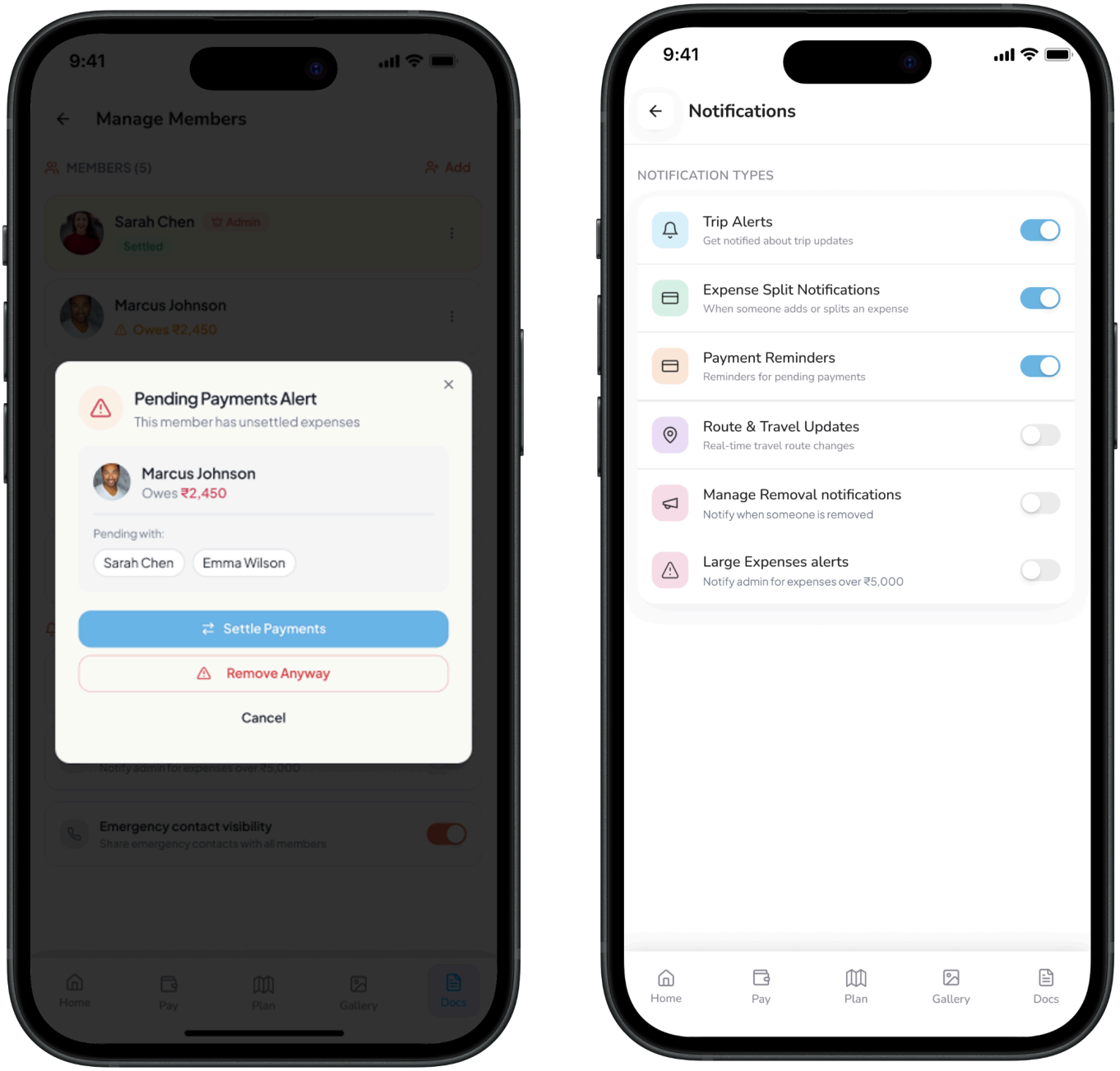


Password Change Interface

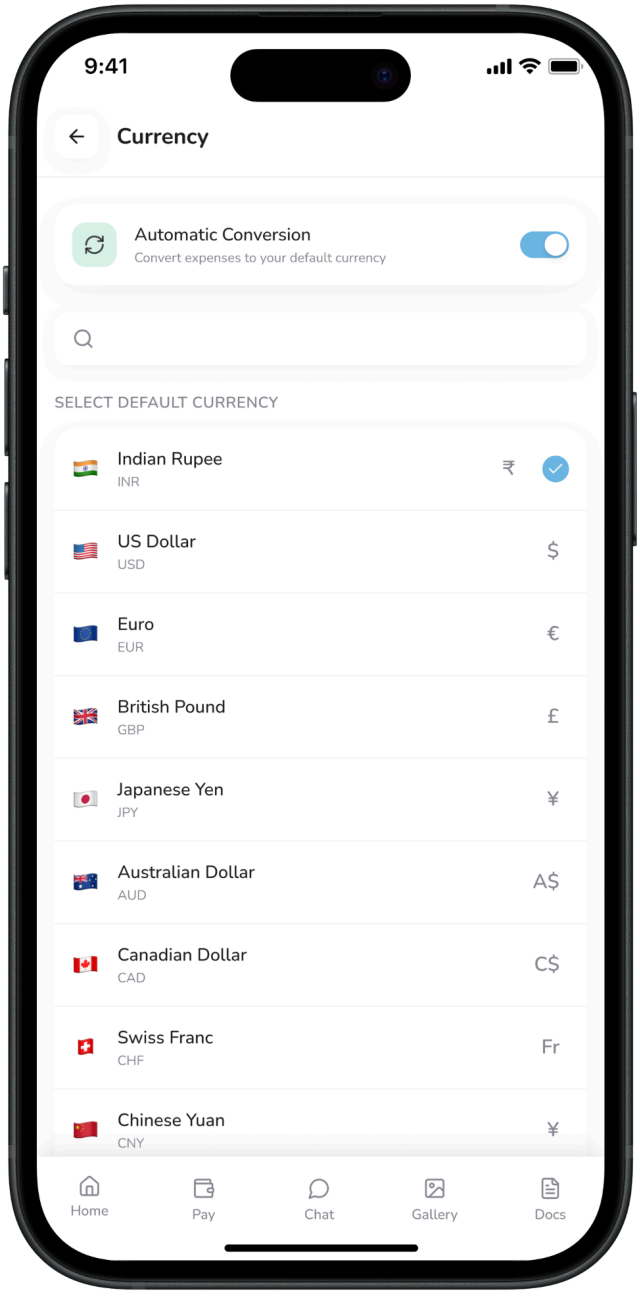
1.2.5 Trip Settings



Users can view trip settings and manage members, notifications, currency, and expense-related options

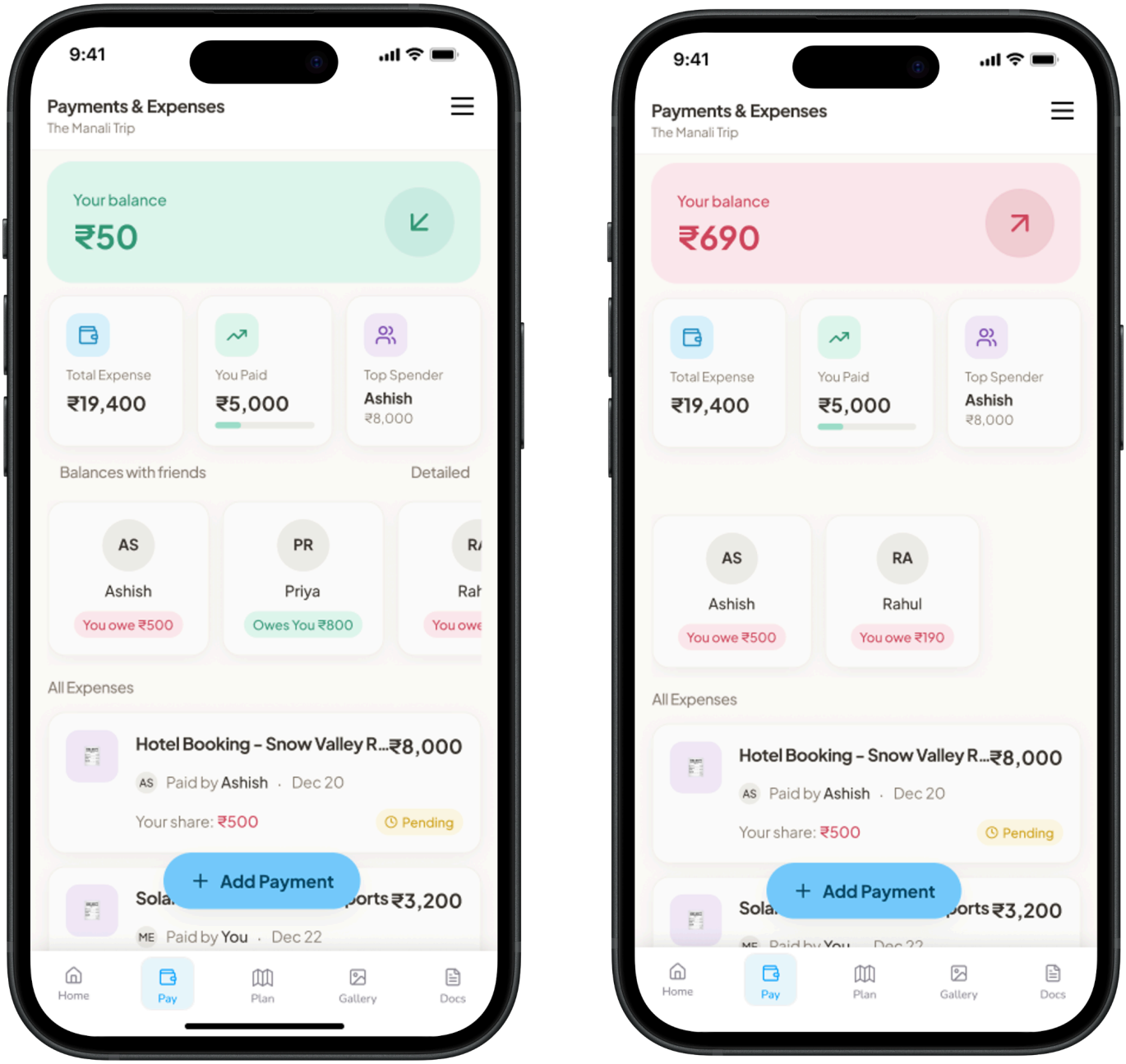


Pending Payments alerts and notification settings screen

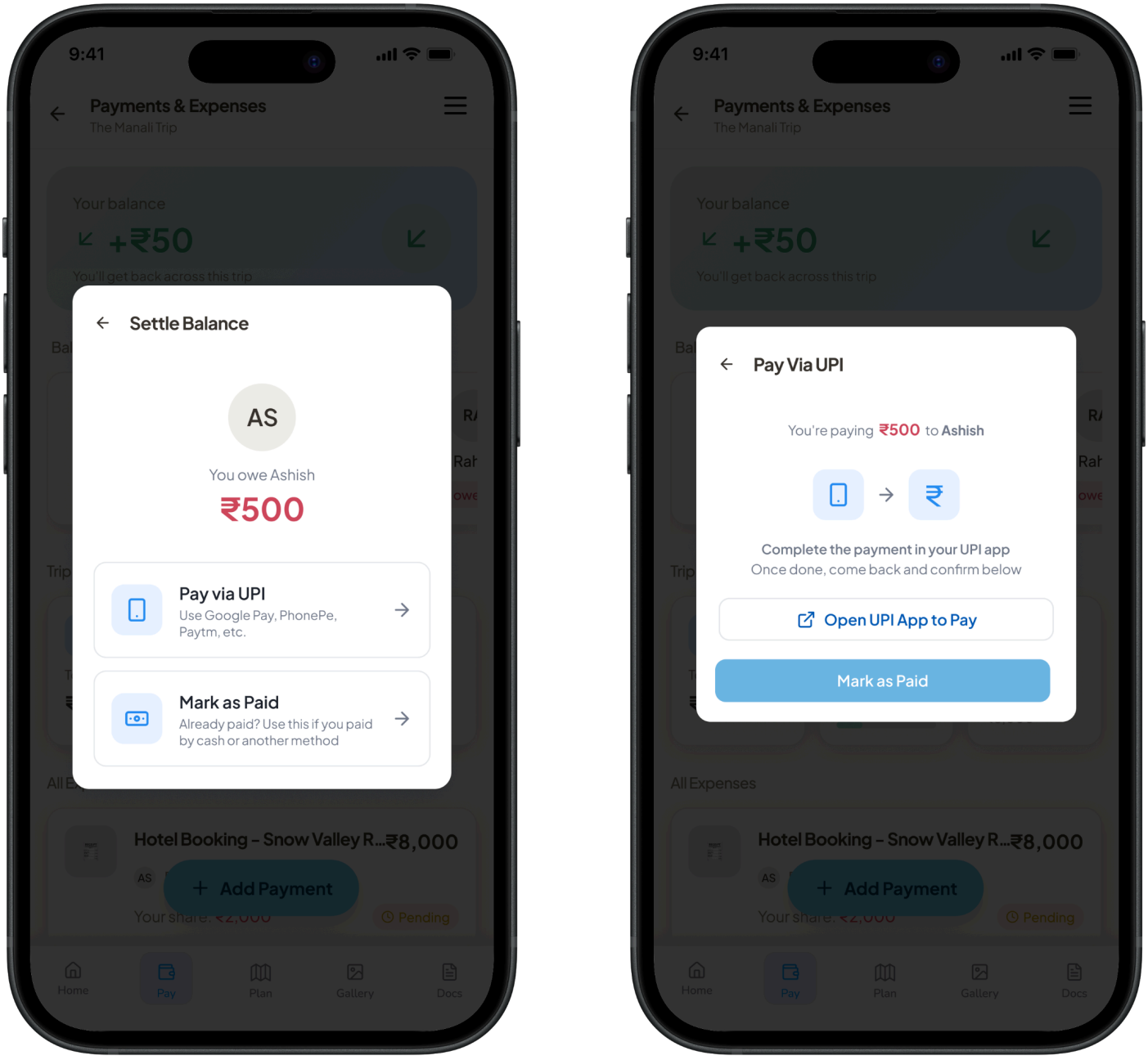


Select between different currencies

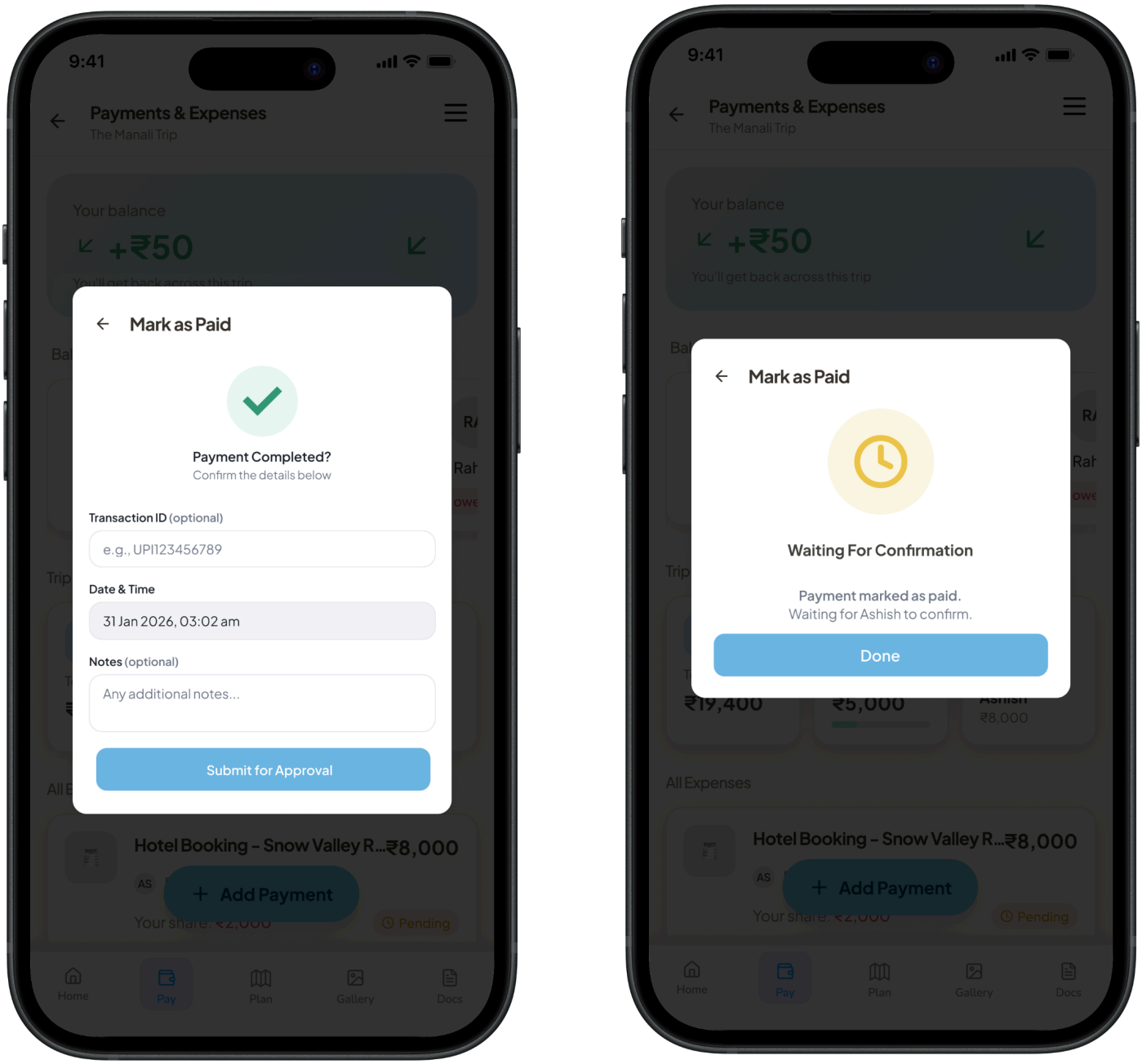
1.2.6 Payments And Expenses Dashboard



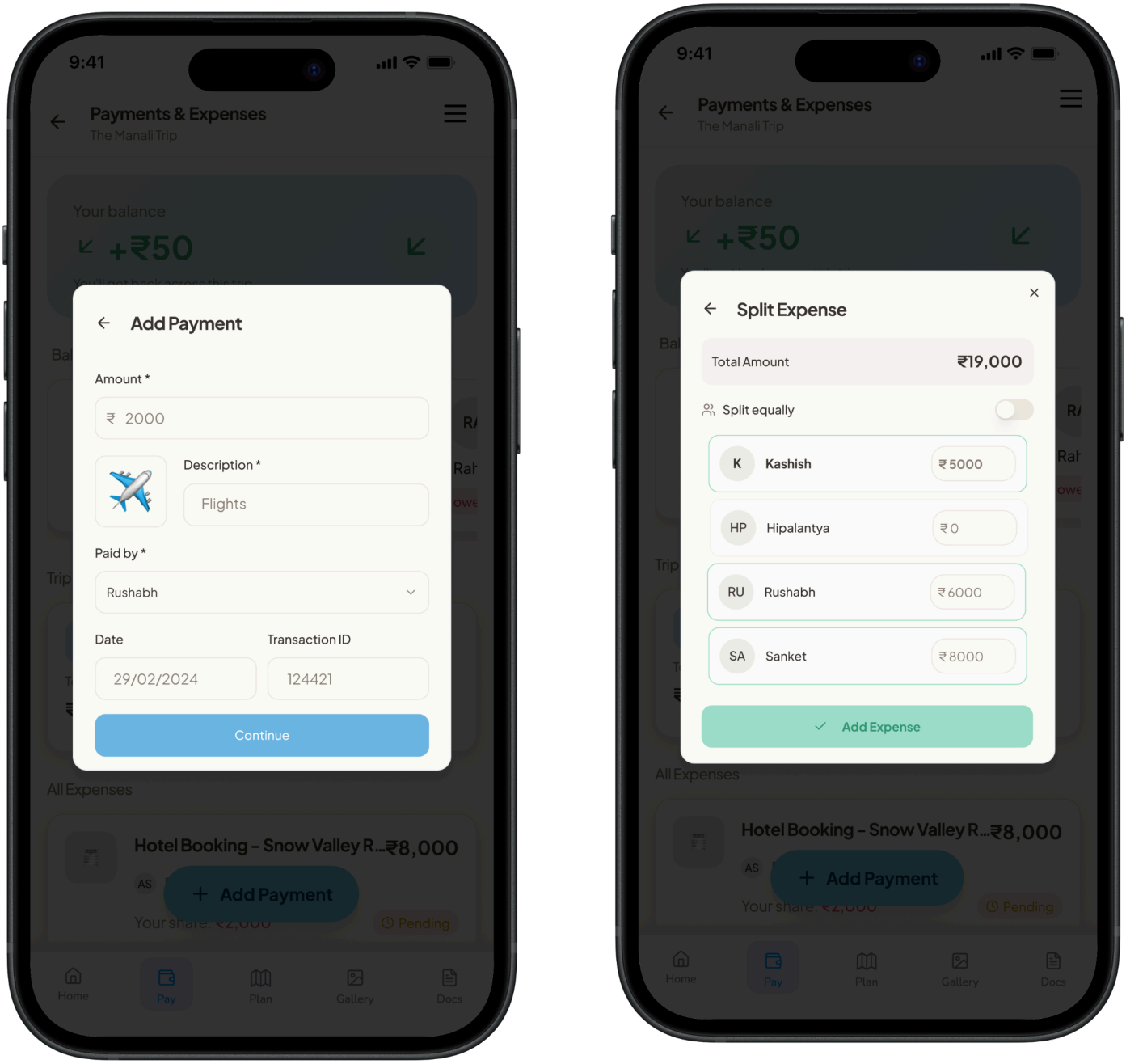
Payments dashboard showing balances, expenses, and an add payment action.



Settle balance flow with UPI payment redirect and mark-as-paid options.



Payment Confirmation Screens

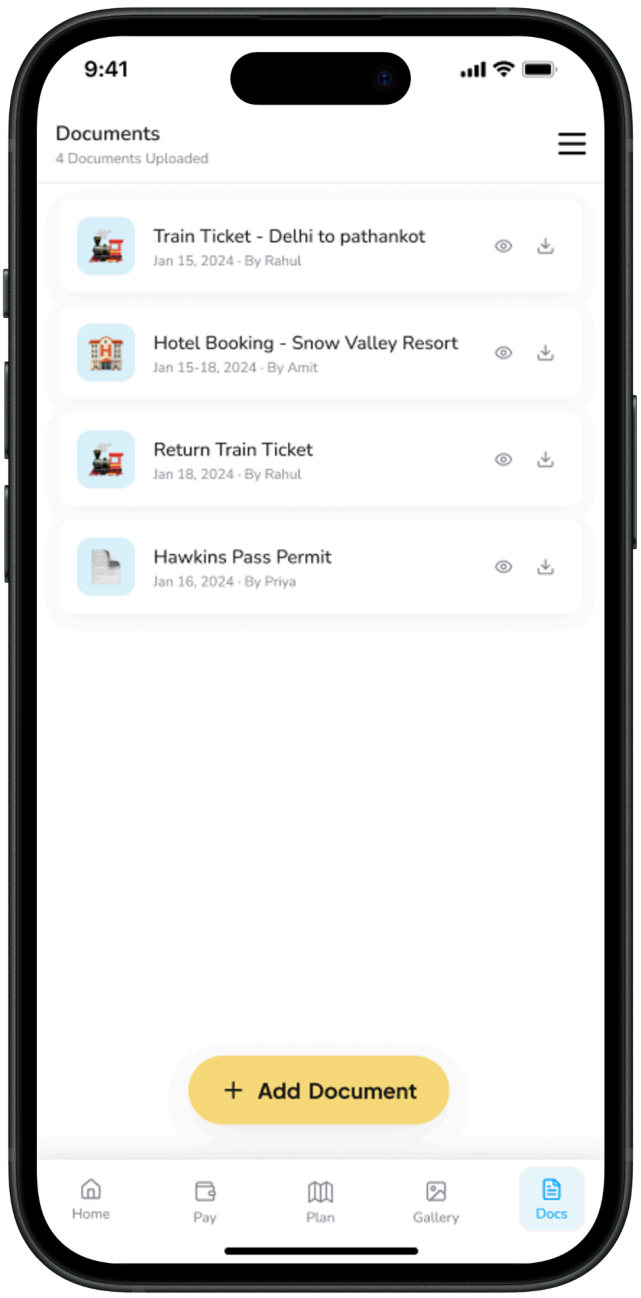


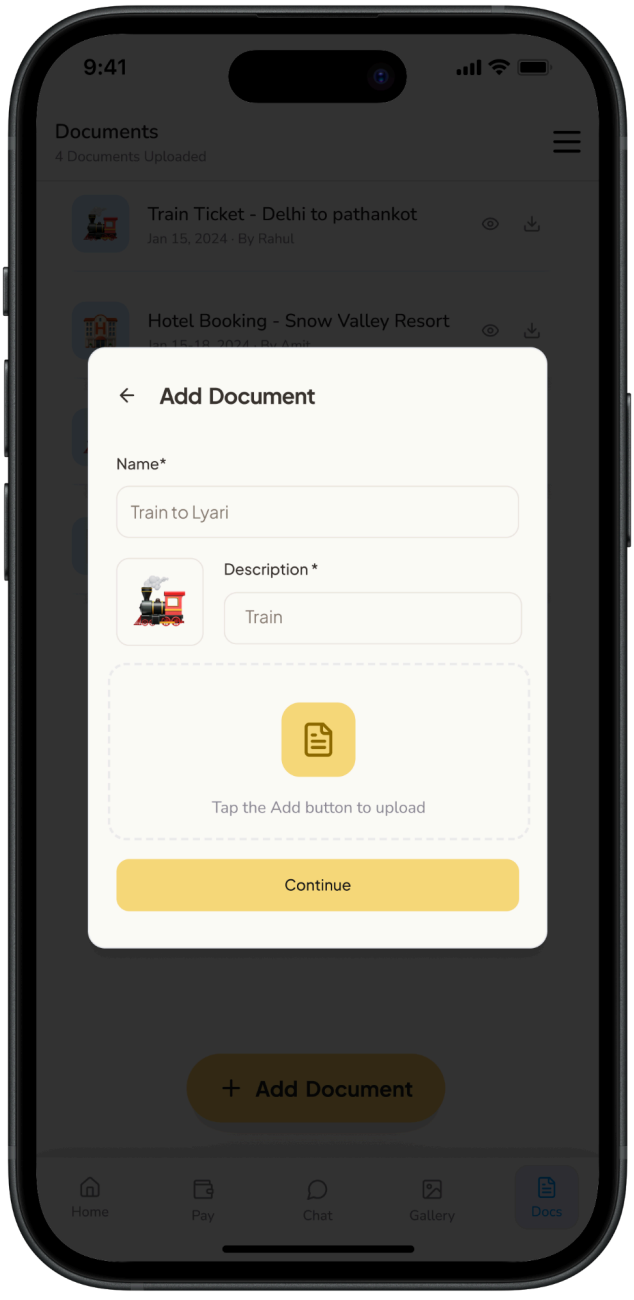
A payments screen with add payment and split expense option between users.

1.2.7 Gallery

A photo grid gallery with an add media button

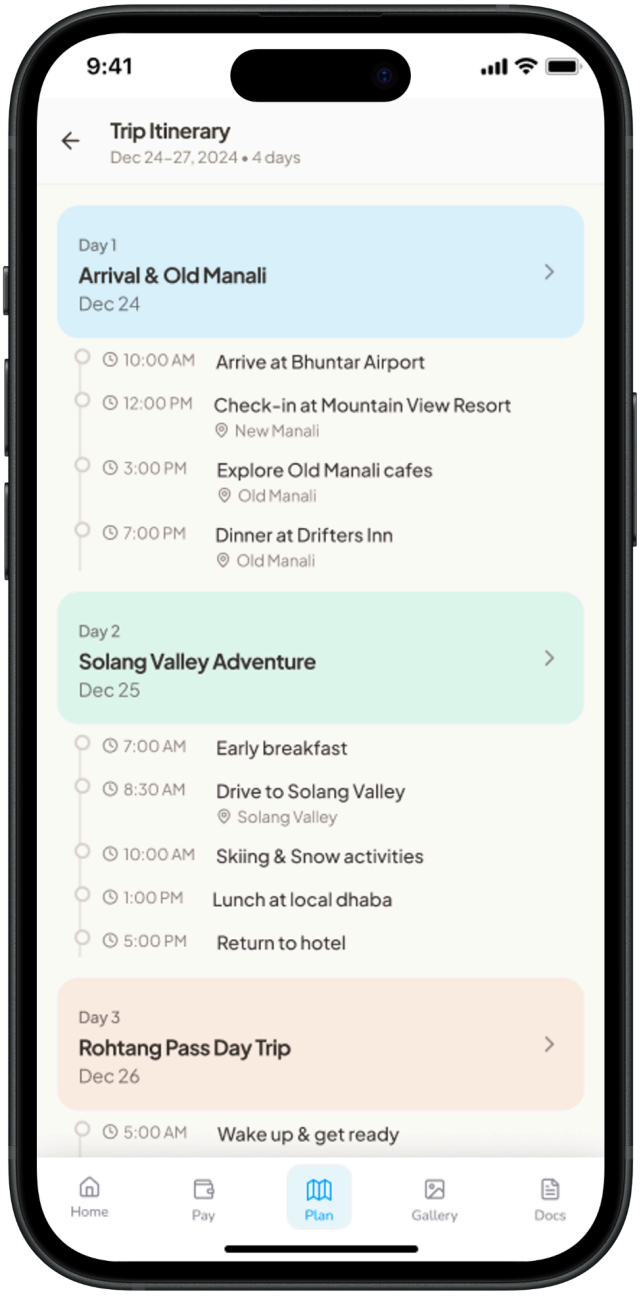
1.2.8 Documents





A documents list screen with an add-document bottom sheet for uploading files.

1.2.9 Group Chat And Trip Itinerary Planner



A Trip route planner screen

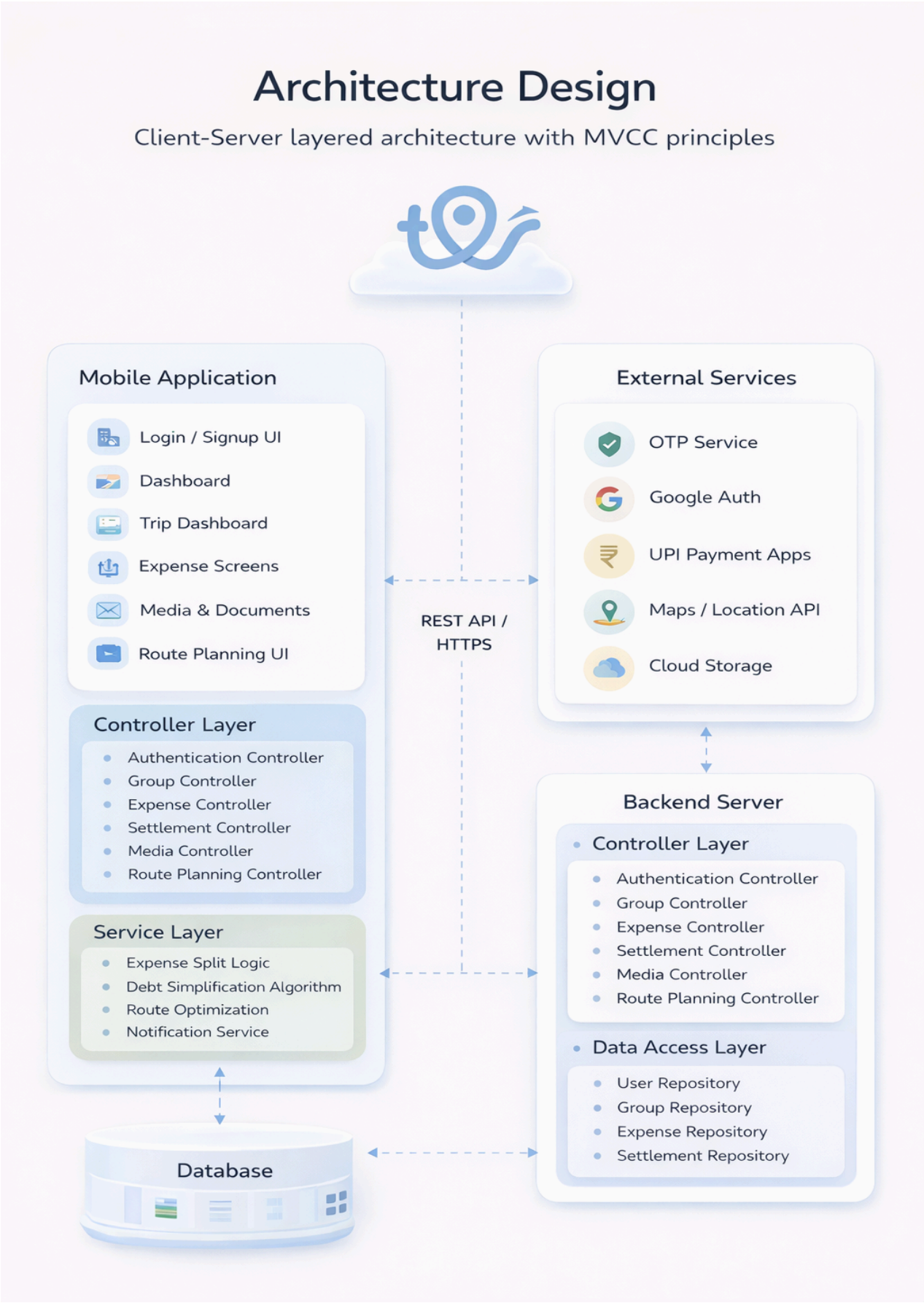
2. Architectural Design

2.1 Architectural Style

We will use a **Client–Server architecture with layered MVC principles**.This is appropriate because:

- The mobile app handles UI and interaction
- Backend handles business logic and algorithms
- Database handles persistence
- External services are integrated through APIs

2.2 Architecture Diagram


Architecture Design
Client-Server layered architecture with MVCC principles
Mobile Application
Login / Signup UI
Dashboard
Trip Dashboard
Expense Screens
Media & Documents
Route Planning UI
Controller Layer
Authentication Controller
Group Controller
Expense Controller
Settlement Controller
Media Controller
Route Planning Controller
Service Layer
Expense Split Logic
Debt Simplification Algorithm
Route Optimization
Notification Service
Database
REST API / HTTPS
External Services
OTP Service
Google Auth
UPI Payment Apps
Maps / Location API
Cloud Storage
Backend Server
Controller Layer
Authentication Controller
Group Controller
Expense Controller
Settlement Controller
Media Controller
Route Planning Controller
Data Access Layer
User Repository
Group Repository
Expense Repository
Settlement Repository

2.3 Component Description

2.3.1 Mobile Application Layer

Responsible for:

- Rendering UI
- Sending requests
- Handling offline cache
- User interaction

This layer follows MVC internally:

- View → UI screens
- Controller → API calls and state management
- Model → local storage/session

2.3.2 Backend Server

Handles:

- Business logic
- Expense calculations
- Debt simplification
- Route planning
- Authentication validation

Divided into layers:

Controller Layer:
Handles incoming API requests.

Service Layer:
Contains core algorithms and logic.

Data Access Layer:
Communicates with the database.

2.3.3 Database

Stores persistent data:

- Users
- Groups
- Expenses
- Settlements
- Media metadata

Ensures atomic transactions and consistency.

2.3.4 External Services

Travelly depends on:

- OTP provider
- Google authentication
- UPI payment apps
- Maps API
- Cloud storage

These are accessed via secure APIs.

2.4 Mapping to Non-Functional Requirements

The proposed architecture for Travelly follows a **client–server layered architecture with MVC principles**, separating the system into presentation, business logic, and data storage layers. This separation helps the system meet the non-functional requirements defined in the SRS.

2.4.1 Performance

Performance is improved by distributing tasks across layers. The mobile application handles UI rendering and local caching, while the backend processes computations such as expense calculations and route optimization. Efficient API design and database access help maintain acceptable response times.

2.4.2 Scalability

The modular architecture allows independent scaling of the backend, database, and storage services. As the number of users, groups, and media files increases, system resources can be expanded without major redesign.

2.4.3 Reliability

Centralized business logic and transactional database operations ensure consistent and accurate computation of expenses, balances, and settlements. Error handling mechanisms allow the system to recover from temporary failures.

2.4.4 Availability

Persistent storage, periodic backups, and separation of system components help maintain availability. Limited offline functionality through local caching allows users to access previously synchronized data when network connectivity is unavailable.

2.4.5 Security

Secure communication protocols, OTP-based authentication, and controlled access to group data ensure system security. Payment processing is performed through UPI redirection, reducing exposure to sensitive financial credentials.

2.4.6 Maintainability

The layered design separates UI, business logic, and data management, making the system easier to maintain and extend. Updates or feature additions can be implemented with minimal impact on other components.

2.4.7 Usability

The architecture supports a responsive and intuitive user interface, enabling users to perform common tasks such as adding expenses and settling balances efficiently.

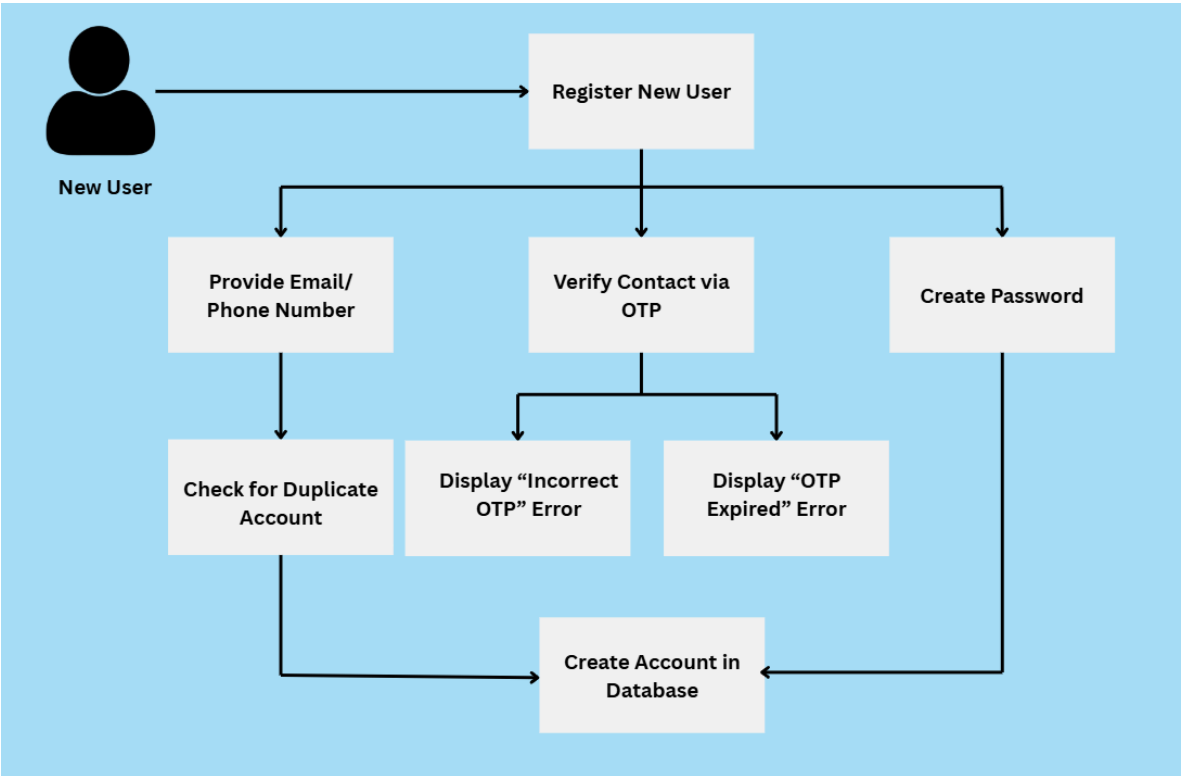
2.4.8 Limitations

The system depends on network connectivity for synchronization and relies on third-party services such as maps and payment applications, which may occasionally affect functionality. These limitations are acceptable within the system requirements.

3. Object Oriented Design

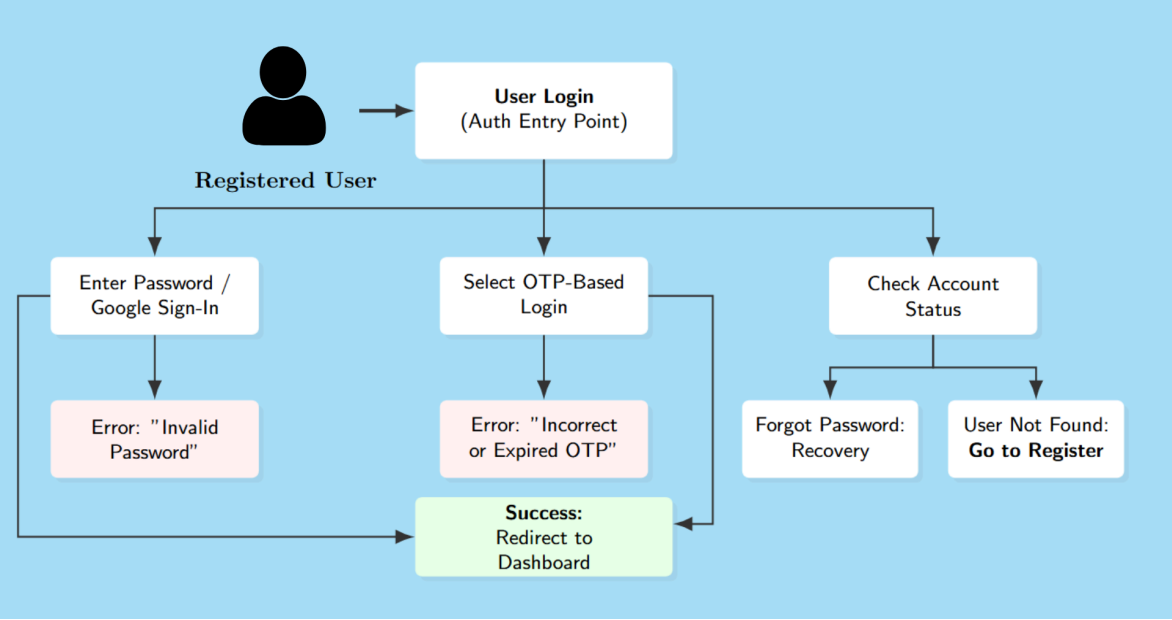
3.1 Use Case Diagrams

3.1.1 Use Case #1: User Registration (U1)



Authors	This use case was written by Ayush Jha and Khushi Khandelwal.
Purpose	To allow a new user to create an account on the Travelly application.
Requirements Traceability	FR-1
Priority	High - Mandatory for accessing the system.
Preconditions	The user must not already be registered in the system.
Post conditions	● A new user account is successfully created and stored in the system database. ● The user's email address or phone number is verified and linked to the account.
Actors	New User, System
Exceptions	● Duplicate Registration: If the provided email address or phone number is already registered, the system shall display the message: *"Account already exists. Please log in or reset your password."* ● OTP Verification Failure: 1. If the user enters an incorrect OTP, the system shall display: *"Incorrect OTP entered. Please try again."* 2. If the OTP validity period expires, the system shall display: *"OTP expired. Please request a new OTP."* ● OTP Delivery Failure: If the OTP cannot be delivered due to network or service issues, the system shall prompt the user to retry.
Includes	None
Notes/Issues	● The system shall send an OTP to the email address or phone number provided during registration for verification. ● OTPs shall have a predefined validity period (e.g., 10 minutes) to enhance security. ● Successful OTP verification is mandatory before allowing the user to set a password and complete registration.

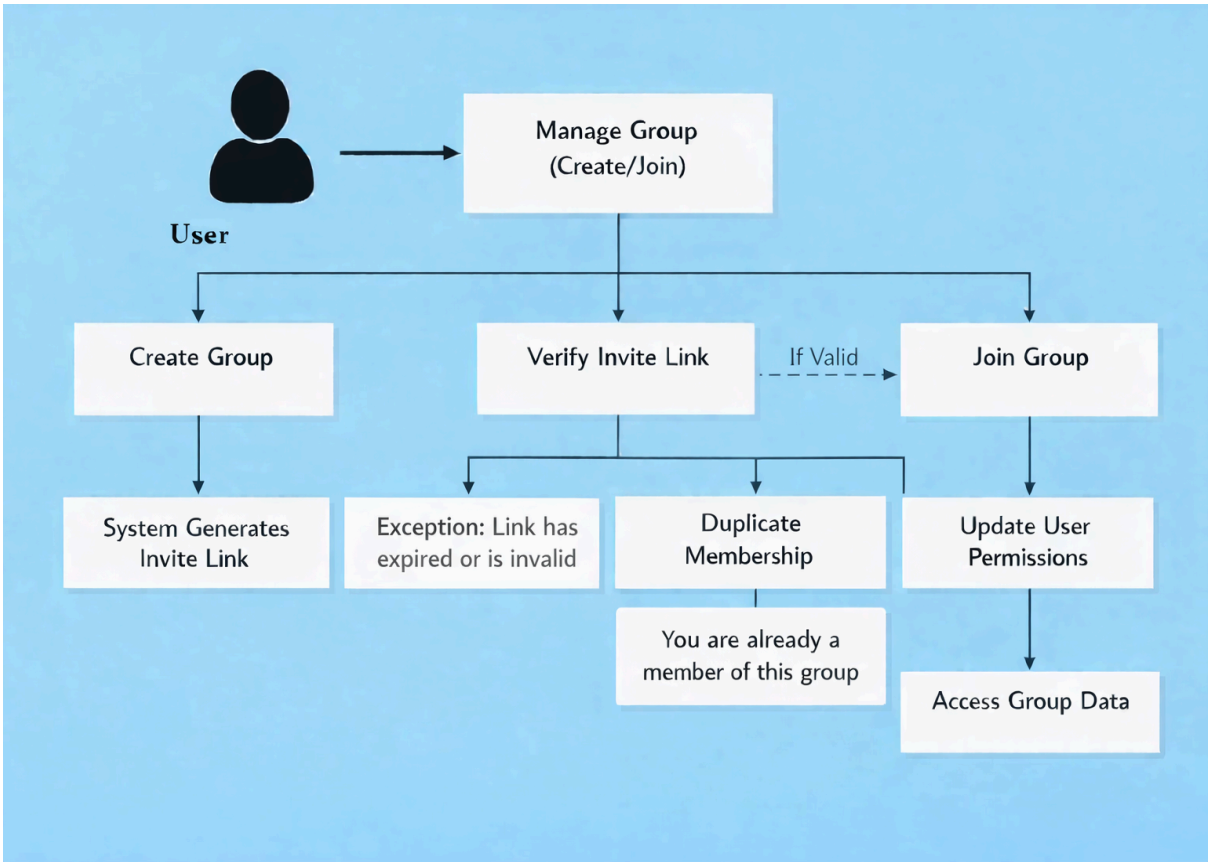
3.1.2 Use Case #2: User Login (U2)



Authors	This use case was written by Tanmay Roy and Rigved Koli.
Purpose	To authenticate a registered user and allow access to the Travelly application.
Requirements Traceability	FR-2, FR-3
Priority	High - Mandatory for accessing application features.
Preconditions	The user must be registered in the system.
Post conditions	The user is successfully authenticated and redirected to the home/dashboard screen.
Actors	User, System
Exceptions	• Unregistered User: If the provided email address or phone number is not registered, the system shall display: *"User not found. Please register before logging in."* • Invalid Credentials: If the user enters an incorrect password, the system shall display: *"Invalid password. Please try again."* • Password Recovery Triggered: If the user selects the "Forgot Password" option, the

	system shall initiate the password recovery process by sending an OTP to the registered email address or phone number. ● OTP Login Failure: 1. If the user selects OTP-based login and enters an incorrect OTP, the system shall display: *"Incorrect OTP entered. Please try again."* *2.* If the OTP expires, the system shall display: *"OTP expired. Please request a new OTP."*
Includes	**Use Case #1: User Registration (U1)** (for unregistered users redirected to registration)
Notes/Issues	● The system shall support multiple authentication methods, including password-based login, Google Sign-In, and OTP-based login. ● OTP-based login and password recovery shall follow the same security constraints regarding OTP validity duration. ● Successful authentication is required before granting access to any protected application features.

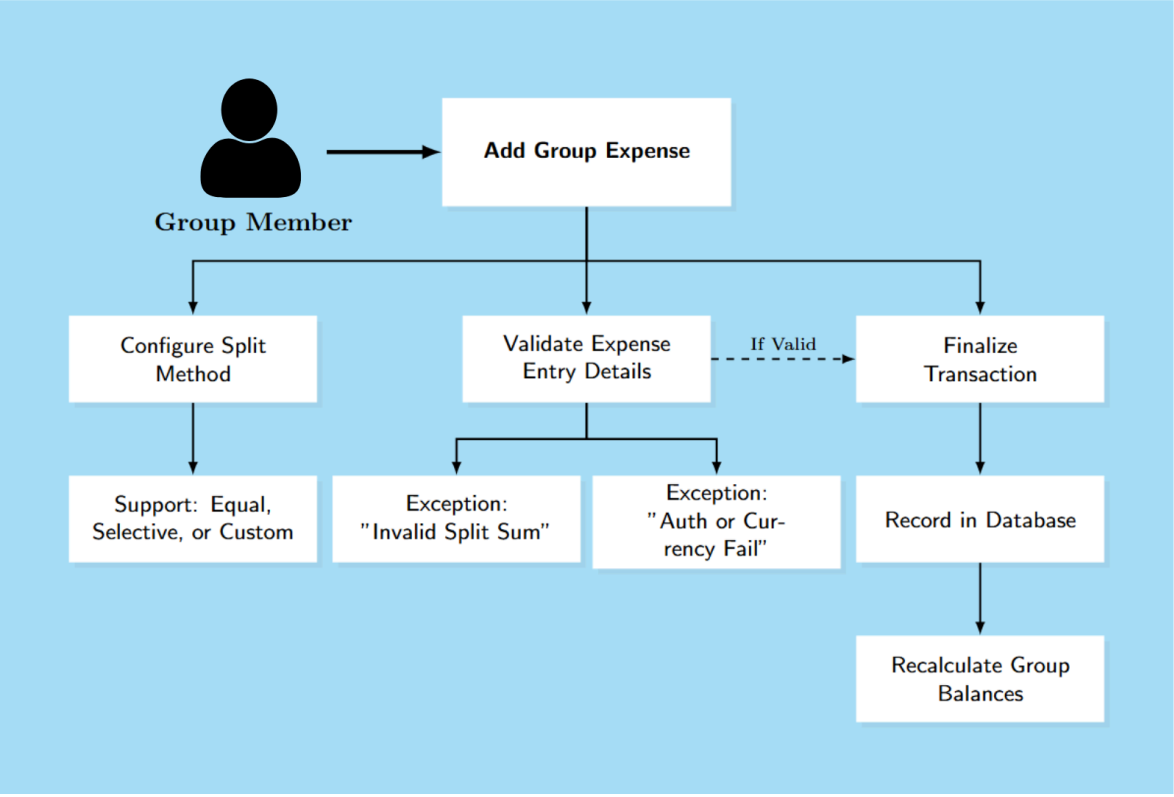
3.1.3 Use Case #3: Create or Join Group (U3)



Authors	This use case was written by Rushabh Pandya and Sanket Agarwal.
Purpose	To allow a logged-in user to create a new group or join an existing group using a valid invite link.
Requirements Traceability	FR-5, FR-6
Priority	High - Core functionality for group management
Preconditions	The user must be logged in to the application.
Post conditions	• The user becomes a member of the selected group. • The user gains access to group-related data, including expenses, balances, photos, and documents.
Actors	User, System

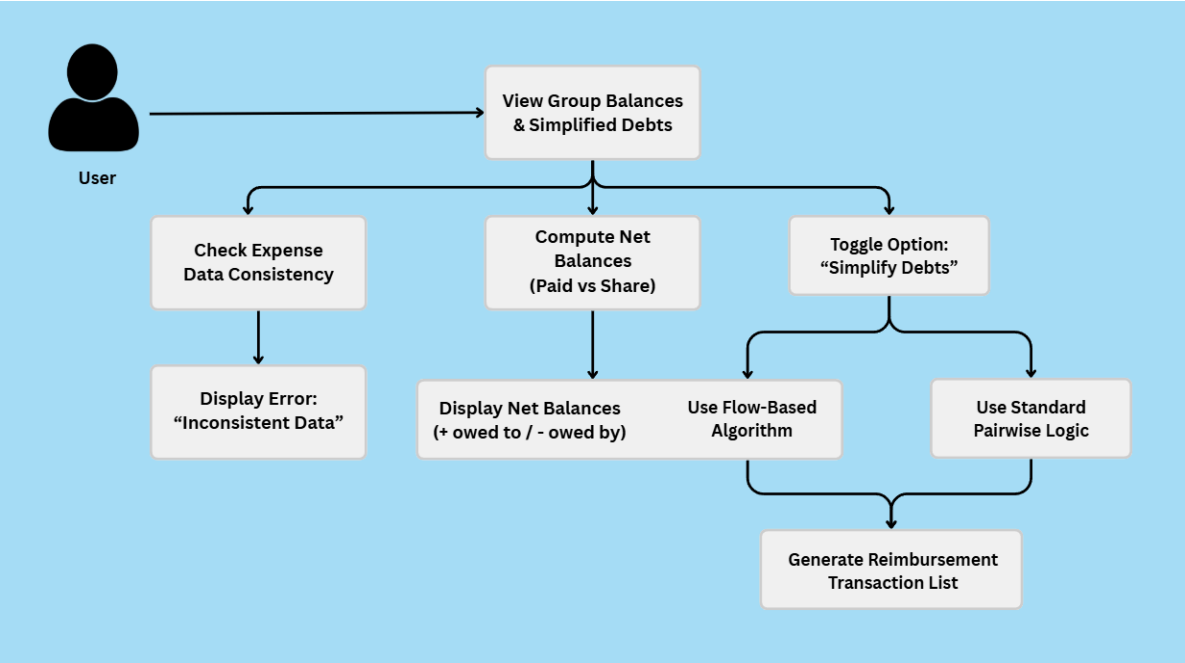
Exceptions	• Invalid or Expired Invite Link: The system shall display an error indicating that the invite link is invalid or has expired. • Group Not Found: The system shall display an error if the group associated with the invite link no longer exists. • Duplicate Membership: The system shall prevent the user from joining a group they are already a member of and display an appropriate message.
Includes	None
Notes/Issues	Late-joining members shall be able to view all existing group data, including previously recorded expenses, balances, photos, and documents.

3.1.4 Use Case #4: Add Group Expense (U4)



Authors	This use case was written by Mohd Sarim Ali and Ronit Kumar.
Purpose	To allow a group member to record a shared expense within a group.
Requirements Traceability	FR8
Priority	High - Core functionality
Preconditions	The user must be a member of the trip group.
Post conditions	● The expense is successfully recorded in the system. ● Group balances are updated to reflect the newly added expense.
Actors	User, System, Database
Exceptions	● Invalid Split Configuration: If the split amounts do not sum to the total expense amount, the system shall reject the entry and display an error. ● Currency Conversion Failure: If currency conversion cannot be completed, the system shall notify the user and prevent expense submission. ● Unauthorized Action: If a user attempts to log an expense for a non- member of the group, the system shall deny the action.
Includes	None
Notes/Issues	● Expense entries shall be reflected immediately in the group's expense history and balance overview. ● The system supports multiple splitting methods: equal split, selective split, and custom split amounts. ● Any modification or removal of recorded expenses may trigger recomputation of group balances.

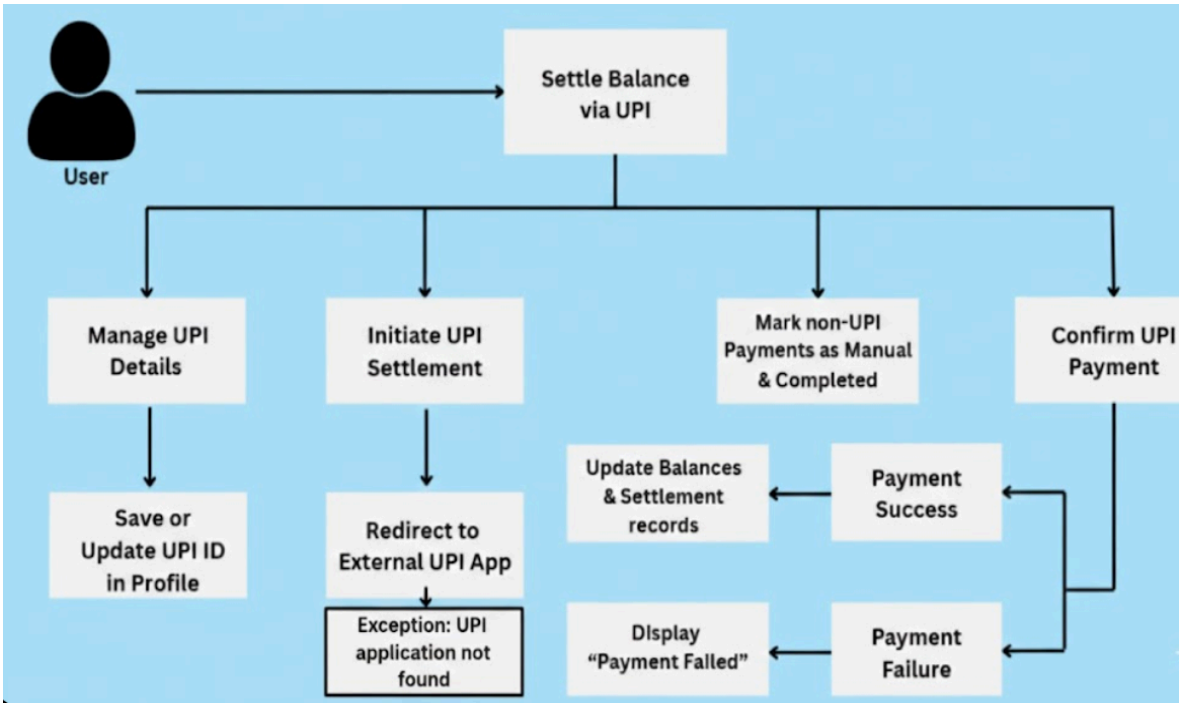
3.1.5 Use Case #5: View Group Balances & Simplified Debts (U5)



Authors	This use case was written by Pranjali Singh and Raghav Madan.
Purpose	To allow users to view net group balances and the set of reimbursement transactions required to settle outstanding amounts.
Requirements Traceability	FR-9
Priority	High - Core computation feature
Preconditions	At least one expense exists in the group.
Post conditions	• Net balances for all group members are computed and displayed. • A list of reimbursement transactions is generated based on the selected settlement mode.
Actors	User, System

Exceptions	Inconsistent Expense Data: If expense records are incomplete or inconsistent, the system shall notify the user and prevent settlement computation.
Includes	None
Notes/Issues	• The system computes reimbursement transactions using flow-based techniques to ensure correctness and efficiency. • Users may toggle the "Simplify Debts" option to reduce the total number of reimbursement transactions, which may alter original debtor-creditor relationships. • A positive balance indicates money owed to the member; a negative balance indicates money owed by the member.

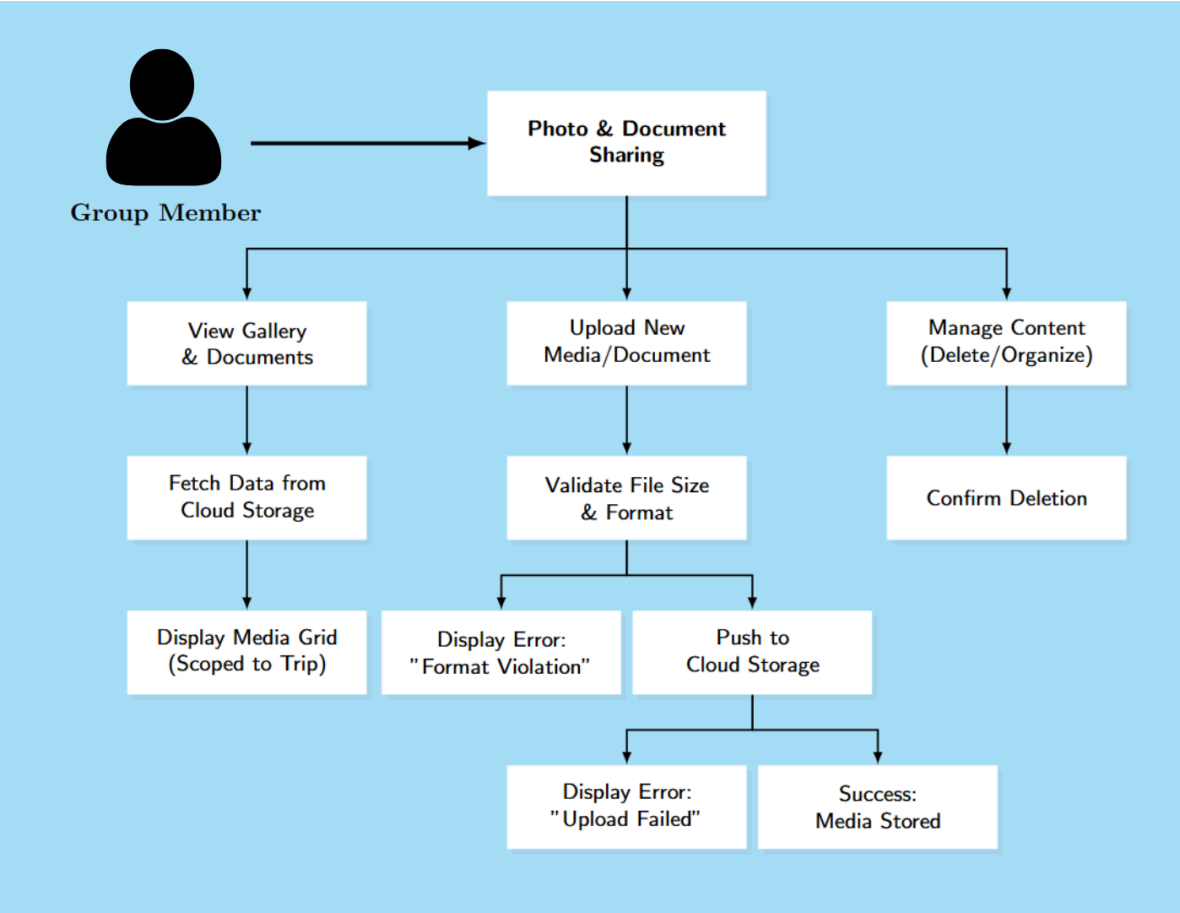
3.1.6 Use Case #6: Settle Balance via UPI (U6)



Authors	This use case was written by Pranjali Singh and Mohd Sarim Ali.
Purpose	To allow users to settle outstanding balances using UPI-based payment redirection or manual confirmation.

Requirements Traceability	FR-10
Priority	High - Core functionality
Preconditions	• The user must have a non-zero outstanding balance. • The payee must have a valid UPI ID saved in their profile.
Post conditions	The settlement is recorded in the system and group balances are updated accordingly.
Actors	User, Payment Gateway, System
Exceptions	• Payment Failure: If the payment is unsuccessful, the system shall not update balances and shall notify the user. • Payment Cancelled: If the user cancels the payment, no settlement is recorded. • UPI Application Unavailable: If no compatible UPI application is installed, the system shall notify the user.
Includes	None
Notes/Issues	• Settlement updates require explicit user confirmation and are not assumed automatically. • Manual settlements may be recorded when payments occur outside the UPI flow.

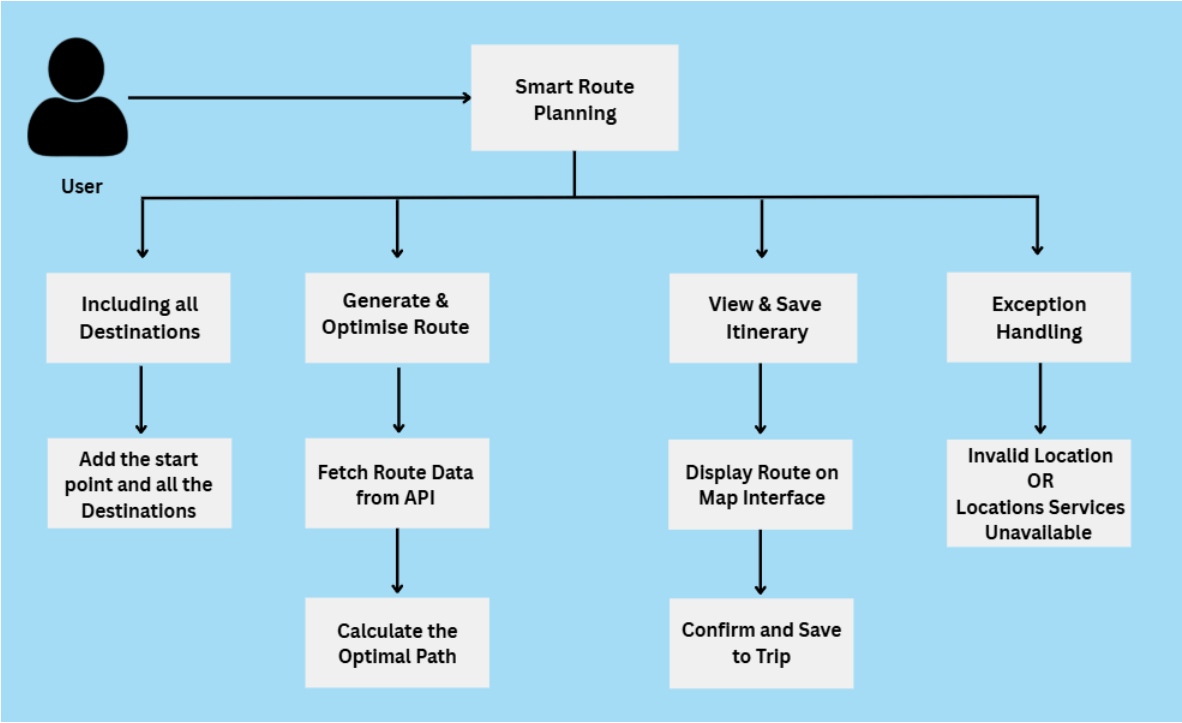
3.1.7 Use Case #7: Photo & Document Sharing (U7)



Authors	This use case was written by Ayush Jha and Rushabh Pandya.
Purpose	To allow group members to upload, view, and manage photos and documents associated with a trip.
Requirements Traceability	FR-11
Priority	Medium - Important feature for trip documentation
Preconditions	The user must be a member of the trip group.
Post conditions	Media files are successfully stored and accessible to group members.
Actors	User, System, Cloud Storage

Exceptions	• File Size or Format Violation: If the uploaded file exceeds allowed limits or is unsupported, the system shall reject the upload. • Upload Failure: If media upload fails due to connectivity or storage issues, the system shall notify the user.
Includes	None
Notes/Issues	• Access to photos and documents is restricted to group members only. • Media visibility is scoped to the corresponding trip.

3.1.8 Use Case #8: FR 12: Route Optimization and Visit Planning (U8)



Authors	This use case was written by Raghav Madan and Khushi Khandelwal.
Purpose	To generate an optimized visiting order for selected locations within a trip.

Requirements Traceability	FR-12
Priority	Medium - Useful feature for trip planning
Preconditions	The user must select a starting location, an ending location, and at least one additional location to visit.
Post conditions	An optimized visiting order covering all selected locations is generated and displayed.
Actors	User, System, Location Service
Exceptions	● Location Services Unavailable: If distance or location data cannot be retrieved, the system shall notify the user. ● Invalid Location Input: If selected locations are invalid or unreachable, the system shall prevent route generation.
Includes	None
Notes/Issues	● Exact optimality is not guaranteed for large input sets. ● The system uses external location services to calculate distances between locations.

3.2 Class Details

Below, we provide a detailed overview of each class and the associated attributes and methods:

3.2.1 User_account Class:

a) Attributes:

i) userID: Unique identifier for the user account
ii) email: string storing registered email address (nullable if phone-only)
iii) phoneNumber: string storing registered phone number (nullable if email-only)
iv) passwordHash: string storing hashed password for password-based login
v) providers: list/set storing enabled auth providers (password /login via google / otp)
vi) createdAt: datetime storing account creation time
vii) lastLoginAt: datetime storing last successful login time (nullable)
viii) isDeleted: boolean indicating whether the account is permanently deleted

b) Methods:

i) setPassword(rawPassword): Hashes and stores the password for secure authentication
ii) verifyPassword(rawPassword): Verifies a raw password against stored passwordHash
iii) linkProvider(provider): Links/enables an authentication provider for the account
iv) unlinkProvider(provider): Unlinks/disables an authentication provider from the account
v) deleteAccount(confirm): Permanently deletes the account if confirm is true

3.2.2 UserProfile Class:

a) Attributes:

i) userID: Unique identifier linking profile to User_account
ii) name: string storing user's display name
iii) email: string storing profile email (change requires OTP verification)
iv) phoneNumber: string storing profile phone number (change requires OTP verification)
v) avatarUrl: string storing profile picture URL/path (nullable)
vi) upiID: string storing user's UPI ID used to receive payments (nullable)
vii) updatedAt: datetime storing last profile update time

b) Methods:

i) updateName(name): Updates the user's display name in the profile

ii) requestEmailChange(newEmail): Creates and sends OTP to verify changing email

iii) confirmEmailChange(otpID, otp): Confirms email change after validating OTP

iv) requestPhoneChange(newPhoneNumber): Creates and sends OTP to verify changing phone number

v) confirmPhoneChange(otpID, otp): Confirms phone number change after validating OTP

vi) setUpiID(upiID): Validates and saves the user's UPI ID for settlements

vii) clearUpiID(): Removes saved UPI ID from the profile

viii) setAvatar(avatarUrl): sets profile photo

3.2.3 UserPreferences Class:

a) Attributes:

i) userID: Unique identifier linking preferences to the user

ii) themeMode: enum storing selected theme (LIGHT / DARK / SYSTEM)

iii) notificationsEnabled: boolean enabling/disabling all notifications

iv) biometricEnabled: boolean indicating whether biometric auth is enabled

v) updatedAt: datetime storing last preferences update time

b) Methods:

i) setTheme(themeMode): Updates theme preference used by the UI

ii) setNotificationsEnabled(enabled): Enables or disables push/in-app notifications globally

iii) setBiometricEnabled(enabled): Enables or disables biometric verification for the app

3.2.4 OTPChallenge Class:

a) Attributes:

i) otpID: Unique identifier for the OTP request

ii) purpose: enum storing OTP purpose (REGISTER / LOGIN / RESET_PASSWORD / CHANGE_EMAIL / CHANGE_PHONE)

iii) destination: string storing the email/phone where OTP is sent

iv) expiresAt: datetime storing OTP expiry time

v) verified: boolean indicating whether OTP is verified successfully

b) Methods:

i) verify(otp): Validates OTP and sets verified status if correct

ii) isExpired(now): Returns true if current time is after expiresAt

3.2.5 AuthSession Class:

a) Attributes:

i) sessionID: Unique identifier for the login session
ii) userID: Unique identifier of the logged-in user
iii) createdAt: datetime storing session creation time
iv) expiresAt: datetime storing session expiry time
v) revoked: boolean indicating whether session is revoked (logout)

b) Methods:

i) isActive(now): Returns true if session is not revoked and not expired
ii) logout(): Revokes the session and invalidates further access

3.2.6 AuthManager Class:

a) Attributes:

i) activeSessions: list storing active AuthSession records (to track who is currently logged in)
ii) otpChallenges: list storing active OTPChallenge records (stores issued OTP requests)

b) Methods:

i) startRegistration(email or Phone): Starts registration by creating and sending OTP
ii) completeRegistration(otpID, otp, password, name): Verifies OTP and creates User_account + UserProfile
iii) loginWithPassword(emailOrPhone, password): Authenticates user via password and creates AuthSession
iv) startOtpLogin(emailOrPhone): Sends OTP to user for OTP-based login
v) completeOtpLogin(otpID, otp): Verifies OTP and creates AuthSession
vi) loginWithGoogle(googleToken): Verifies Google token and creates AuthSession
vii) startPasswordReset(email or Phone): Sends OTP to start password reset flow
viii) completePasswordReset(otpID, otp, newPassword): Verifies OTP and updates passwordHash
ix) logout(sessionID): Revokes the active session to log out user
x) deleteProfile(userID, confirm): Deletes user profile/account permanently if confirm is true

3.2.7 AppInfo Class:

a) Attributes:

i) appName: string storing application name
ii) version: string storing current application version
iii) features: list of strings summarizing major app features
iv) devTeam: list of strings storing developer/team information

b) Methods:

i) getAbout(): Returns appName/version/features/devTeam for About Travelly page

3.2.8 TripGroup Class:

a) Attributes:

i) groupID: Unique identifier for the trip group
ii) title: string storing group name/title
iii) photoUrl: string storing group photo URL/path (default used if not provided)
iv) defaultCurrency: string storing default currency code for new entries
v) inviteToken: string storing invite link token for joining
vi) inviteActive: boolean indicating whether invite link is valid
vii) createdByUserID: Unique identifier of the user who created the group
viii) createdAt: datetime storing group creation time

b) Methods:

i) updateTitle(title): Updates group title shown across the application
ii) updatePhoto(photoUrl): Updates group photo; can set to default if empty/null
iii) setDefaultCurrency(currency): Sets default currency used when user adds new expenses/transfers
iv) getInviteLink(): Returns active invite link, generated from inviteToken
iv) revokeInvite(): Invalidates inviteToken so no new users can join via old link

3.2.9 GroupMember Class:

a) Attributes:

i) memberID: Unique identifier for a member record in a group
ii) groupID: Unique identifier of the group this member belongs to
iii) displayName: string storing member display name (can be renamed)
iv) linkedUserID: Unique identifier of actual user (nullable for placeholders)
v) isPlaceholder: boolean indicating pre-added member not yet linked to a user
vi) addedByUserID: Unique identifier of the user who added this member placeholder
vii) joinedAt: datetime storing time the member joined/claimed (nullable)
viii) status: enum storing member state (ACTIVE /REMOVED)
ix) removedAt: datetime storing the time the member was removed (nullable)
x) removedByUserID: Unique identifier of user who removed the member

b) Methods:

i) linkUser(userID): Links a placeholder member slot to a real registered user
ii) rename(newDisplayName): Updates member displayName in the group
iii) isActive(): Returns true if member is active
iv) markRemoved(removedByUserID): marks member as removed and records removedAt and removedByUserID

3.2.10 GroupSettings Class:

a) Attributes:

i) simplifyDebts: boolean indicating whether simplified reimbursements are enabled

ii) largeExpenseAlertEnabled: boolean indicating whether large-expense notifications are enabled for this group

iii) largeExpenseThresholdAmount: decimal storing threshold amount for “large expense” (default = 5000)

iv) largeExpenseThresholdCurrency: string storing threshold currency (default = "INR")

v) updatedAt: datetime storing last settings update time

b) Methods:

i) setSimplifyDebts(enabled): Enables/disables simplified debt optimization

ii) setLargeExpenseAlertEnabled(enabled): Enables/disables large expense alerts for the group

iii) setLargeExpenseThreshold(amount, currency): Updates threshold used for large expense alerts (default 5000 INR)

3.2.11 GroupManager Class:

a) Attributes:

i) groups: list storing all TripGroup records

ii) members: list storing all GroupMember records

b) Methods:

i) createGroup(creatorUserID, title, photoUrl, defaultCurrency, preAddedNames): Creates a group, generates invite link, and creates placeholder members

ii) addParticipants(groupID, addedByUserID, names): Adds new placeholder members to existing group using same invite link

iii) joinGroup(inviteToken, userID, selectedOrNewName): Joins group via link and links/creates member identity for that user

iv) deleteGroup(groupID, requestedByUserID, confirm): Hard-deletes the group and all related data if confirm is true (any member allowed)

v) getGroupSnapshot(groupID, userID): Returns full group state (members, expenses, reimbursements, media) to ensure late joiners see all info

vi) removeParticipant(groupID, memberID, confirm): removes participant from group

3.2.12 Expense Class:

a) Attributes:

i) expenseID: Unique identifier for the expense

ii) groupID: Unique identifier of the group where expense is recorded
iii) title: string storing expense title
iv) amount: decimal storing total amount
v) currency: string storing currency code of this expense
vi) payerMemberID: Unique identifier of member who paid the expense
vii) paidDate: date storing the payment date
viii) splitMethod: enum storing split type (EQUAL/ CUSTOM)
ix) shares: list storing ExpenseShare entries for this expense
x) createdByUserID: Unique identifier of user who created this expense
xi) createdAt: datetime storing expense creation time

b) Methods:

i) setSplitEqualAll(memberIDs): Sets shares so all members pay equal parts of amount
ii) setSplitEqualSelected(selectedMemberIDs): Sets shares equally among selected members only
iii) setSplitCustom(memberToAmountMap): Sets shares exactly as provided per member
iv) validateSplit(): Ensures shares are valid and sum equals amount in same currency

3.2.13 ExpenseShare Class:

a) Attributes:

i) shareID: Unique identifier for the share entry
ii) expenseID: Unique identifier of the parent expense
iii) memberID: Unique identifier of the member who owes this share
iv) shareAmount: decimal storing owed share amount (same currency as expense)
v) currency: string storing currency code (mirrors expense currency)
vi) included: boolean indicating whether member is included in split

b) Methods:

i) include(): Marks the member as included in the expense split
ii) exclude(): Marks the member as excluded from the expense split

3.2.14 MoneyTransfer Class:

a) Attributes:

i) transferID: Unique identifier for the transfer
ii) groupID: Unique identifier of the group where transfer is recorded
iii) title: string storing transfer title/description
iv) amount: decimal storing transfer amount
v) currency: string storing currency code for the transfer
vi) senderMemberID: Unique identifier of member who sends money
vii) receiverMemberID: Unique identifier of member who receives money

viii) createdByUserID: Unique identifier of user who recorded transfer
ix) createdAt: datetime storing transfer creation time

b) Methods:

i) update(title, amount, currency, senderMemberID, receiverMemberID): Edits transfer details after creation

3.2.15 ExpenseManager Class:

a) Attributes:

i) expenses: list storing Expense records
ii) transfers: list storing MoneyTransfer records

b) Methods:

i) addExpense(expense): Stores a new expense and updates group's transaction history
ii) addTransfer(transfer): Stores a new transfer and updates group's transaction history
iii) getHistory(groupID): Returns combined expense + transfer history ordered by time
iv) getTotalExpenditure(groupID): Computes total spending grouped by currency
v) getIndividualSpending(groupID): Computes each member's spending grouped by currency
vi) isLargeExpense(expense, groupSettings, groupID): Returns true if expense crosses the configured threshold

3.2.16 LedgerEntry Class:

a) Attributes:

i) entryID: Unique identifier for the ledger entry
ii) groupID: Unique identifier of the group
iii) sourceType: string storing source type (EXPENSE / TRANSFER)
iv) sourceID: Unique identifier of the source transaction (expenseID/transferID)
v) debtorMemberID: Unique identifier of member who owes money
vi) creditorMemberID: Unique identifier of member who is owed money
vii) amount: decimal storing owed amount (stored as-is without conversion)
viii) currency: string storing currency code for this owed amount
ix) createdAt: datetime storing creation time of ledger entry

b) Methods:

i) reverse(): Creates a reverse ledger entry useful for corrections/rollback logic
ii) deleteEntryForMember(groupID, memberID): delete all entries for involved member

3.2.17 MemberBalance Class:

a) Attributes:

i) groupID: Unique identifier of the group
ii) memberID: Unique identifier of the member
iii) balancesByCurrency: map storing net balance per currency (positive = receive, negative = pay)
iv) computedAt: datetime storing last computation time

b) Methods:

i) getBalance(currency): Returns the member's net balance for the given currency
ii) isCreditor(currency): Returns true if balance in that currency is positive
iii) isDebtor(currency): Returns true if balance in that currency is negative

3.2.18 ReimbursementTransaction Class:

a) Attributes:

i) reimbursementID: Unique identifier for reimbursement suggestion
ii) groupID: Unique identifier of the group
iii) debtorMemberID: Unique identifier of member who should pay
iv) creditorMemberID: Unique identifier of member who should receive
v) amount: decimal storing settlement amount
vi) currency: string storing currency code for settlement
vii) status: enum storing reimbursement status (OPEN / REQUESTED / PAY_INITIATED / SETTLED / MARKED_PAID_MANUAL)
viii) createdAt: datetime storing creation time
ix) updatedAt: datetime storing last update time

b) Methods:

i) request(): Marks status as REQUESTED and triggers a settlement request notification
ii) markPayInitiated(): Marks status as PAY_INITIATED when payer opens UPI flow
iii) settle(): Marks status as SETTLED after payer confirms payment success
iv) markPaidManual(confirm): Marks as paid manually if confirm is true
v) settleAuto(GroupMember): Marks payments of a group member as settled if that group member is removed

3.2.19 SettlementRecord Class:

a) Attributes:

i) settlementID: Unique identifier for settlement record
ii) groupID: Unique identifier of the group
iii) reimbursementID: Unique identifier of the reimbursement being settled
iv) payerMemberID: Unique identifier of member who paid
v) payeeMemberID: Unique identifier of member who received

vi) amount: decimal storing settled amount
vii) currency: string storing currency code for settlement
viii) method: string storing settlement method (UPI / MANUAL)
ix) note: string storing optional transaction note (nullable)
x) confirmedAt: datetime storing confirmation timestamp
xi) confirmedByUserID: Unique identifier of user who confirmed settlement

b) Methods:

i) generateReceipt(): Generates a readable receipt summary for the completed settlement

3.2.20 DebtOptimizationEngine Class:

a) Attributes:

i) ledgerEntries: list storing LedgerEntry records used for balance computation
ii) groupSettings: GroupSettings storing simplifyDebts toggle for the group

b) Methods:

i) computeBalances(groupID): Computes MemberBalance for each member grouped by currency
ii) generateReimbursementsPreserve(groupID): Generates reimbursements preserving original debtor->creditor relations per currency
iii) generateReimbursementsSimplified(groupID): Generates fewer reimbursements ignoring original relations per currency

3.2.21 UpiPaymentIntent Class:

a) Attributes:

i) intentID: Unique identifier for UPI payment intent
ii) reimbursementID: Unique identifier of reimbursement being paid
iii) payerUserID: Unique identifier of user initiating payment
iv) payeeUpiID: string storing payee UPI ID (must be valid to enable pay)
v) amount: decimal storing amount to pay
vi) currency: string storing currency code
vii) note: string storing optional transaction note (nullable)
viii) createdAt: datetime storing intent creation time
ix) returnedAt: datetime storing return time from UPI app (nullable)
x) userConfirmedSuccess: boolean storing popup confirmation result (nullable)

b) Methods:

i) buildDeepLink(): Builds the UPI deep link URI to open a selected UPI app with prefilled details
ii) recordReturn(success): Stores the user's confirmation (Yes/No) after returning from UPI app

3.2.22 PaymentGatewayService Class:

a) Attributes:

i) supportedApps: list storing supported UPI apps (GPay / PhonePe / Paytm etc.)

b) Methods:

i) createIntent(payeeUpiID, amount, currency, note): Creates a UpiPaymentIntent for a reimbursement payment

ii) redirectToApp(intent): Opens the UPI app using the deep link generated from the intent

iii) handleReturn(intentID, success): Records return result and passes outcome to settlement flow

3.2.23 SettlementManager Class:

a) Attributes:

i) reimbursements: list storing ReimbursementTransaction records

ii) settlements: list storing SettlementRecord records

iii) paymentGateway: PaymentGatewayService for UPI intent/redirect handling

iv) debtEngine: DebtOptimizationEngine used to recompute balances/reimbursements

b) Methods:

i) viewReimbursements(groupID): Returns reimbursements list and balances overview for the group

ii) requestPayment(reimbursementID): Sends a payment request/reminder notification to debtor

iii) pay(reimbursementID, note): Starts UPI payment flow by generating intent and redirecting to UPI app

iv) confirmPayment(intentID, didGoThrough): Marks reimbursement settled if didGoThrough is true and stores SettlementRecord

v) markAsPaidManual(reimbursementID, confirm): Marks reimbursement settled manually and stores SettlementRecord if confirm is true

vi) recompute(groupID): Recomputes balances and reimbursement suggestions after changes

3.2.24 Notification Class:

a) Attributes:

i) notificationID: Unique identifier for the notification

ii) userID: Unique identifier of notification recipient

iii) groupID: Unique identifier of related group (nullable)

iv) type: enum storing notification type (SETTLEMENT_REQUEST / PAYMENT_REMINDER / GROUP_UPDATE / SECURITY_ALERT)

v) title: string storing notification title

vi) body: string storing notification message body

vii) createdAt: datetime storing notification creation time
viii) isRead: boolean indicating whether notification is read

b) Methods:

i) markAsRead(): Marks notification as read so it no longer appears in unread list

3.2.25 NotificationManager Class:

a) Attributes:

i) notifications: list storing Notification records

b) Methods:

i) sendSettlementRequest(userID, reimbursementID): Sends a notification when someone requests settlement
ii) sendPaymentReminder(userID, reimbursementID): Sends a reminder notification to complete payment
iii) sendGroupUpdate(userID, groupID, message): Sends notification for group-related updates
iv) sendSecurityAlert(userID, message): Sends notification for security-critical events
v) getUnread(userID): Returns all notifications where isRead is false
vi) markRead(notificationID): Marks a specific notification as read

3.2.26 MediaItem Class:

a) Attributes:

i) mediaID: Unique identifier for the media item
ii) groupID: Unique identifier of the group that owns this media
iii) uploaderUserID: Unique identifier of user who uploaded the media
iv) category: enum storing media type (PHOTO / DOCUMENT)
v) fileName: string storing original file name
vi) mimeType: string storing file MIME type for format validation
vii) sizeBytes: long storing file size in bytes
viii) objectKey: string storing object storage key/path
ix) uploadedAt: datetime storing upload time

b) Methods:

i) getDownloadLink(): Generates a downloadable link/URL for the stored media object
ii) hardDelete(): Permanently deletes media metadata and removes file from storage

3.2.27 MediaConstraints Class:

a) Attributes:

i) maxFileSizeBytes: long storing maximum allowed upload size
ii) allowedPhotoTypes: set storing allowed photo MIME types
iii) allowedDocTypes: set storing allowed document MIME types

b) Methods:

i) validate(mimeType, sizeBytes): Validates upload based on size and allowed format rules

3.2.28 FileStorageService Class:

a) Attributes:

i) bucketName: string storing object storage bucket/container name
ii) basePath: string storing base path/prefix for stored objects

b) Methods:

i) upload(objectKey, bytes, mimeType): Uploads raw file bytes to object storage under objectKey
ii) download(objectKey): Downloads raw file bytes from object storage using objectKey
iii) generateUrl(objectKey): Generates a secure URL for downloading the object
iv) delete(objectKey): Permanently deletes the object from storage

3.2.29 MediaManager Class:

a) Attributes:

i) mediaItems: list storing MediaItem metadata records
ii) constraints: MediaConstraints storing validation rules
iii) storageService: FileStorageService for object storage operations

b) Methods:

i) uploadMedia(groupID, userID, category, file): Validates and uploads media and stores metadata
ii) listPhotos(groupID): Returns all MediaItems with category PHOTO for grid display
iii) listDocuments(groupID): Returns all MediaItems with category DOCUMENT for list display
iv) downloadMedia(mediaID): Downloads the selected media file using its objectKey
v) deleteMedia(mediaID, confirm): Hard-deletes media if confirm is true (metadata + stored object)

3.2.30 Location Class:

a) Attributes:

i) placeID: string storing external maps place identifier (nullable)
ii) name: string storing location name

iii) address: string storing full address (nullable)
iv) latitude: double storing latitude coordinate
v) longitude: double storing longitude coordinate

b) Methods:
i) toCoordinates(): Returns latitude and longitude as coordinates for distance computation

3.2.31 RoutePlanRequest Class:

a) Attributes:
i) requestID: Unique identifier for the route plan request
ii) groupID: Unique identifier of group under which plan is created
iii) createdByUserID: Unique identifier of user who created the plan
iv) start: Location storing starting location
v) end: Location storing ending location
vi) stops: list storing intermediate locations to visit
vii) createdAt: datetime storing request creation time

b) Methods:
i) validate(): Ensures start/end exist and stops list is valid before optimization

3.2.32 RoutePlanResult Class:

a) Attributes:
i) requestID: Unique identifier linking result to RoutePlanRequest
ii) optimizedOrder: list storing visiting order (start -> stops -> end)
iii) totalDistance: double storing computed total distance
iv) computedAt: datetime storing computation time
v) algorithmUsed: string storing algorithm name/identifier

b) Methods:
i) getOrder(): Returns optimizedOrder sequence for UI display
ii) getDistance(): Returns totalDistance value for UI display

3.2.33 LocationServiceClient Class:

a) Attributes:
i) providerName: string storing maps/location provider name
ii) apiKeyRef: string storing API key reference/config identifier

b) Methods:
i) suggestPlaces(query): Returns suggested locations based on user query input
ii) getDistance(locationA, locationB): Returns distance between two locations using maps API

iii) getDistanceMatrix(locations): Returns matrix of pairwise distances used by route optimization

3.2.34 RouteOptimizer Class:

a) Attributes:

i) locationClient: LocationServiceClient used to compute distances

b) Methods:

i) optimize(start, end, stops): Computes best visiting order minimizing total distance

ii) computeTotalDistance(order): Computes total distance for a given visiting order

3.2.35 RoutePlanningManager Class:

a) Attributes:

i) requests: list storing RoutePlanRequest records

ii) results: list storing RoutePlanResult records

iii) optimizer: RouteOptimizer used to compute best visiting order

b) Methods:

i) createPlan(groupID, userID, start, end, stops): Creates request, calls optimizer, and stores RoutePlanResult

ii) getPlan(requestID): Returns saved RoutePlanResult for the given request

3.2.36 BiometricAuthManager Class:

a) Attributes:

i) supported: boolean indicating whether device supports biometric authentication

ii) enabledUsers: set storing userIDs who enabled biometric auth

b) Methods:

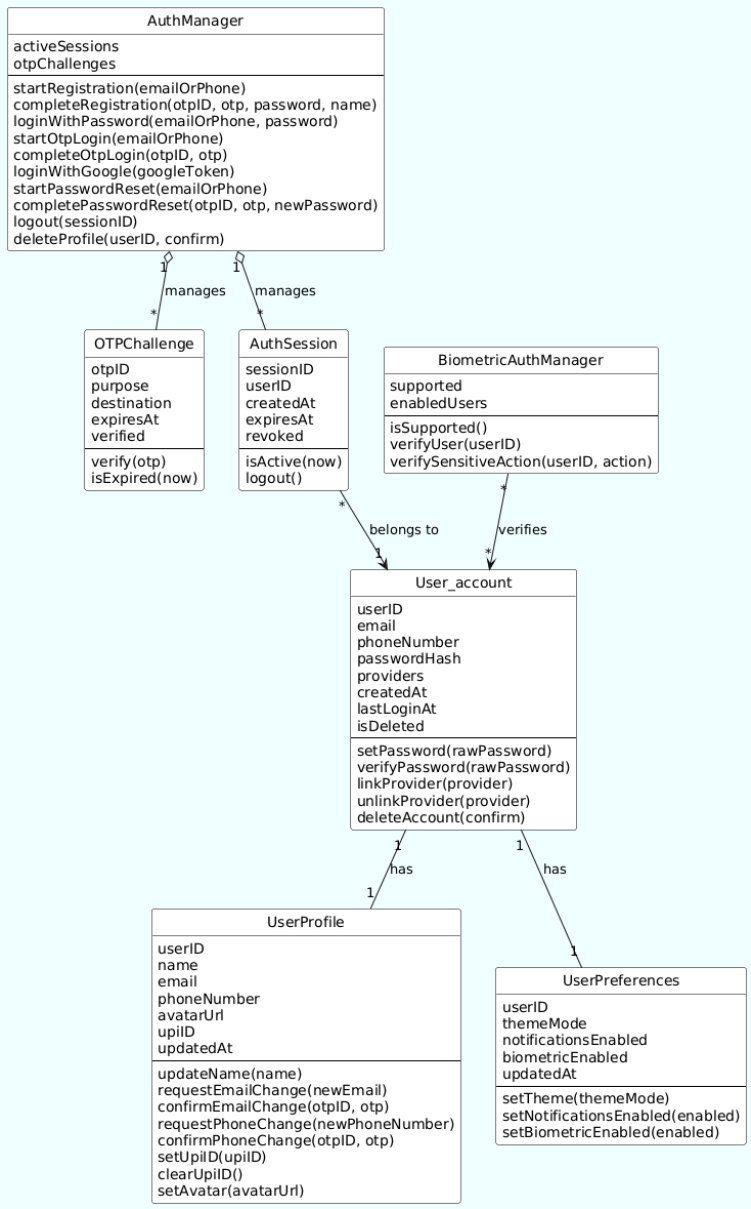
i) isSupported(): Checks device capability and returns supported status

ii) verifyUser(userID): Prompts biometric verification to authenticate the user

iii) verifySensitiveAction(userID, action): Uses biometric verification before sensitive actions (delete group/account)

3.3 Class Diagram

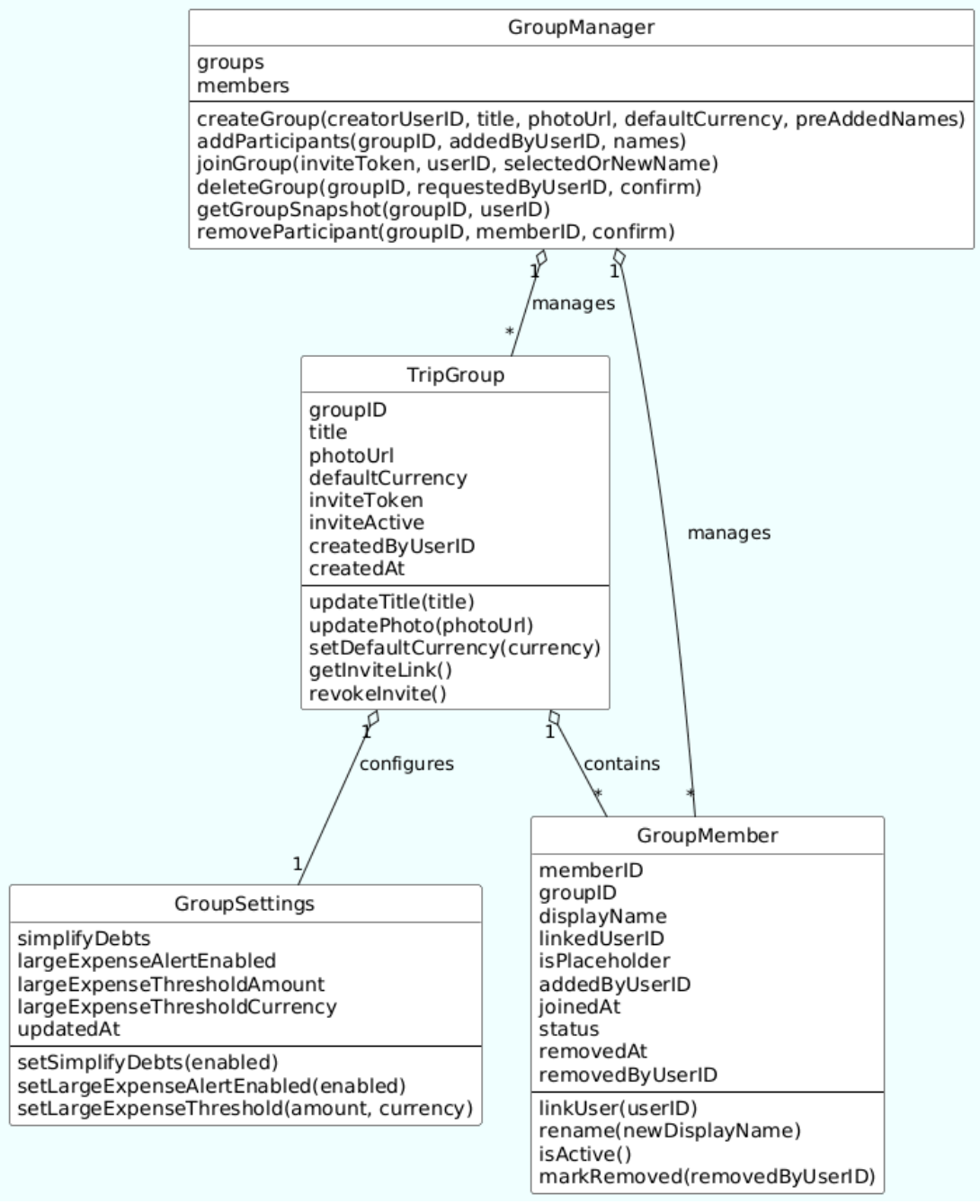
3.3.1 Authentication & User Account Subsystem



This class diagram represents the authentication and account management subsystem of Travelly. It includes the core classes responsible for user registration, login, OTP-based verification, password management, session handling, and user preferences. The diagram highlights how AuthManager coordinates OTP challenges

and active sessions while maintaining associations with user profile and preference information. This subsystem ensures secure access control and supports multiple authentication providers as required in the SRS.

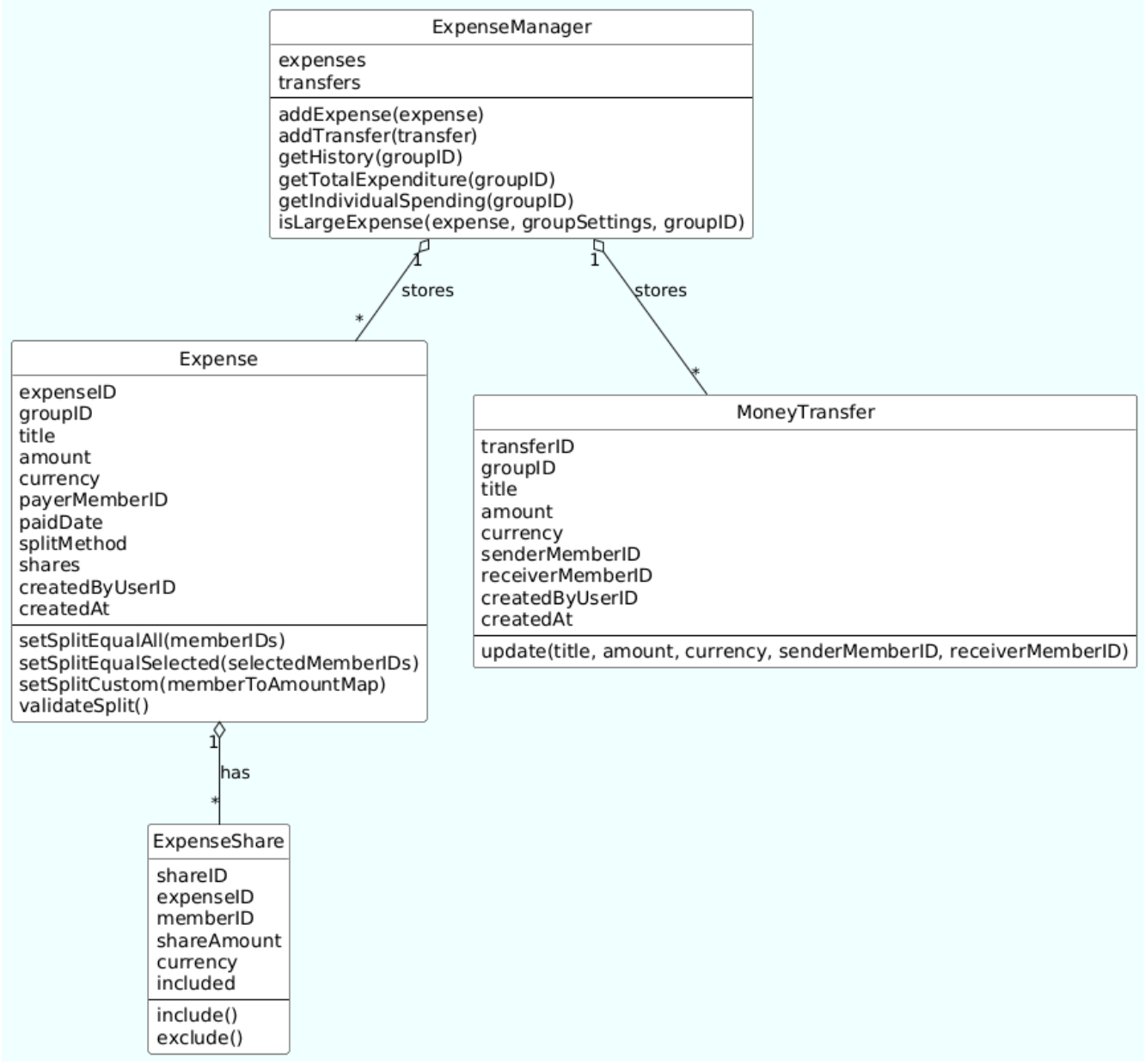
3.3.2 Trip Group and Membership Subsystem



This class diagram models the trip group creation and membership management functionality in Travelly. It captures the structure of a trip group (TripGroup), its members (GroupMember), and configurable group settings (GroupSettings). The diagram also

shows how the GroupManager acts as the controlling class to handle group creation, invite link joining, participant management, and group deletion. This subsystem supports late-joining members and placeholder participants as described in the SRS.

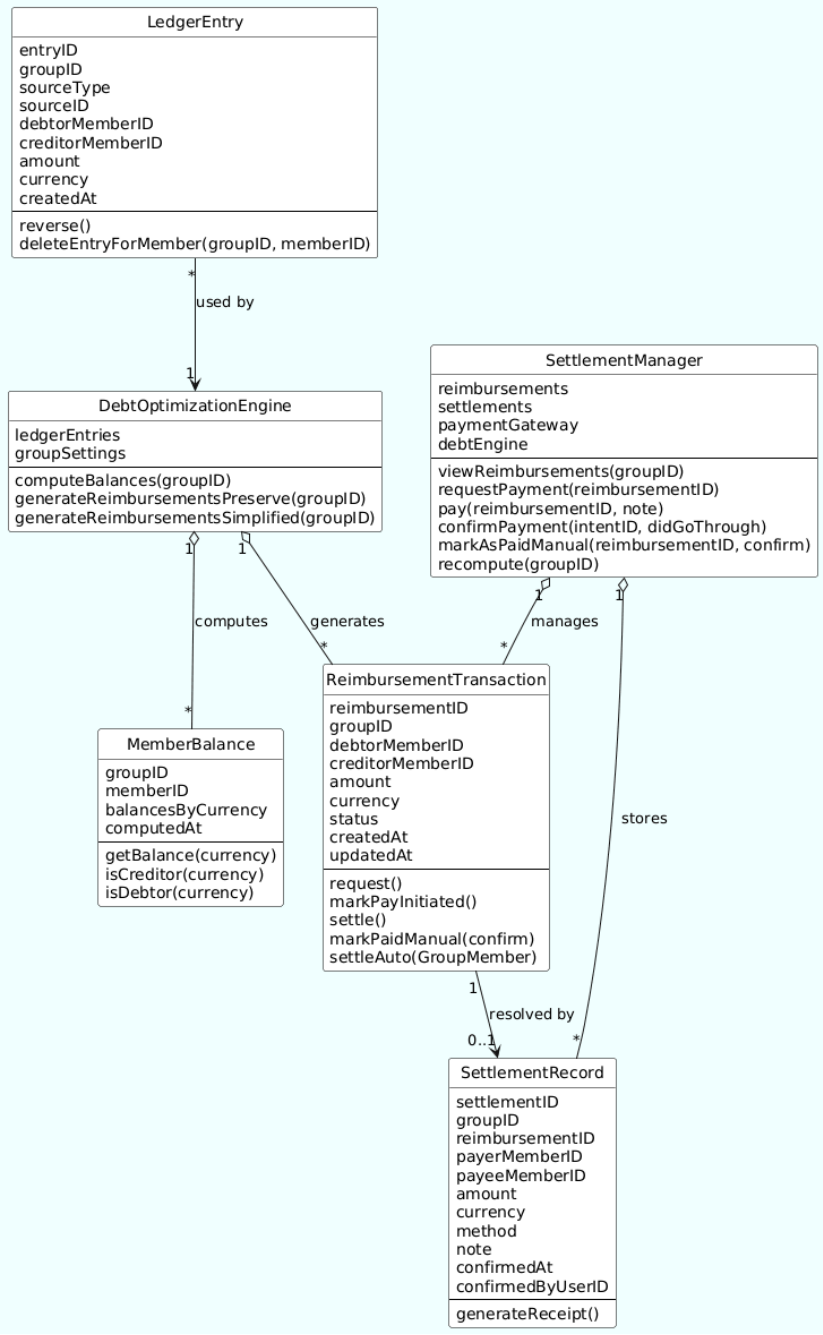
3.3.3 Balance Computation and Settlement Subsystem



This class diagram represents the expense logging and transaction tracking subsystem of Travelly. It includes the Expense class with its associated ExpenseShare objects to support equal and custom splitting methods. The MoneyTransfer class models direct transfers between members. The ExpenseManager acts as the primary management class responsible for storing expenses, transfers, and generating transaction history summaries.

This subsystem ensures accurate recording of trip expenses and supports transparency for all group members.

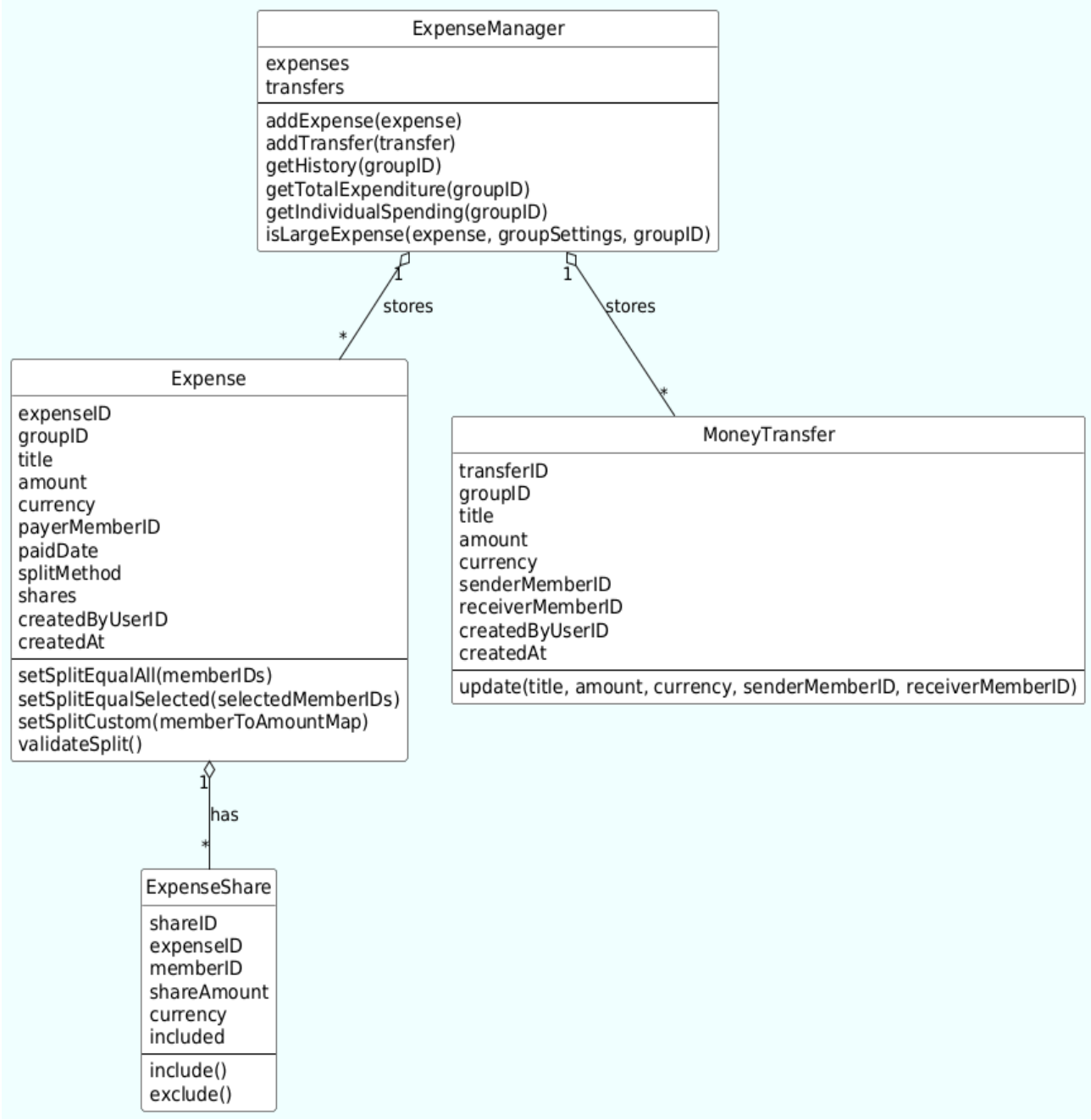
3.3.4 Expense and Transaction Subsystem



This class diagram illustrates the balance computation and reimbursement generation subsystem of Travelly. The system maintains debtor-creditor relations using LedgerEntry objects, computes per-member balances using MemberBalance, and generates settlement

suggestions through ReimbursementTransaction. The DebtOptimizationEngine is responsible for producing reimbursement instructions based on the selected debt simplification mode. The SettlementManager coordinates reimbursements, records final settlements, and triggers recomputation after settlement actions.

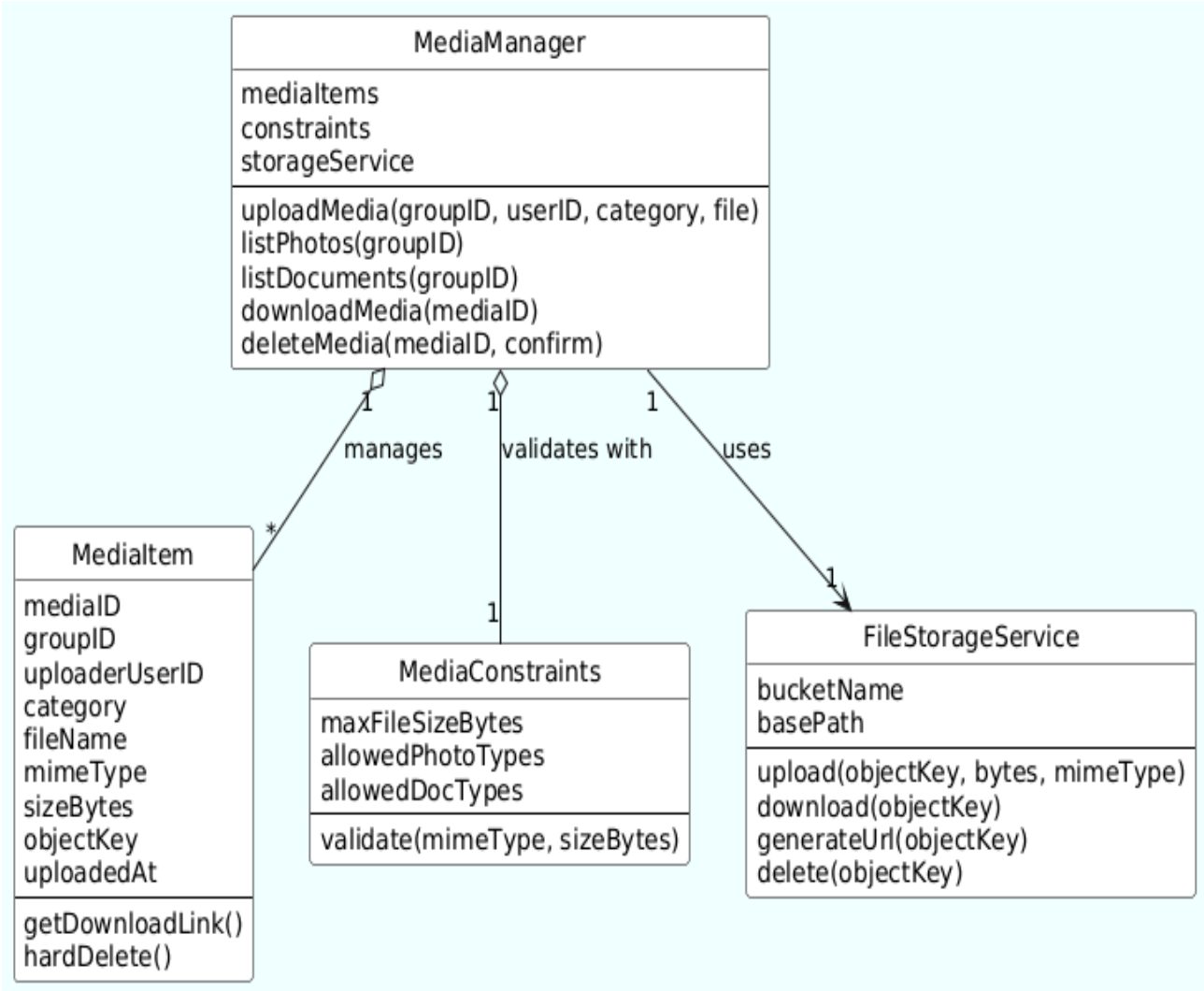
3.3.5 UPI Payment and Settlement Integration Subsystem



This class diagram models the UPI-based settlement workflow in Travelly. It includes the UpiPaymentIntent class for generating UPI deep links with pre-filled payment details and the PaymentGatewayService class for handling redirection to external UPI apps. The

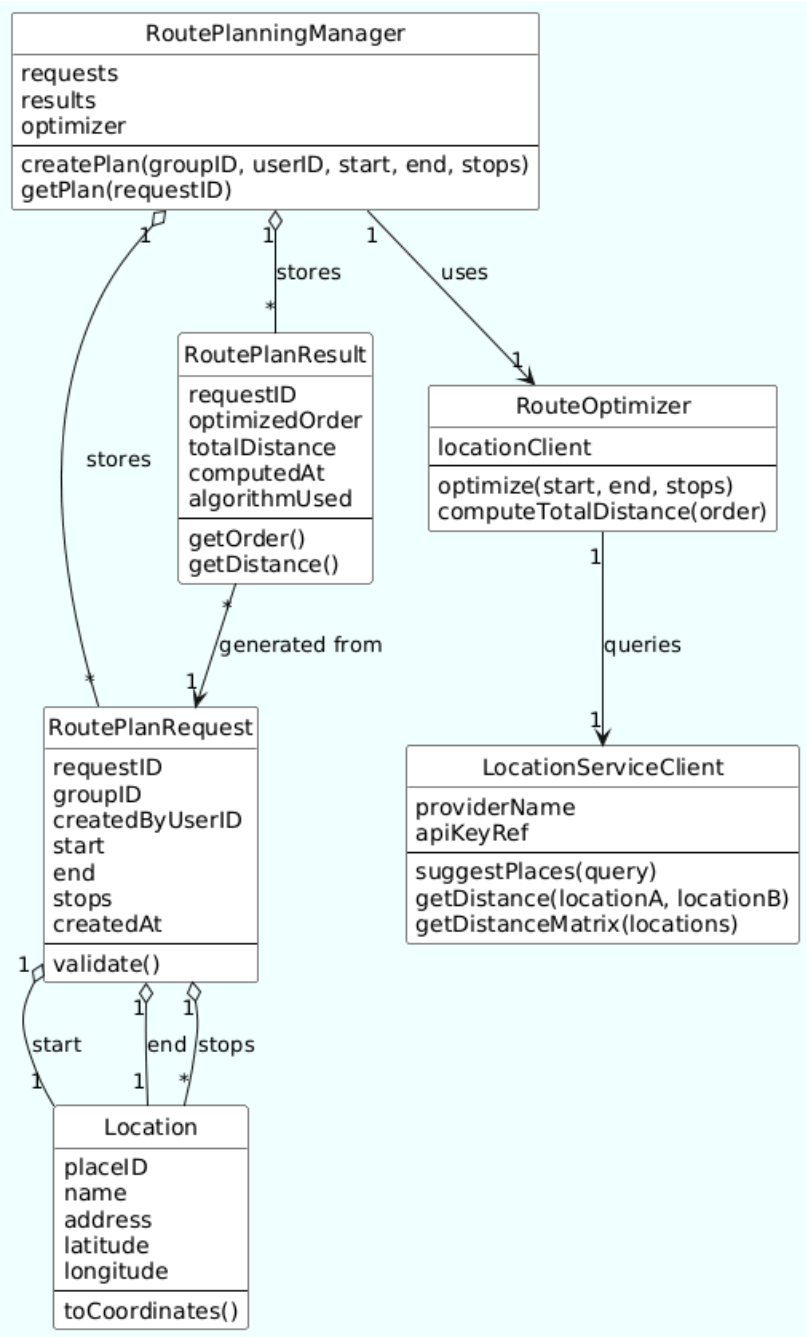
SettlementManager interacts with these classes to initiate payment flows and confirm settlement status based on user confirmation. This subsystem supports both UPI-based settlement and manual settlement marking as defined in the SRS.

3.3.6 Media Storage and Management Subsystem



This class diagram models the trip-wise photo and document management subsystem of Travelly. The MediaItem class represents uploaded files, while MediaConstraints defines validation rules such as file size limits and allowed MIME types. The FileStorageService handles upload, download, URL generation, and deletion operations for cloud-based object storage. The MediaManager coordinates these components to provide secure and efficient media management for group members, as required in the SRS.

3.3.7 Route Planning Subsystem

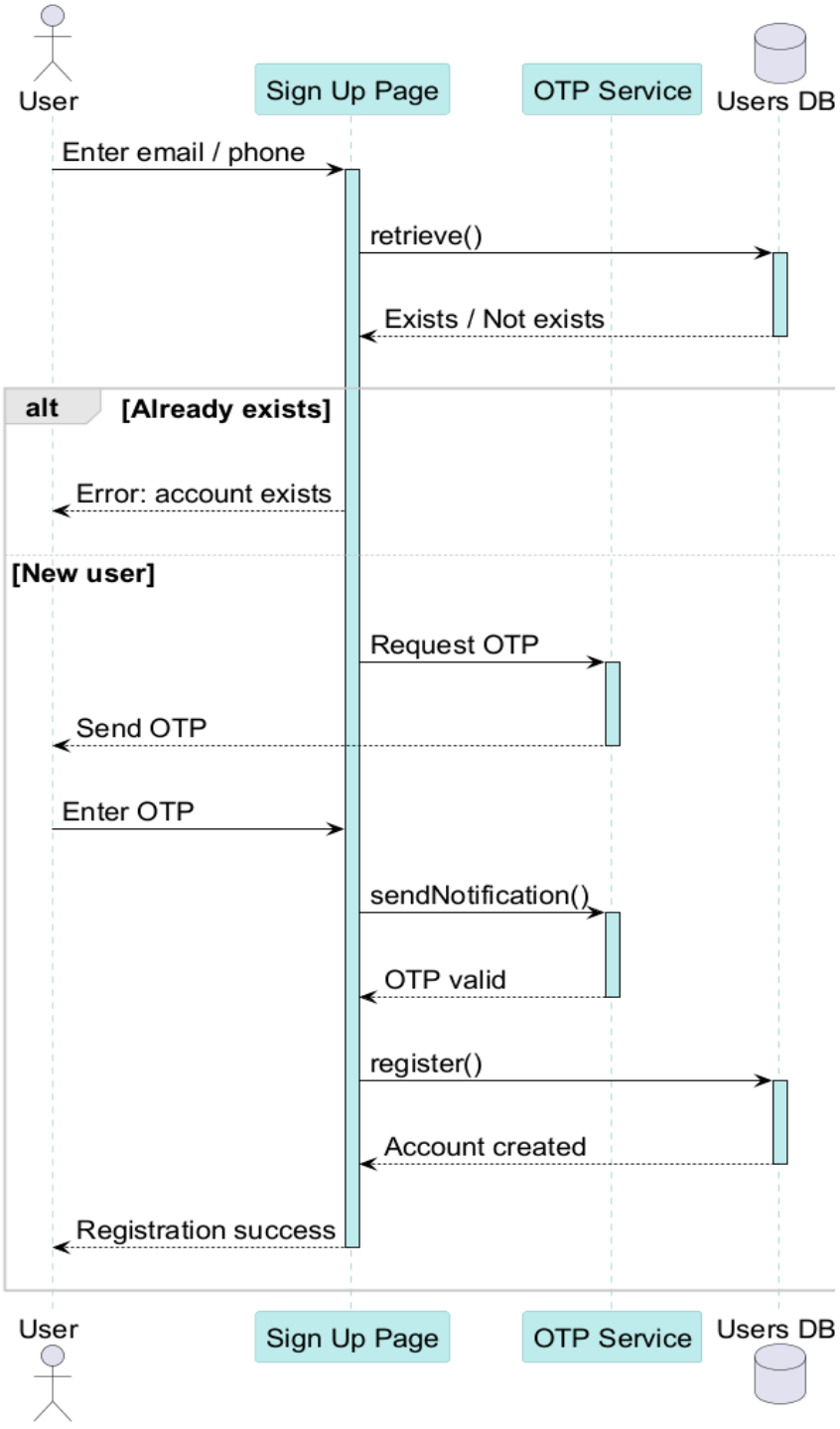


This class diagram represents the route planning subsystem of Travelly. The user-selected destinations are modeled using the Location class and stored as part of a RoutePlanRequest. The system generates an optimized route stored in RoutePlanResult. The LocationServiceClient interfaces with external maps/location APIs to obtain suggestions and distance matrices. The RouteOptimizer performs route computation, while the RoutePlanningManager coordinates request creation, optimization execution, and

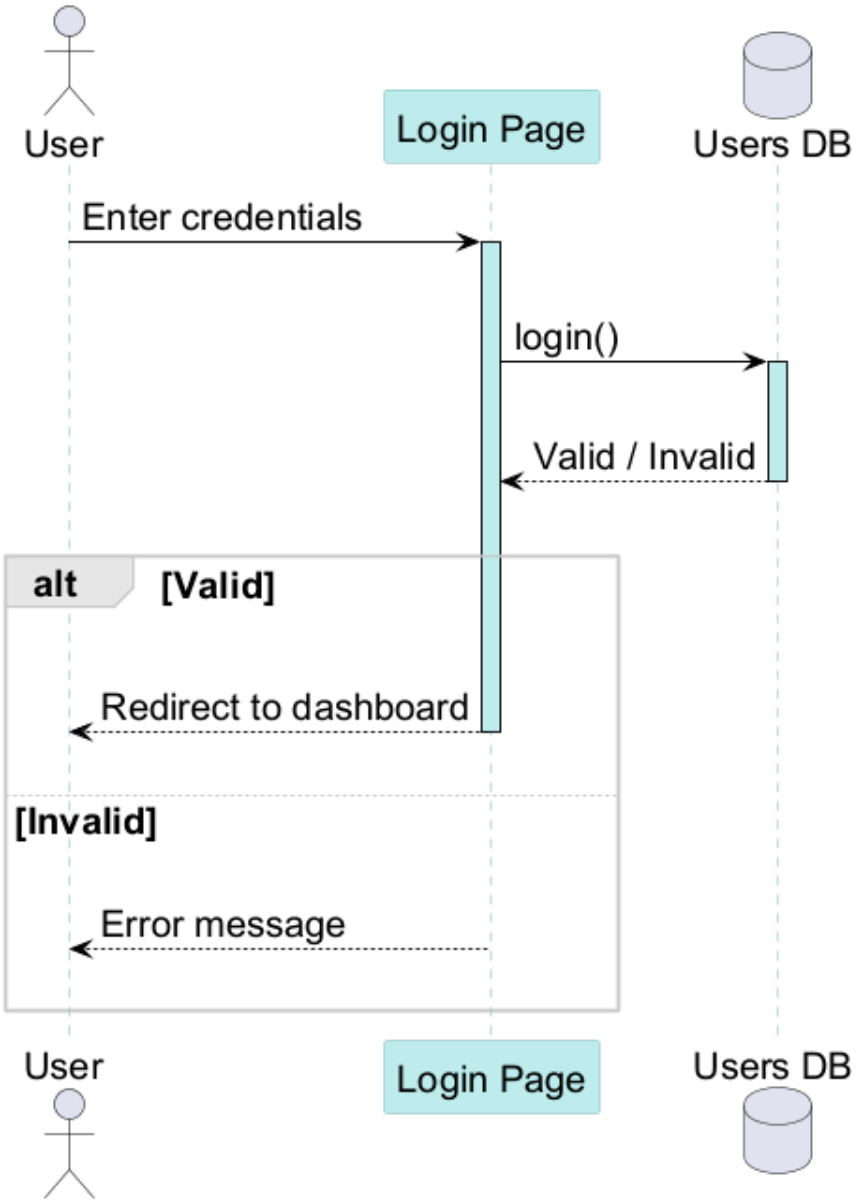
retrieval of stored route plans. This subsystem fulfills the Smart Route Planner feature defined in the SRS.

3.4 Sequence Diagrams

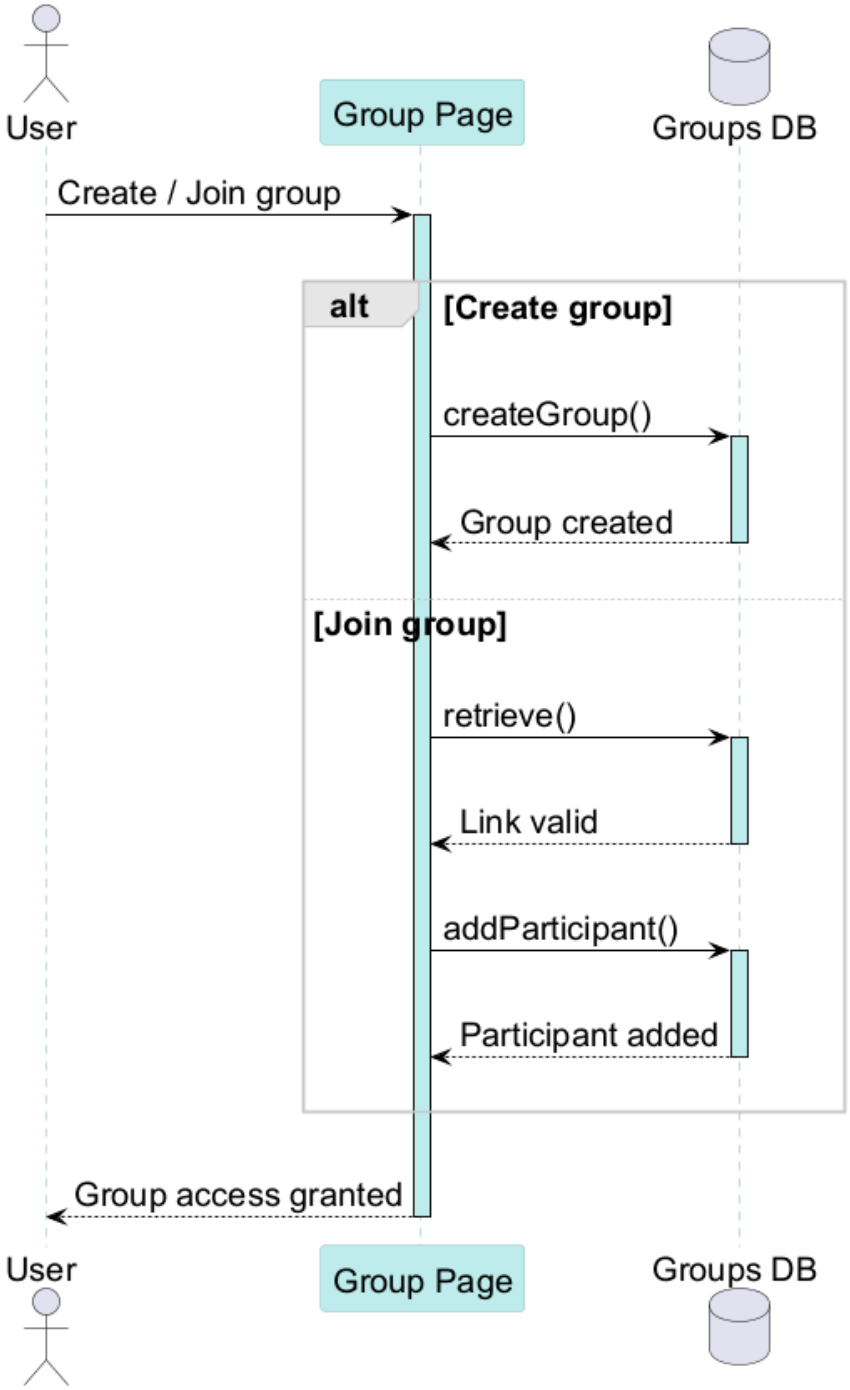
3.4.1 Signup Page



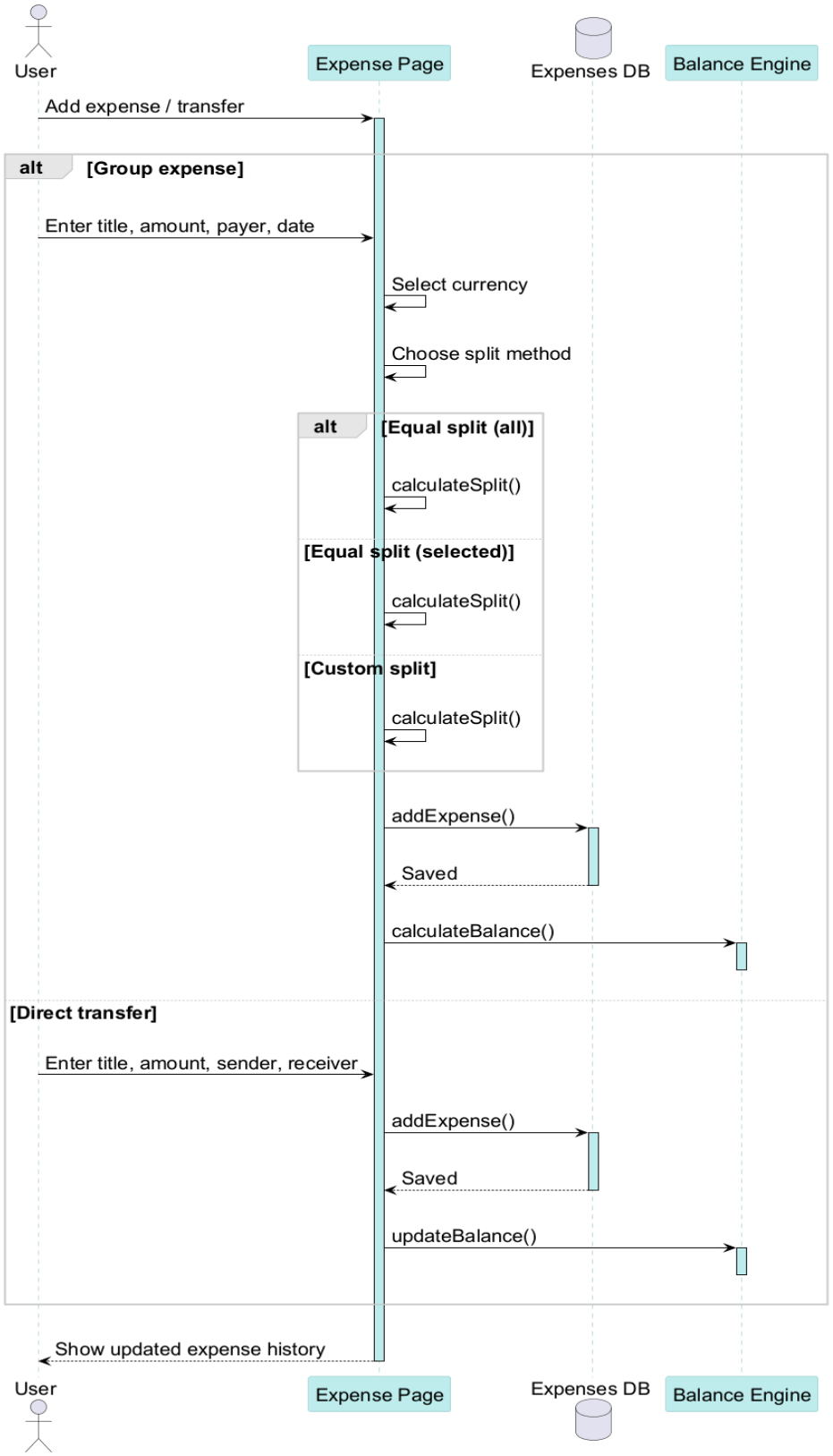
3.4.2 Login Page :



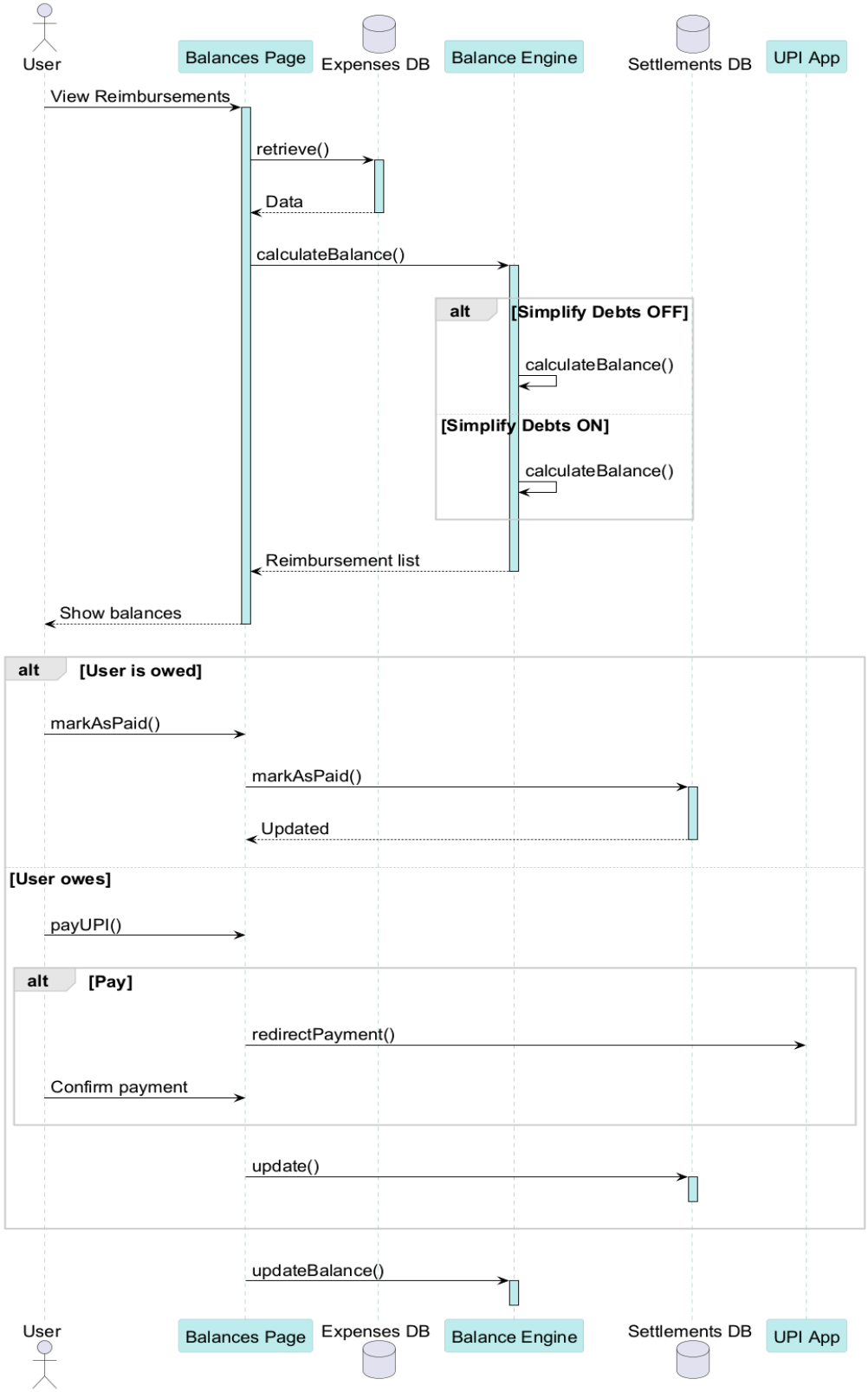
3.4.3 Group Page :



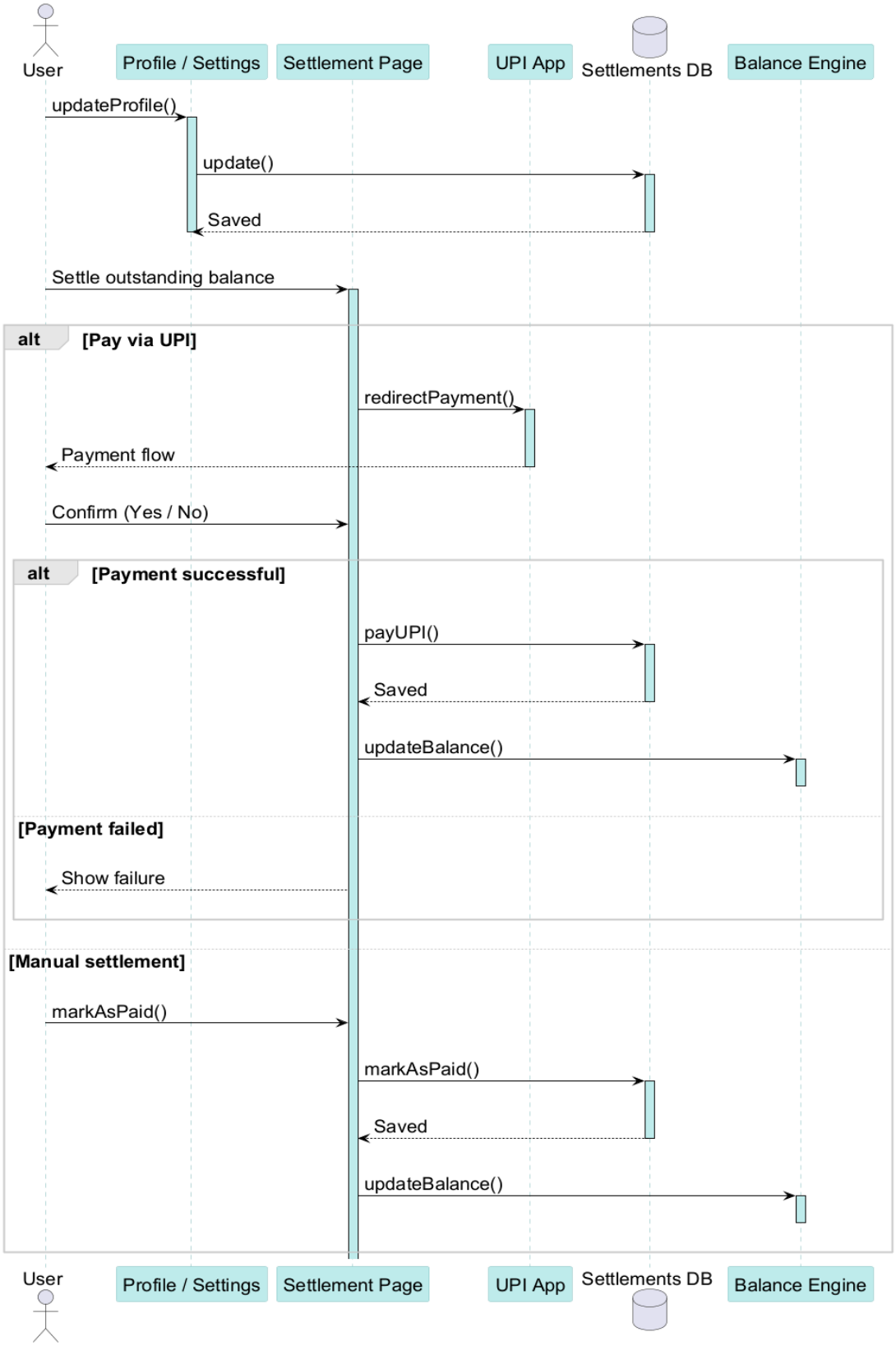
3.4.4 Expense Page :


User
Expense Page
Expenses DB
Balance Engine
Add expense / transfer
alt
[Group expense]
Enter title, amount, payer, date
Select currency
Choose split method
alt
[Equal split (all)]
calculateSplit()
[Equal split (selected)]
calculateSplit()
[Custom split]
calculateSplit()
addExpense()
Saved
calculateBalance()
[Direct transfer]
Enter title, amount, sender, receiver
addExpense()
Saved
updateBalance()
Show updated expense history
User
Expense Page
Expenses DB
Balance Engine

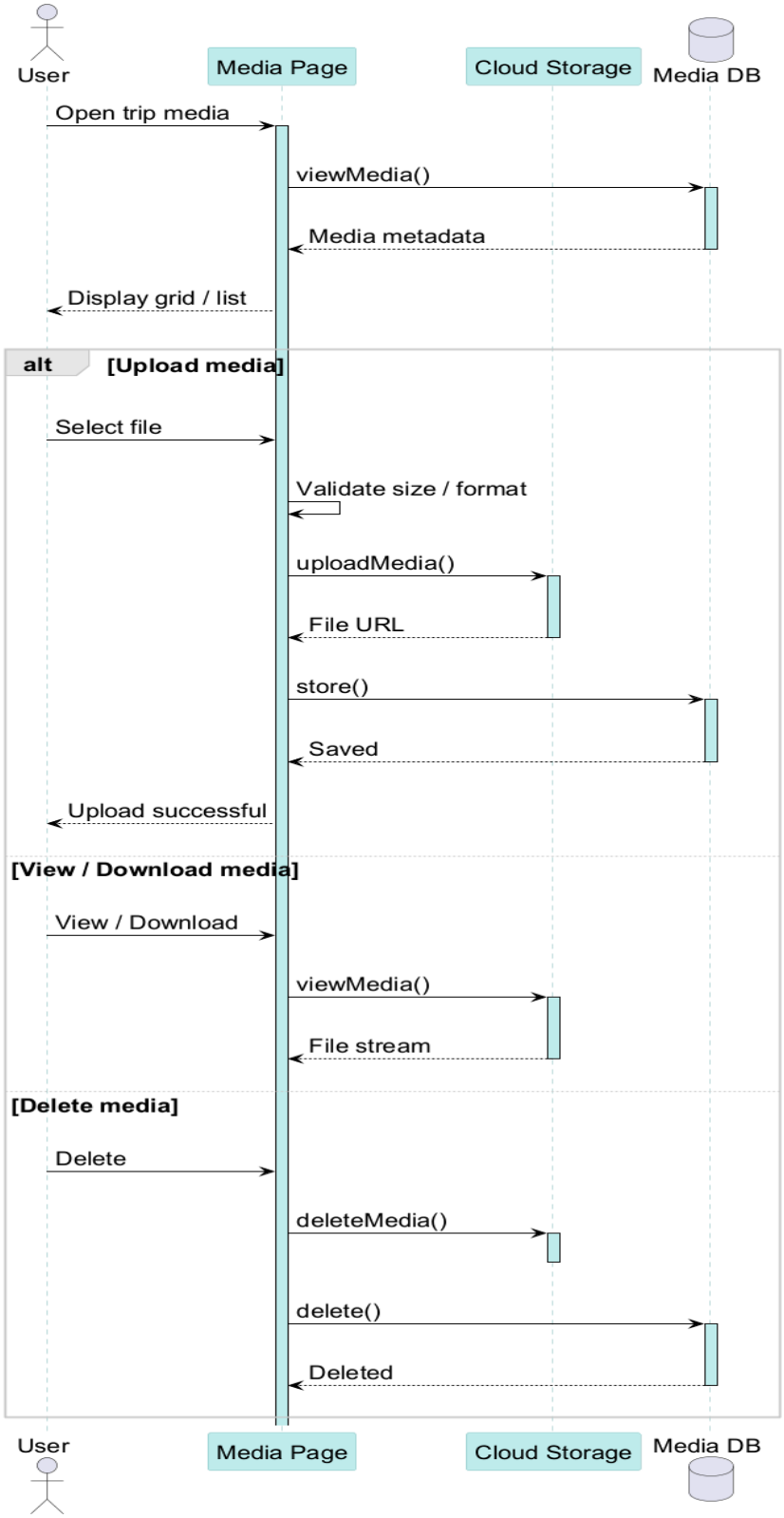
3.4.5 Balances Page:



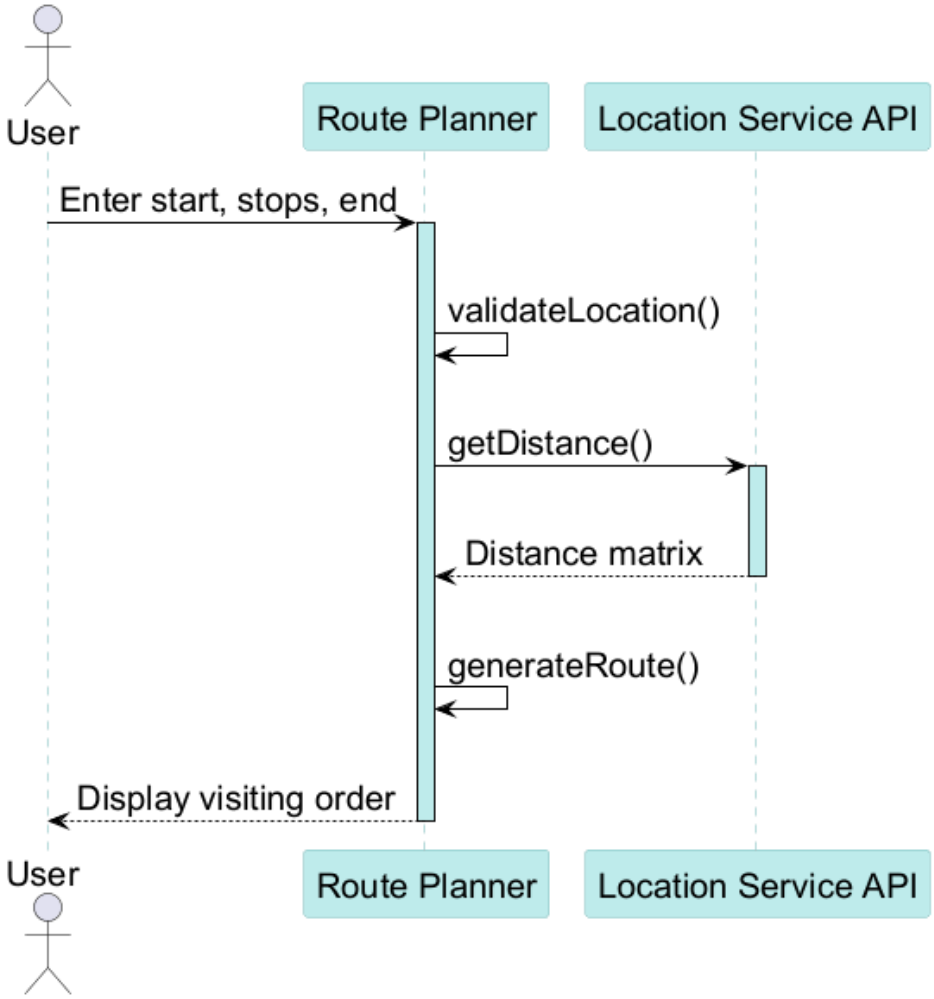
3.4.6 Settlement Page :



3.4.7 Media Page :

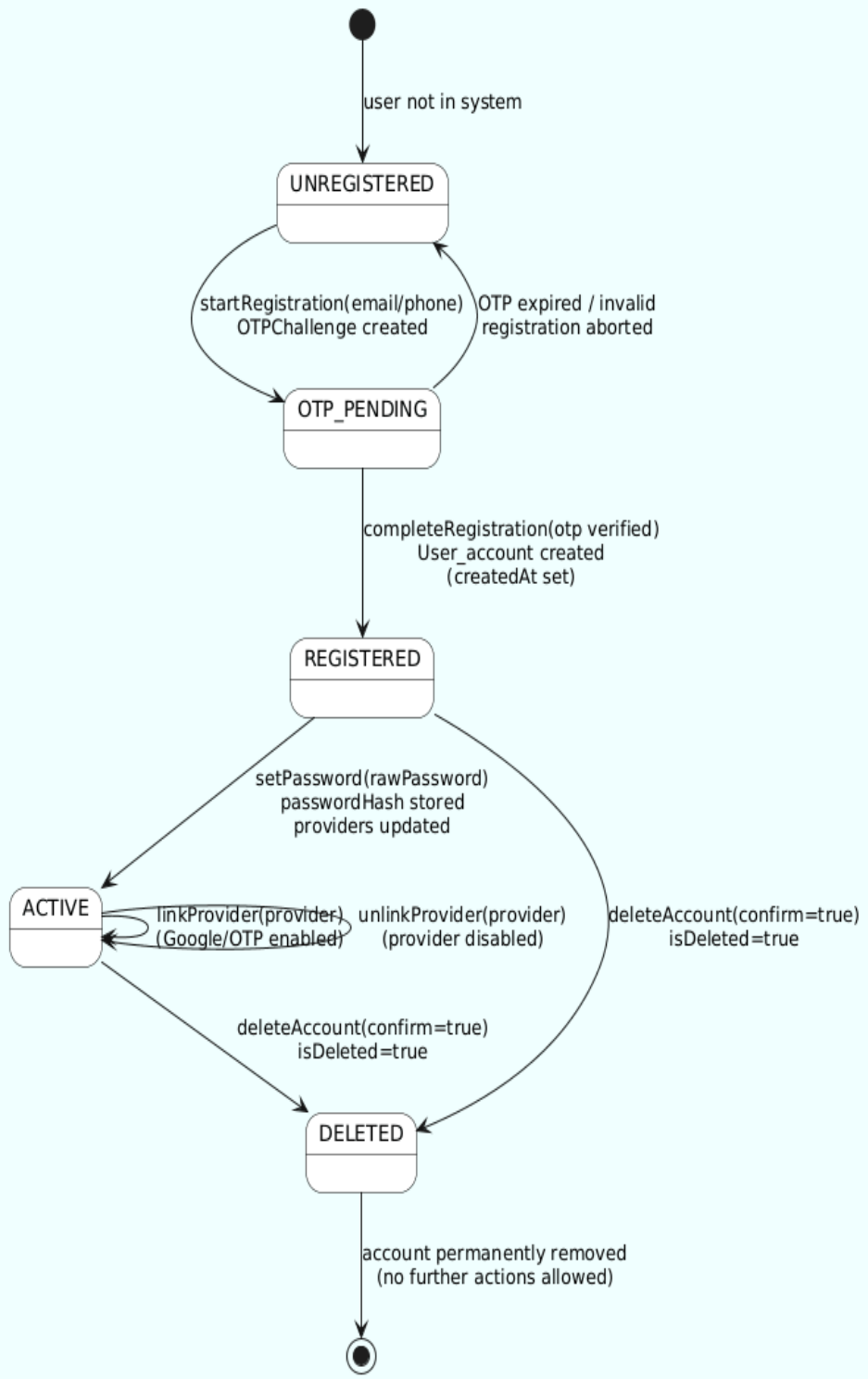


3.4.8 Route Planner :

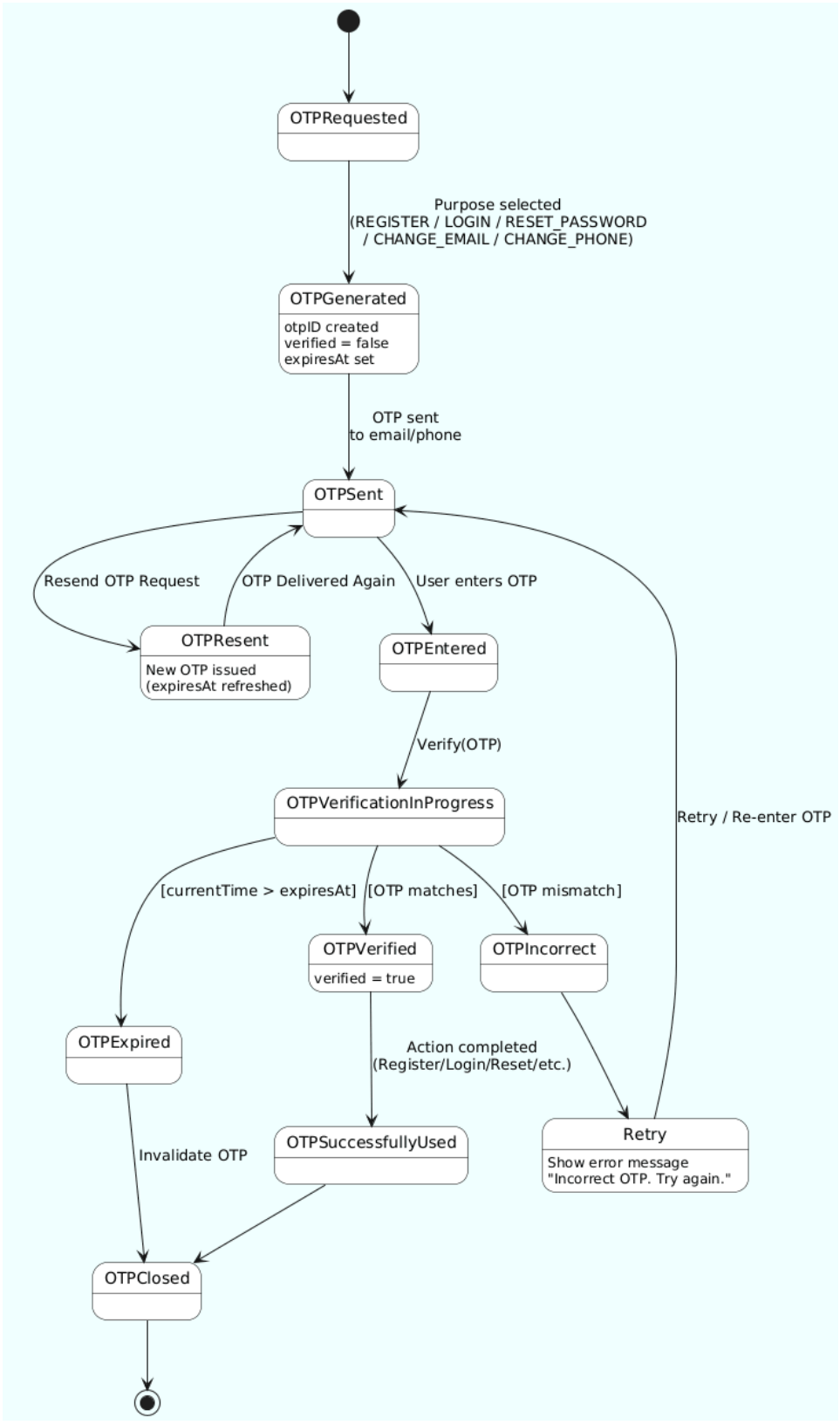


3.5 State Diagrams

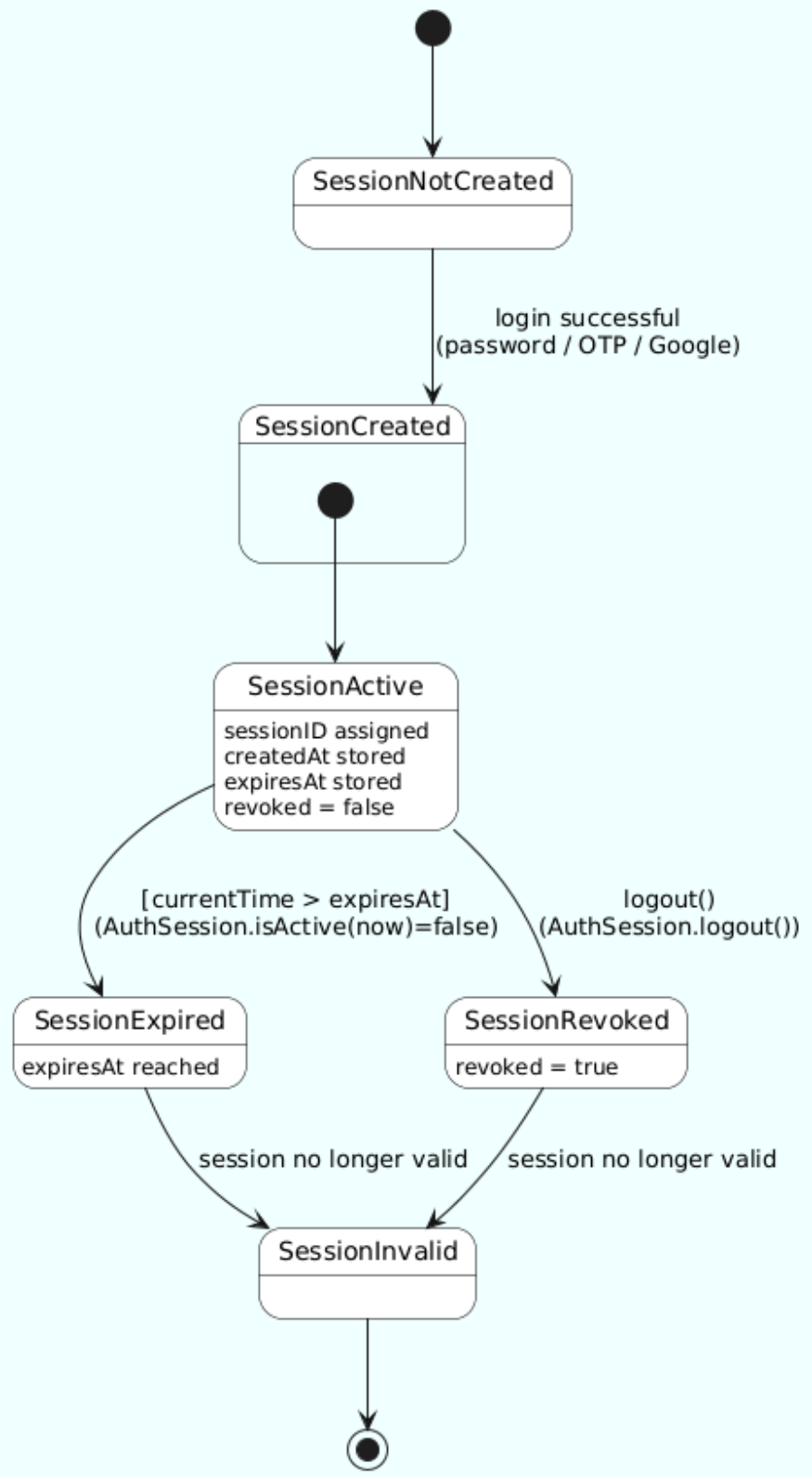
3.5.1 User Account Lifecycle (User_account)



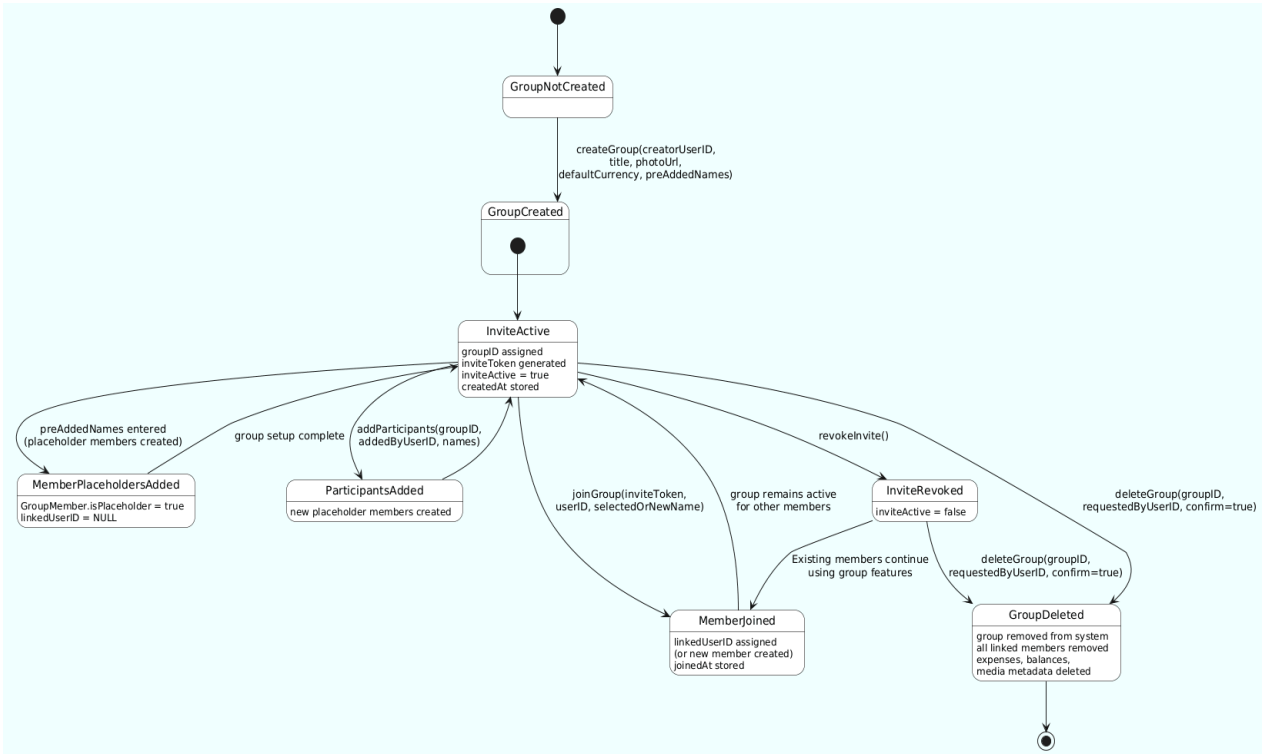
3.5.2 Authentication Session Lifecycle (AuthSession)



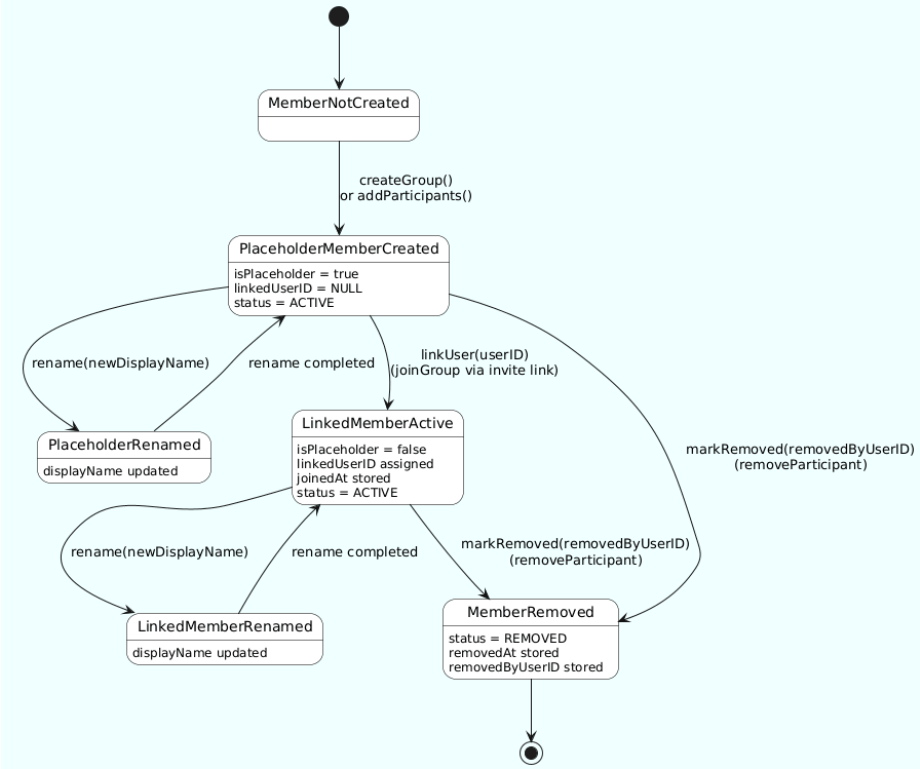
3.5.3 OTP Verification Lifecycle (OTPChallenge)



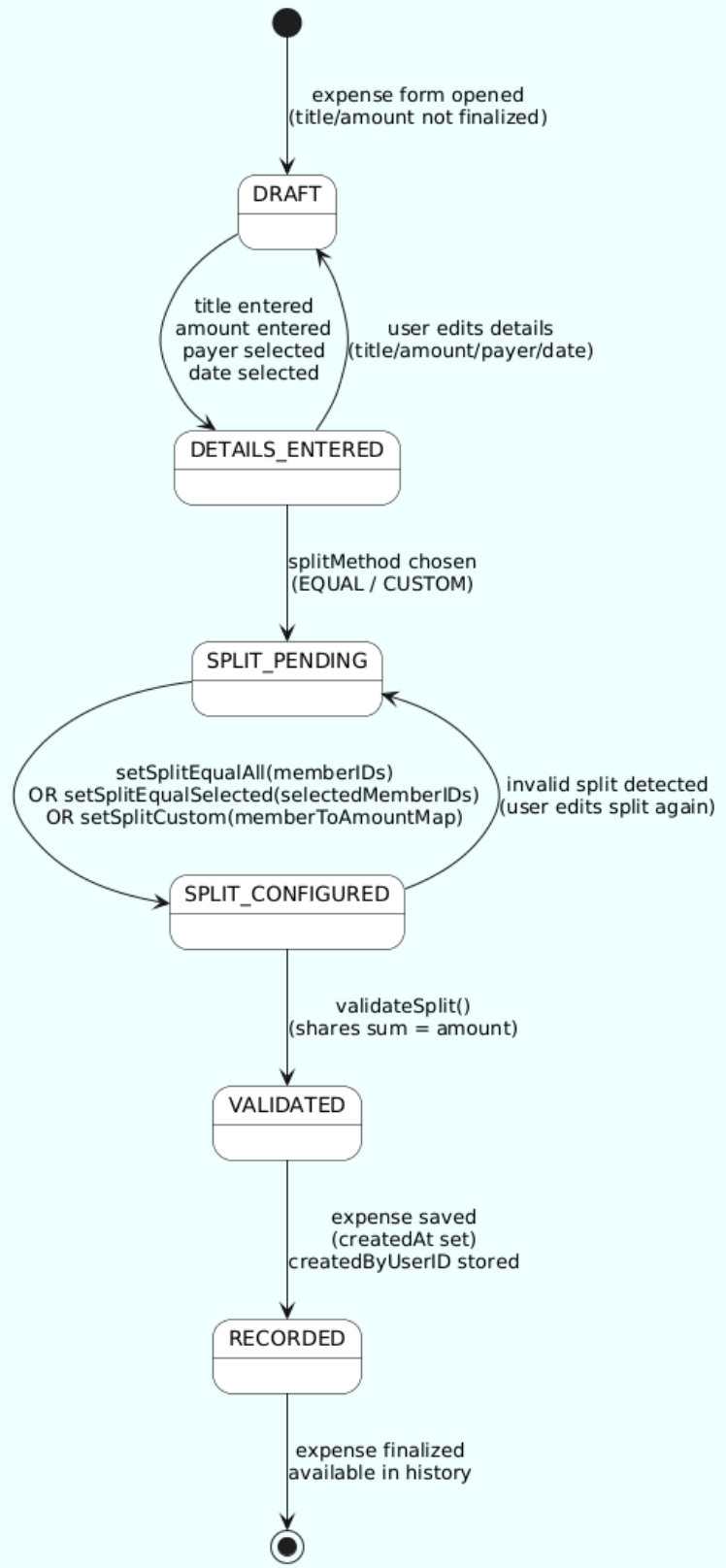
3.5.4 Trip Group Lifecycle (TripGroup)



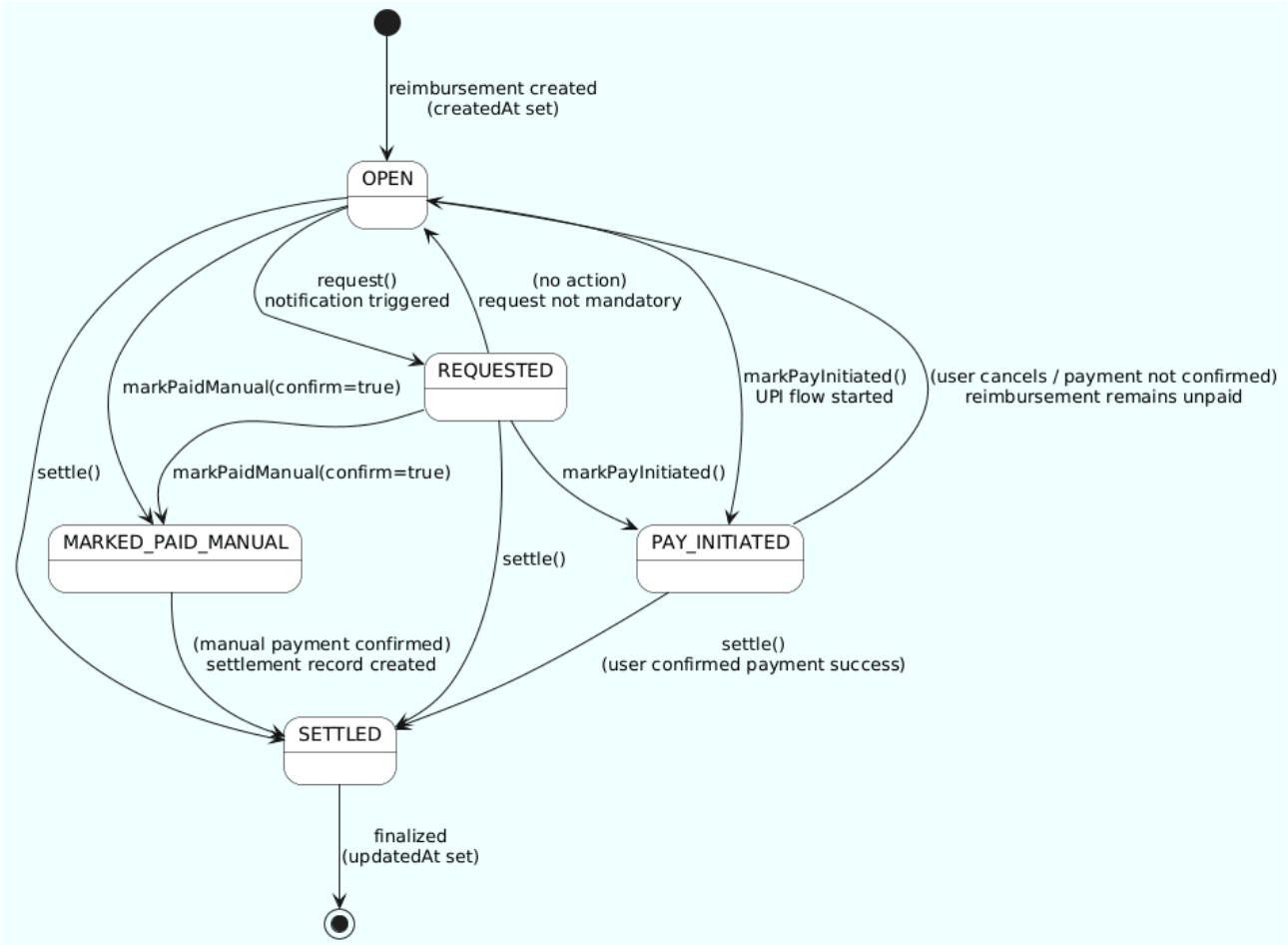
3.5.5 Group Member Lifecycle (GroupMember)



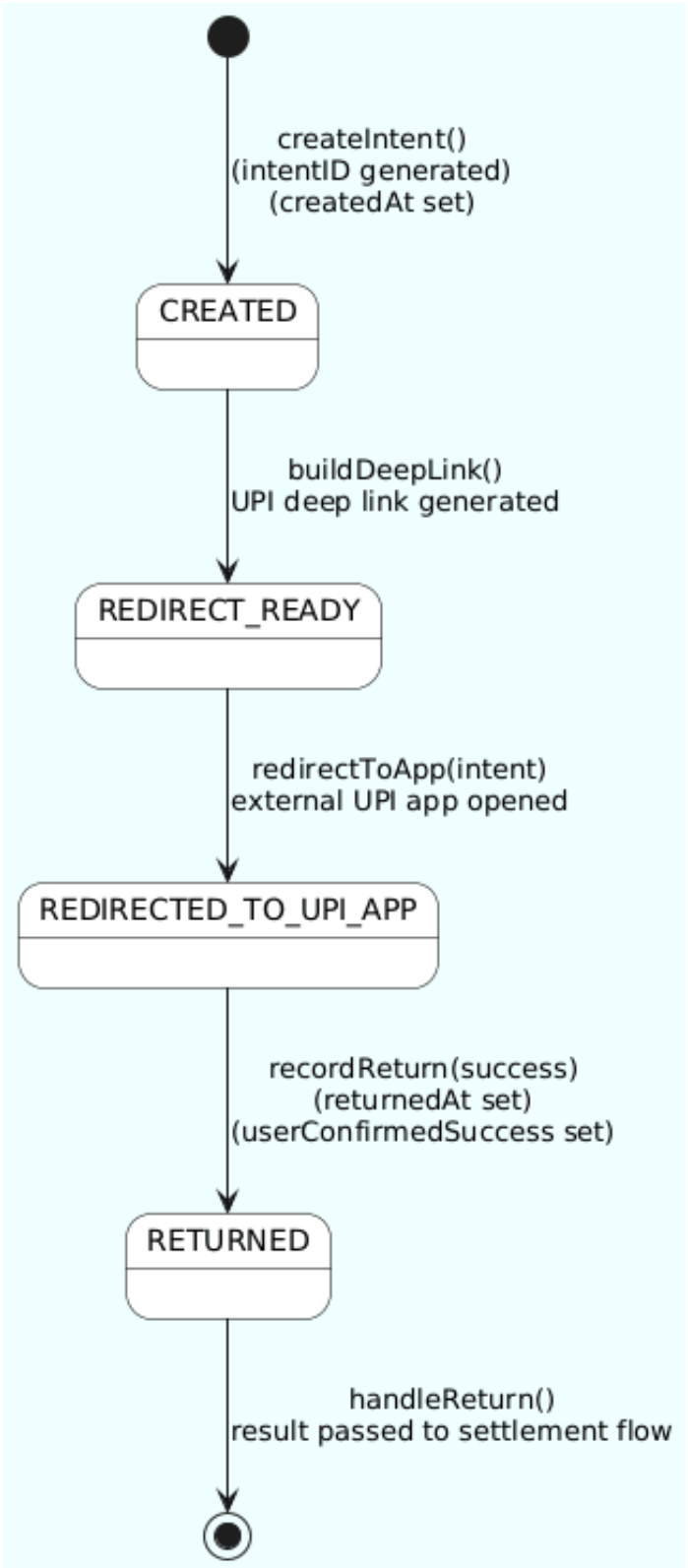
3.5.6 Expense Lifecycle (Expense)



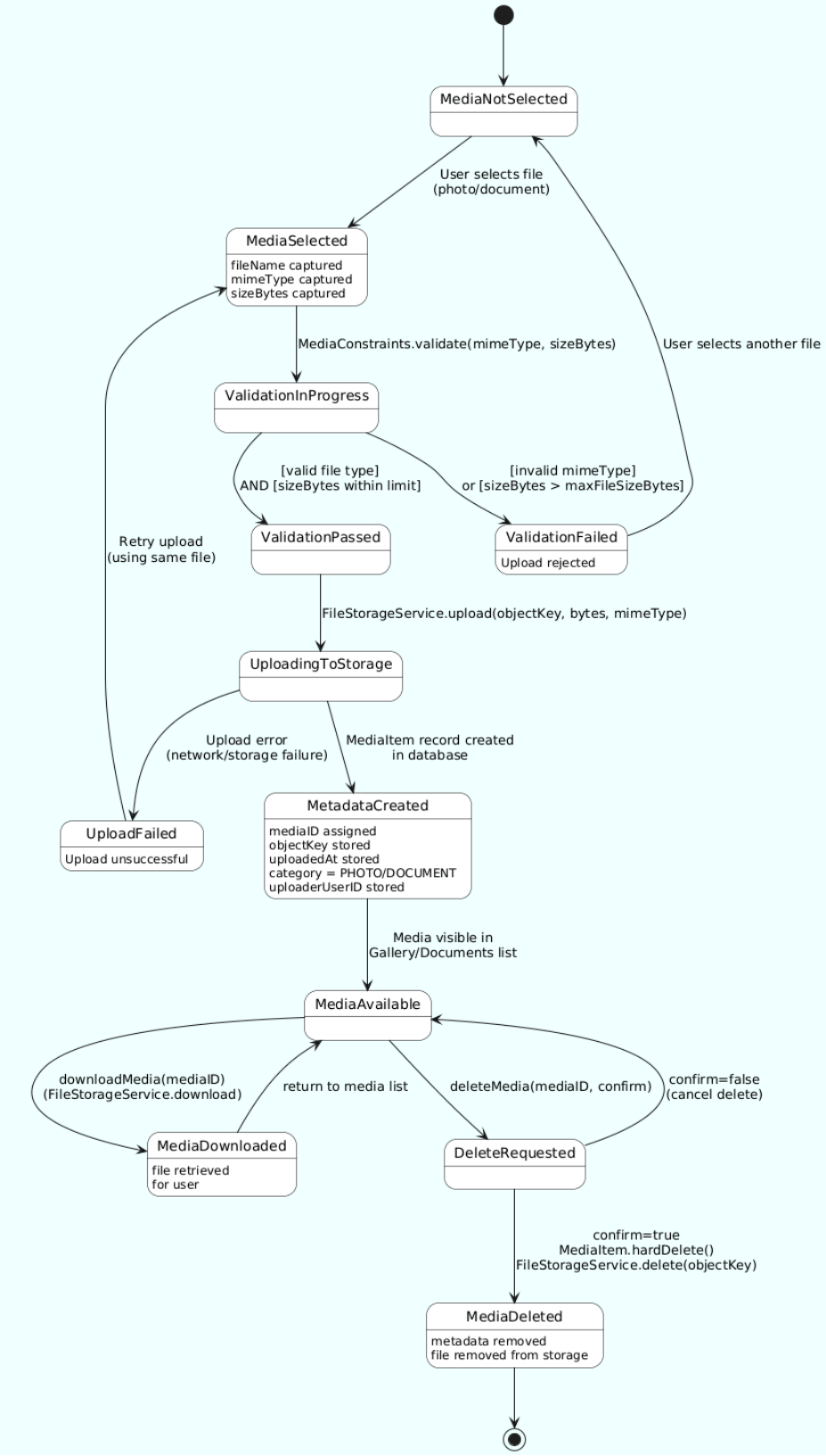
3.5.7 Reimbursement Transaction Lifecycle (ReimbursementTransaction)



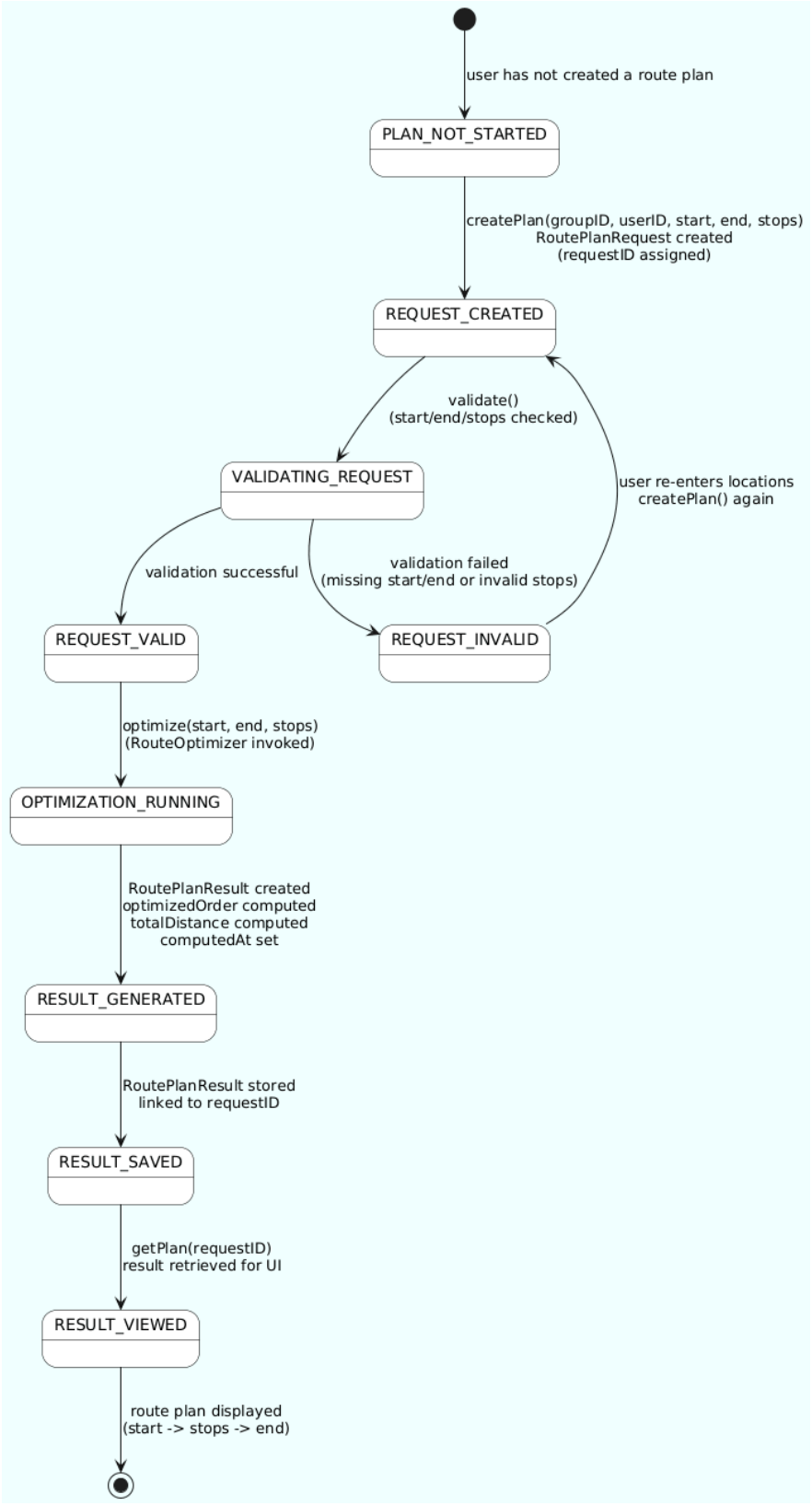
3.5.8 UPI Payment Intent Lifecycle (UpiPaymentIntent)



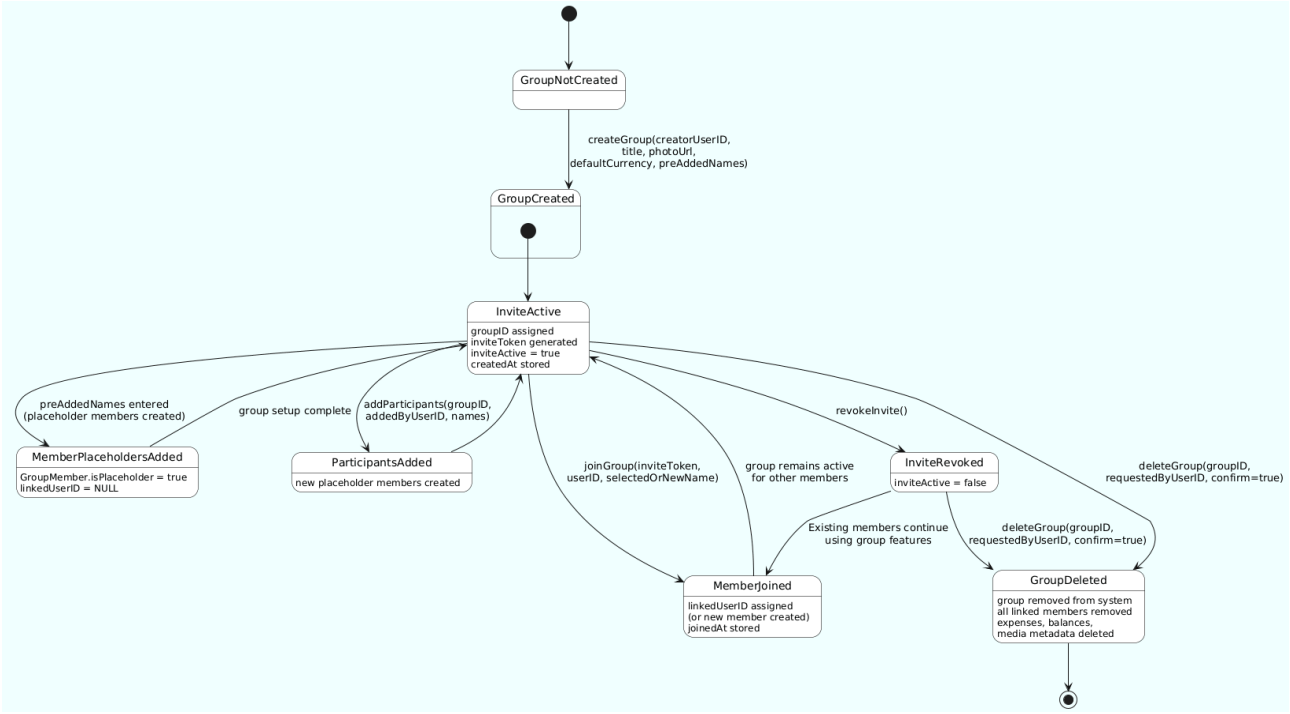
3.5.9 Media Upload Lifecycle (MediaItem)



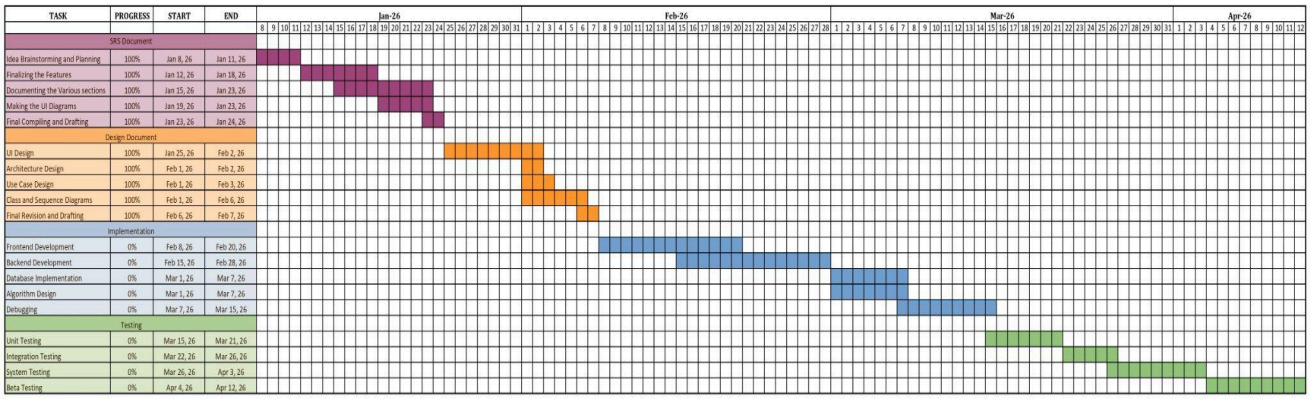
3.5.10 Routeplanning Lifecycle (RoutePlan Request + RoutePlan Result)



3.5.10 Profile Update Lifecycle (UserProfile)



4. Project Plan



TASK	PROGRESS	START	END
SRS Document			
Idea Brainstorming and Planning	100%	Jan 8, 26	Jan 11, 26
Finalizing the Features	100%	Jan 12, 26	Jan 18, 26
Documenting the Various sections	100%	Jan 15, 26	Jan 23, 26
Making the UI Diagrams	100%	Jan 19, 26	Jan 23, 26
Final Compiling and Drafting	100%	Jan 23, 26	Jan 24, 26
Design Document			
UI Design	100%	Jan 25, 26	Feb 2, 26
Architecture Design	100%	Feb 1, 26	Feb 2, 26
Use Case Design	100%	Feb 1, 26	Feb 3, 26
Class and Sequence Diagrams	100%	Feb 1, 26	Feb 6, 26
Final Revision and Drafting	100%	Feb 6, 26	Feb 7, 26
Implementation			
Frontend Development	0%	Feb 8, 26	Feb 20, 26
Backend Development	0%	Feb 15, 26	Feb 28, 26
Database Implementation	0%	Mar 1, 26	Mar 7, 26
Algorithm Design	0%	Mar 1, 26	Mar 7, 26
Debugging	0%	Mar 7, 26	Mar 15, 26
Testing			
Unit Testing	0%	Mar 15, 26	Mar 21, 26
Integration Testing	0%	Mar 22, 26	Mar 26, 26
System Testing	0%	Mar 26, 26	Apr 3, 26
Beta Testing	0%	Apr 4, 26	Apr 12, 26

TEAM MEMBERS & RESPONSIBILITIES	
MEMBERS	**TASK**
Ayush Jha	Full Stack Implementation, Code Improvement, Alpha Testing
Khushi Khandelwal	Frontend Development, Unit Testing, Addressing Feedback
Raghav Madan	Backend Development, Integration Testing, Code Improvement
Pranjali Singh	Full Stack Implementation, System Testing, Beta Testing
Ronit Kumar	Backend Development, Alpha Testing, Code Improvement
Rigved Koli	Frontend Development, Unit Testing, UI Refinement
Rushabh Pandya	Backend Development, Integration Testing, Alpha Testing
Sanket Agarwal	Full Stack Implementation, Code Improvement, Unit Testing
Tanmay Roy	Frontend Development, System Testing, Addressing Feedback
Mohd Sarim Ali	Backend Development, Manual for Beta Testing, Beta Testing

Appendix A - Group Log

Date	Timing	Duration	Venue	Description
09/01/2026	14:00-17:00	3 hours	RM Ground Floor	Discussed about the possible app ideas which included but were not limited to Smart Classroom app, Trip Planning and payment settlement app and mental wellness application.
11/01/2026	15:00-18:00	3 hours	Hall 5 Reading Room	Finalised the idea about the project and submitted the initial form mentioning a very surface level overview of what we were planning to do.
18/01/2026	9:00 -11:00	2 hours	Hall 5 Reading Room	Discussed the progress on SRS document, the UI designs to be used in the SRS and divided the remainder of the work between 2 teams one for UI and one for description.
22/01/2026	16:00-16:30	30 mins	RM Ground Floor	Discussed our doubts with the TA.
24/01/2026	17:00-18:00	1 hour	Hall 5 Reading Room	Finalised and Submitted SRS
02/02/2026	17:00-19:00	2 hours	RM Ground Floor	Discussed the project plan with team members and assigned individual responsibilities.
06/02/2026	14:30-17:30	3 hours	Hall 5 Reading Room	Finalised the design document.